

rubrapack Manual

Building and signing MSI and MSIX packages - and how the formats work

Version 0.7.0

<https://github.com/rubidus-api/rubrapack>
<https://rubidus-api.github.io/rubrapack/>

The English edition is the original.
MIT License.

Contents

Part I - Tutorial

1. Before you start	2
1.1 What an installer does	2
1.2 What you write, what rubrapack makes	2
1.3 Getting rubrapack	3
1.4 A terminal, for those who have not used one	3
1.5 The folder you work in	3
1.6 Words used in this tutorial	4
2. A first installer: one program	5
2.1 The source	5
2.2 Building	6
2.3 Installing and removing it	6
2.4 What happened inside	6
2.5 Check it without building	7
2.6 The same, with questions instead of a text editor	7
3. Versions and upgrades	8
3.1 One source for every version	8
3.2 What Windows does with two versions	9
3.3 Version numbers	9
3.4 Two identifiers	9
3.5 Check the upgrade before users do	10
3.6 Earlier versions made by another tool	10
4. More files and folders	11
4.1 The folder	11
4.2 The source	11
4.3 Folders: [dir.ID]	12
4.4 One file: [file.ID]	13
4.5 Many files: [files.ID]	13
4.6 An empty folder: [folder.ID]	13
4.7 Cleaning up: [remove.ID]	13
4.8 A second copy: [copy.ID]	13
4.9 Two more keys you may meet	14
4.10 What happened inside	14
5. Shortcuts, icons and the Installed apps entry	15
5.1 The source	15
5.2 Shortcuts: [shortcut.ID]	16
5.3 The Installed apps entry: [arp]	16
5.4 A name in another script	16
5.5 Try it	16
5.6 What happened inside	17
6. Dialogs, a license, and starting the program	18
6.1 The source	18
6.2 Choosing a set of pages: ui	19
6.3 The license: license	19
6.4 The install folder	19
6.5 A picture at the top: banner	19
6.6 Your own words: [ui-text.ID]	19
6.7 Starting the program at the end: launch	20
6.8 Try it	20

6.9 What happened inside	20
7. Several languages	21
7.1 The source	21
7.2 Adding a language: languages	22
7.3 A license per language: license-xx	22
7.4 A text per language: text-xx	22
7.5 Languages other than English and Korean	22
7.6 What is not translated	22
7.7 Try it	22
7.8 What happened inside	23
8. Optional parts: features and the user's choices	24
8.1 Features	24
8.2 The source	24
8.3 Declaring features: [feature.ID]	25
8.4 Putting files in features: feature	26
8.5 What the user sees	26
8.6 Choosing from the command line	26
8.7 Changing the choice later	27
8.8 What happened inside	27
9. Conditions and searches	28
9.1 Properties	28
9.2 The source	28
9.3 Your own property: [property.NAME]	29
9.4 Installing something only when: when	30
9.5 Refusing to install: [require.ID]	30
9.6 Looking around first: [search.ID]	30
9.7 Remembering the install folder	30
9.8 What happened inside	30
10. Who it installs for, and a guarded folder	32
10.1 Three scopes	32
10.2 The source	32
10.3 A dual package	33
10.4 A guarded install folder: guard	33
10.5 Try it	33
10.6 What happened inside	34
11. The registry, environment variables, INI files, file types and links	35
11.1 The registry in two minutes	35
11.2 The source	35
11.3 Registry values: [registry.ID]	37
11.4 Environment variables: [env.ID]	38
11.5 INI files: [ini.ID]	38
11.6 File types: [assoc.ID]	38
11.7 Links: [protocol.ID]	39
11.8 Try it	39
11.9 What happened inside	39
12. Services, fonts, permissions	40
12.1 The source	40
12.2 A service: [service.ID]	41
12.3 A font: [font.ID]	41
12.4 Access rights: [permission.ID]	41
12.5 Restarting: reboot	42
12.6 Try it	42

12.7 What happened inside	42
13. Running your program during installation	43
13.1 Why not just run a program?	43
13.2 The source	43
13.3 [action.ID]	43
13.4 What happens when	44
13.5 Try it	44
13.6 What happened inside	44
14. Your own dialog pages	46
14.1 The source	46
14.2 A page: [dialog.ID]	48
14.3 Where things go on the page	49
14.4 Controls: [dialog-control.ID]	49
14.5 Every value needs a default: [property.NAME]	49
14.6 Try it	49
14.7 What happened inside	50
15. Cabinets, compression, big packages, and several architectures	51
15.1 The folder	51
15.2 The source	51
15.3 Cabinets and compression	52
15.4 Several architectures: --arch	52
15.5 The same bytes every time: --reproducible	52
15.6 Names from a Mac: --nfc	53
15.7 Variables from the command line: -D	53
15.8 What happened inside	53
16. Signing and timestamps	54
16.1 What a signature is	54
16.2 Signing with a key file	54
16.3 Timestamps: signatures that outlive the certificate	55
16.4 A key in the Windows certificate store	55
16.5 A hardware token: PKCS#11	55
16.6 Checking a signature: verify	56
16.7 External cabinets are not covered	56
16.8 What happened inside	56
17. MSIX packages and bundles	57
17.1 MSI or MSIX?	57
17.2 The folder	57
17.3 The source	58
17.4 The package's identity: [msix]	59
17.5 The application: [msix-app.ID]	59
17.6 Extras: [msix-extension.ID]	59
17.7 What the other tables become	59
17.8 Building for testing: --unsigned-test	60
17.9 Signing it	60
17.10 All architectures in one file: a bundle	60
17.11 Compression	61
17.12 What happened inside	61
18. Checking and looking inside	62
18.1 Checking a source: lint	62
18.2 Checking a package: lint <file>	63
18.3 Looking at the tables: inspect	63
18.4 Files and streams	64

18.5 Unpacking without installing: extract	65
18.6 Signatures: verify	65
18.7 MSIX packages	65
18.8 Faster test builds: --compress none	65
18.9 GUIDs: guid	65
18.10 Help on the command line	66
19. Starting fast and automating	67
19.1 A source from answers: new	67
19.2 Changing a source: edit	68
19.3 A release script	69
19.4 On a build server	69
19.5 With an AI assistant	69
19.6 Where to go from here	70

Part II - Reference

20. Source files and the rubrapack command	72
20.1 Getting started	73
20.2 Example	75
20.3 The TOML subset	76
20.4 Tables	76
20.5 Command line	87
20.6 Diagnostic codes	90

Part III - Background knowledge

21. Bits, bytes and hexadecimal	93
21.1 Bits	93
21.2 Binary numbers	93
21.3 Bytes	94
21.4 Hexadecimal	94
21.5 Reading a hex dump	94
21.6 Units: KB, MB and KiB, MiB	95
21.7 Where this is used	95
22. Numbers in files, and bit flags	96
22.1 One, two, four, eight bytes	96
22.2 Byte order: little-endian	96
22.3 Negative numbers: two's complement	97
22.4 Bit flags	97
22.5 Testing and setting a bit: AND, OR	98
22.6 Where this is used	98
23. Text: characters, code points and encodings	99
23.1 Characters and code points	99
23.2 ASCII	99
23.3 Code pages: one byte, 256 characters	100
23.4 UTF-8	100
23.5 UTF-16	101
23.6 The byte order mark	101
23.7 Unicode normalization	101
23.8 When text looks wrong	102
23.9 Where this is used	102
24. GUIDs and hashes	103
24.1 GUIDs	103
24.2 Hashes	104

24.3 GUIDs from hashes: guid --from	105
24.4 Where this is used	105
25. Compression	106
25.1 Why data can be made smaller	106
25.2 LZ77: "copy from earlier"	106
25.3 Huffman coding: short codes for common symbols	107
25.4 Deflate, bit by bit	107
25.5 MSZIP: deflate in a cabinet	108
25.6 Deflate in an MSIX	108
25.7 Where this is used	108
26. Files inside a file	109
26.1 How a disk stores files	109
26.2 An MSI is a file system in a file	109
26.3 An MSIX is an archive	110
26.4 Why two designs?	111
26.5 Where this is used	111
27. Tables, keys and references	112
27.1 Tables, rows and columns	112
27.2 Primary keys: what makes a row unique	112
27.3 Foreign keys: rows that point at rows	113
27.4 Keys you give, keys rubrapack makes	113
27.5 Storing strings once: the string pool	113
27.6 Tables about tables	114
27.7 The IDT text format	114
27.8 Where this is used	114
28. Keys, certificates and signatures	115
28.1 Two keys instead of one	115
28.2 Signing and checking	115
28.3 Certificates: whose key is this?	116
28.4 Chains and trust	116
28.5 Timestamps	117
28.6 Where keys live: files and tokens	117
28.7 Where this is used	117

Part IV - File formats

29. Package formats for implementers	119
29.1 How to read the facts	120
29.2 Conventions	120
29.3 The pipeline, in one picture	120
30. Compound File Binary (CFB)	121
30.1 Sectors	121
30.2 Header (first 512 bytes) [spec]	121
30.3 FAT, DIFAT, mini FAT [spec]	122
30.4 Directory [spec]	122
30.5 A simple, deterministic writer	123
30.6 Reading safely	123
30.7 Worked example: the tutorial's hello.msi	123
31. The MSI database inside the compound file	125
31.1 Streams	125
31.2 The string pool: _StringPool and _StringData	126
31.3 System tables	126
31.4 Table streams	127

31.5 IDT archive files (what MsiDatabaseExport writes)	127
31.6 Reading safely	128
31.7 Worked example: the tutorial's hello.msi	128
32. Summary information	130
32.1 Layout [spec] [observed]	130
32.2 Properties an installer package sets	130
32.3 The code page trap - keep summary strings ASCII [observed]	131
32.4 Worked example: the tutorial's hello.msi	131
33. The smallest package that installs, repairs, upgrades and uninstalls	133
33.1 Tables and columns	133
33.2 Property	134
33.3 Directory	134
33.4 Components, features, files	134
33.5 Empty folders	135
33.6 Media and the cabinet	135
33.7 Major upgrade and downgrade refusal	135
33.8 Running an installed program to register and unregister	135
33.9 Registry values	136
33.10 Shortcuts	136
33.11 Removing files and duplicating them	136
33.12 Environment variables	137
33.13 INI files	137
33.14 Searches and launch conditions	137
33.15 Services, fonts, permissions	138
33.16 Cabinets, administrative images, advertisement	138
33.17 Per-user and dual packages	138
33.18 Files in use	138
33.19 Sequences	138
33.20 Dialogs	139
33.21 What this recipe does not cover yet	141
34. Program files: machine type and version	142
34.1 Finding the headers [spec]	142
34.2 The version resource [spec]	142
34.3 What goes into the package [observed]	142
34.4 Reading safely	143
35. Cabinets, MSZIP and deflate	144
35.1 Cabinet layout [spec: MS-CAB]	144
35.2 MSZIP [spec: MS-MCI]	144
35.3 A deflate encoder in brief [spec: RFC 1951]	145
35.4 Worked example: the tutorial's hello.msi	145
36. Deterministic identities and reproducible packages	146
36.1 Name-based GUIDs (UUID version 8) [spec: RFC 9562]	146
36.2 Keys	146
36.3 Reproducible packages	147
37. Authenticode signatures	148
37.1 Where the signature lives in a PE file	148
37.2 The PE digest	148
37.3 The CMS SignedData	149
37.4 MSI packages	149
37.5 Timestamps	150
37.6 ECDSA	150
37.7 What Windows reports	150

37.8 Worked example: the tutorial's hello.msi	150
38. Checking your output against Windows	152
38.1 The database opens and decodes	152
38.2 The package installs	152
38.3 Checks before writing	153
38.4 Microsoft's ICE rules [observed]	153
38.5 Things that look fine but are not	154
39. MSIX packages	155
39.1 The ZIP archive	155
39.2 The block map (AppxBlockMap.xml)	155
39.3 [Content_Types].xml	156
39.4 The manifest (AppxManifest.xml)	156
39.5 The virtual registry (Registry.dat, User.dat)	156
39.6 The virtual file system (VFS\...)	157
39.7 Extensions	157
39.8 Bundles (.msixbundle)	158
39.9 Signatures (AppxSignature.p7x)	159
39.10 Installing an unsigned package	159
39.11 Worked example: the tutorial's hello.msix	160
40. Registry hive files (REGF)	161
40.1 Layout	161
40.2 The base block	161
40.3 Keys (nk)	162
40.4 Subkey lists (lh, ri)	162
40.5 Values (vk) and value lists	162
40.6 The security cell (sk)	162
40.7 Reproducible hives	162

Part I

Tutorial

1 Before you start

In this chapter

1.1	What an installer does	2
1.2	What you write, what rubrapack makes	2
1.3	Getting rubrapack	3
1.4	A terminal, for those who have not used one	3
1.5	The folder you work in	3
1.6	Words used in this tutorial	4

This part of the manual is a tutorial. It starts with the smallest installer there is - one program copied into Program Files - and adds one thing per chapter until every feature of rubrapack has been used once. Each chapter builds on the one before, so it is best read in order. Nothing is assumed beyond using a computer: where a chapter needs an idea from outside (a GUID, a hash, a registry key), it explains it on the spot or points to the Background knowledge part.

1.1 What an installer does

A program on your own computer is just files in a folder. To give it to someone else, you could send them the folder - but then they have to choose where to put it, make their own Start menu entry, and later find and delete everything by hand. An *installer* does this for them:

- it copies the files to the right place (usually C:\Program Files\<your program>);
- it creates shortcuts, registry values, file type links and whatever else the program needs;
- it registers the program in Settings > Apps > Installed apps, so it can be removed there;
- when you ship version 2, it replaces version 1 cleanly;
- if something goes wrong halfway, it puts everything back as it was.

Windows has a built-in engine for this, **Windows Installer**. You do not write a program that installs; you write a *package*, a file that describes what to install, and Windows Installer carries it out. Two package formats exist:

Format	File	Installed by	Typical use
Windows Installer	.msi	Windows Installer (msiexec), since Windows 2000	desktop programs of every kind; companies deploy MSI through their management tools
MSIX	.msix, .msixbundle	the App Installer of Windows 10 and 11	modern packaged apps, the Microsoft Store

rubrapack makes both, from the same description. Most of this tutorial is about MSI; chapter 17 turns the same example into an MSIX.

1.2 What you write, what rubrapack makes

You write one short text file, the *source*, in [TOML](#) - a simple key = value format. rubrapack reads it and writes the package:

```
hello.toml (you write this)
|
| rubrapack build hello.toml -o hello.msi
```

```

      v
hello.msi  (you give this to users)
      |
      |    double-click, or: msixec /i hello.msi
      v
C:\Program Files\Hello\hello.exe  (Windows Installer puts it there)

```

An MSI file is itself a small database with dozens of tables, identifiers and a compressed archive inside - Part IV of this book shows it byte by byte. You never have to write those tables: rubrapack works them out from the source.

1.3 Getting rubrapack

rubrapack is one program file with nothing else to install. From the releases page, <https://github.com/rubidus-api/rubrapack/releases>, download:

- on Windows: rubrapack-<version>-windows-x64.exe. Rename it to rubrapack.exe.
- on Linux: rubrapack-<version>-linux-x86_64. Rename it to rubrapack and make it executable: `chmod +x rubrapack`.

Put it in a folder that is on your PATH (so the command works from anywhere), or keep it in the folder you work in and type `.\rubrapack.exe` (Windows) or `./rubrapack` (Linux) instead of rubrapack. This tutorial writes plain rubrapack.

You can build packages on Linux, but you need Windows to install and try them. A virtual machine with Windows is enough.

1.4 A terminal, for those who have not used one

rubrapack has no window: you type commands. On Windows, press the Windows key, type terminal (or cmd), and press Enter. A window with a prompt opens, for example:

```
C:\Users\you>
```

The part before > is the *current folder*. Commands work on files in that folder unless you give a path. Three commands are all you need:

Command	What it does
<code>cd C:\work\hello</code>	go into the folder C:\work\hello
<code>dir</code> (Linux: <code>ls</code>)	list the files in the current folder
<code>mkdir dist</code>	make a folder named dist here

Try it: `rubrapack version` prints the version, and `rubrapack help` lists the commands.

```
C:\work\hello> rubrapack version
rubrapack 0.7.0 (proven_c_lib-v0.1.1)
```

1.5 The folder you work in

Every chapter uses the same layout. Make a folder for your project, and inside it a folder dist ("distribution") holding exactly the files you ship:

```

C:\work\hello\
  dist\
    hello.exe      the program - any program of your own will do
    hello.toml     the source (the next chapter writes it)

```

If you have no program at hand, any .exe works for learning; the installer does not run it.

1.6 Words used in this tutorial

Word	Meaning
package	the .msi (or .msix) file you give to users
source	the .toml file you write; rubrapack builds the package from it
install folder	where the files go on the user's computer, e.g. C:\Program Files\Hello
product	your program as Windows sees it: a name, a version, a manufacturer
upgrade code	a fixed identifier that ties every version of your product together (chapter 2)
feature	a part of the product the user may choose to install or not (chapter 8)
component	Windows Installer's unit of installation; rubrapack makes these for you

2 A first installer: one program

In this chapter

2.1	The source	5
2.2	Building	6
2.3	Installing and removing it	6
2.4	What happened inside	6
2.5	Check it without building	7
2.6	The same, with questions instead of a text editor	7

Goal: an installer that copies `hello.exe` into `C:\Program Files\Hello`, shows up in Installed apps, and removes itself cleanly.

2.1 The source

In the folder `C:\work\hello` (see the previous chapter), create a text file `hello.toml` with any text editor - Notepad will do - and write:

```
# tutorial 02: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "1.0.0"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

Line by line:

- A line starting with `#` is a comment; `rubrapack` ignores it.
- `[package]` starts a *table*: the lines after it, up to the next `[ ... ]`, belong to it. The package table says what the product is.
 - `name` is what users see: in Installed apps, in the title of the installer's windows.
 - `manufacturer` is you or your company; Installed apps shows it under the name.
 - `version` has three numbers (or four): major, minor, build. Chapter 3 is about changing it.
 - `arch` is the processor the program is built for: `x64` (almost every PC today), `x86` (32-bit programs) or `arm64`. `rubrapack` checks that `hello.exe` really is an `x64` program.
 - `upgrade-code` is a GUID - a 128-bit number written as 32 hexadecimal digits in braces - that is unique to your product. Make your own with `rubrapack guid` (below), write it here once, and **never change it**: every later version must carry the same one, or Windows will treat it as a different product and install it next to the old one.
- `[dir.INSTALLDIR]` declares a folder with the ID `INSTALLDIR`. An ID is a name you choose (letters, digits and `_`) to refer to a thing from other tables. `path` says where the folder is: `ProgramFiles/`

Hello means C:\Program Files\Hello on an English or Korean Windows alike - the known folder ProgramFiles is resolved on the user's computer.

- [file.Hello] installs one file, with the ID Hello. dir names the folder it goes into, by its ID; source is where the file is now, relative to hello.toml, written with /.

Get your own upgrade code - a new one every time you run it:

```
C:\work\hello> rubrapack guid
{F1B1ED21-3061-4A65-8D77-FBB5ACCB1642}
```

Copy it over the one in the example. (Using the example's code would make your product and every other reader's "the same product" to Windows.)

2.2 Building

```
C:\work\hello> rubrapack build hello.toml -o hello.msi
```

Nothing is printed when all is well, and hello.msi (about 28 KB) appears next to the source. If something is wrong, rubrapack says where and why and writes nothing, for example after a typo in source:

```
hello.toml:14:1: error[RP1507]: source file 'dist/hello.ex' not found
```

The numbers are the line and the column; the code in brackets (RP1507: RP15xx is "the files to install") is explained under Diagnostic codes in the Reference part. Fix the line and build again.

2.3 Installing and removing it

Copy hello.msi to a Windows computer and double-click it. Because it installs into Program Files, which belongs to all users, Windows asks for administrator permission (the User Account Control prompt). Then a small window with a progress bar appears for a moment - this package has no dialogs of its own yet (chapter 6 adds them). Afterwards:

- C:\Program Files\Hello\hello.exe exists;
- Settings > Apps > Installed apps lists **Hello**, version 1.0.0, by Example Software.

Remove it from Installed apps (the ... menu, Uninstall), or from a terminal:

```
msiexec /x hello.msi
```

The file and the folder are gone again. The same package can also be installed without any window, which is how companies deploy software (/qn means "quiet, no user interface"; run the terminal as administrator):

```
msiexec /i hello.msi /qn
msiexec /x hello.msi /qn
```

When something goes wrong on Windows, a log tells why: msiexec /i hello.msi /l*v install.log writes everything Windows Installer did into install.log.

2.4 What happened inside

rubrapack inspect shows what the package holds without installing it. --files lists the files and where they go:

```
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello C_185f8db32271fe25f561 1.2.3.4 1033
```

ProgramFiles64Folder is Windows Installer's name for 64-bit Program Files; 17920 is the size in bytes; 1.2.3.4 and 1033 (US English) are the version and language written inside `hello.exe` (Windows compares them when it replaces a file). An MSI is a database, and `inspect` can print any of its tables; the File table has one row for our one file:

```
C:\work\hello> rubrapack inspect hello.msi File
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 1
```

The first line names the columns, the second gives their types (`s72` is a string of up to 72 characters, `i4` a 4-byte integer, a capital letter means the column may be empty), the third names the table and its key column. `Hello` is your file ID; `C_185f...` is a *component* that `rubrapack` made for it; 512 means "vital" (the installation fails if this file cannot be written). The Property table holds the product's identity:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
ProductName Hello
ProductVersion 1.0.0
UpgradeCode {3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}
...
```

The `ProductCode` is another GUID, which identifies *this version*: `rubrapack` derives it from the upgrade code, the architecture and the version, so it changes by itself when the version does. You do not write it. Part IV explains how these tables are stored in the file.

2.5 Check it without building

`rubrapack lint hello.toml` runs every check the build runs and writes nothing - handy after an edit. `rubrapack lint hello.msi` checks a finished package, even one made by another tool:

```
C:\work\hello> rubrapack lint hello.msi
hello.msi: 0 errors, 0 warnings
```

2.6 The same, with questions instead of a text editor

`rubrapack new hello.toml` asks for the name, the version, the folder with the files and so on, and writes a source like the one above (it also makes up the upgrade code). Chapter 19 describes it and its companion `rubrapack edit`. Writing the source by hand once, as here, is the best way to learn what each line does.

3 Versions and upgrades

In this chapter

3.1	One source for every version	8
3.2	What Windows does with two versions	9
3.3	Version numbers	9
3.4	Two identifiers	9
3.5	Check the upgrade before users do	10
3.6	Earlier versions made by another tool	10

Goal: ship version 1.1.0 of Hello so that installing it replaces 1.0.0, and installing 1.0.0 again afterwards is refused.

3.1 One source for every version

Editing version by hand before each release works, but it is easy to forget. Instead, let the version be a *variable* with a default, and give the real number when you build:

```
# tutorial 03: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
downgrade-message = "A newer Hello is already installed. Remove it first if you want this version."

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

- [define] holds variables. \${VERSION} anywhere in a string is replaced by the variable's value; here the default is 1.0.0.
- -D NAME=VALUE on the command line overrides a variable for one build:

```
C:\work\hello> rubrapack build hello.toml -o hello-1.0.0.msi
C:\work\hello> rubrapack build hello.toml -o hello-1.1.0.msi -D VERSION=1.1.0
```

- downgrade-message is the message a user sees when they try to install an older (or the same) version over a newer one. Without it, rubrapack uses "The same or a newer version of [ProductName] is already installed." ([ProductName] is replaced by the name).

3.2 What Windows does with two versions

Install `hello-1.0.0.msi`, then `hello-1.1.0.msi`. Windows Installer sees that the new package has the same upgrade code and a higher version, removes 1.0.0 and installs 1.1.0 - one entry in Installed apps, now showing 1.1.0. This is called a *major upgrade*, and it is the only kind of upgrade rubrapack builds: every new version is a complete package, and nobody needs the old one to install the new one.

Now try to install `hello-1.0.0.msi` again: Windows shows the downgrade message and changes nothing. Running `hello-1.1.0.msi` a second time is not an error either: it has the same product code as the installed version (see below), so Windows recognises the product as installed and does not install it twice. To repair an installed version - put back a deleted file, for example - run `msiexec /fa hello-1.1.0.msi`.

A package rebuilt from changed files *without* raising the version has the same product code as the installed one but is not the same package, and Windows refuses it with its own message: "Another version of this product is already installed" (error 1638). So raise the version whenever the files change - only a higher version replaces what is installed.

How rubrapack arranges this is visible in the Upgrade table:

```
C:\work\hello> rubrapack inspect hello-1.1.0.msi Upgrade
UpgradeCode VersionMin VersionMax Language Attributes Remove ActionProperty
s38 S20 S20 S255 i4 S255 s72
Upgrade UpgradeCode VersionMin VersionMax Language Attributes
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 1 RP_OLDER_FOUND
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 258 RP_NEWER_FOUND
```

The first row finds installed versions below 1.1.0 (to remove them); the second finds 1.1.0 or higher, which stops the installation with the downgrade message. The attribute numbers are bit flags - 258 is 256 + 2, "the minimum is included" plus "only detect, do not remove". Bit flags in Part III explains them, the tables that install in Part IV this table.

3.3 Version numbers

- Three or four numbers: `major.minor.build` or `major.minor.build.revision`. Windows Installer allows at most 255.255.65535 for the first three.
- **Windows compares only the first three.** `1.0.0.1` and `1.0.0.2` are the same version to it: the second would be refused as "the same". Raise the third number (or a higher one) for every release.
- A wrong version stops the build:

```
hello.toml:5:1: error[RP1308]: version '1.0' must be a.b.c or a.b.c.d with a, b <= 255 and c, d <= 65535
```

3.4 Two identifiers

	Upgrade code	Product code
identifies	the product across all its versions	one version
you write it	once, in upgrade-code; never change it	no: rubrapack derives it
changes	never	whenever the version (first three numbers) changes

```
1.0.0: ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
1.1.0: ProductCode {9CC17977-2023-8B8F-8BB0-2FA82C5B248F}
```

The derived product code is the same every time you build the same version, on any computer. product-code in [package] sets it by hand - only needed to continue a product whose earlier versions came from another tool with a product code of their own.

3.5 Check the upgrade before users do

Some mistakes do not stop a build but break the upgrade on users' computers: a changed upgrade code, a version that is not higher, a file that moved in a way Windows cannot follow. Compare the new package with the one before it:

```
C:\work\hello> rubrapack lint hello-1.1.0.msi --previous hello-1.0.0.msi
hello-1.1.0.msi: 0 errors, 0 warnings (against hello-1.0.0.msi)
```

and the same package against itself, to see what a mistake looks like:

```
C:\work\hello> rubrapack lint hello-1.0.0.msi --previous hello-1.0.0.msi
hello-1.0.0.msi: error[RP2302]: ProductVersion 1.0.0 is not higher than the previous 1.0.0 in
its first three fields: Windows Installer does not upgrade
hello-1.0.0.msi: warning[RP2303]: the ProductCode is the previous version's: that is a minor
upgrade (REINSTALLMODE), not the major upgrade rubrapack builds
hello-1.0.0.msi: 1 error, 1 warning (against hello-1.0.0.msi)
```

Keep every released .msi (for example in a releases folder) so the next one can be checked against it.

3.6 Earlier versions made by another tool

If your product was installed before with packages from another tool, rubrapack's upgrade replaces them as long as they have the same upgrade code. Two keys help when that goes badly:

- `refuse-upgrade-below` = "0.60.0": versions below this one are not upgraded but refused, with a message that tells the user how to remove them first - for old packages known to break when removed as part of an upgrade.
- `refuse-upgrade-message` = "...": that message's first part (the removal command is added).

4 More files and folders

In this chapter

4.1	The folder	11
4.2	The source	11
4.3	Folders: [dir.ID]	12
4.4	One file: [file.ID]	13
4.5	Many files: [files.ID]	13
4.6	An empty folder: [folder.ID]	13
4.7	Cleaning up: [remove.ID]	13
4.8	A second copy: [copy.ID]	13
4.9	Two more keys you may meet	14
4.10	What happened inside	14

Goal: install a whole program folder - the program, a readme under another name, a settings file the user may change, a folder of documents, samples in sub folders, an empty folder for data - and clean up the log files the program writes.

4.1 The folder

```
C:\work\hello\
  dist\
    hello.exe
    readme.txt
    settings.ini
    docs\
      guide.txt
    samples\
      sample1.txt
      sub\
        sample2.txt
  hello.toml
```

4.2 The source

```
# tutorial 04: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.2.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.DocsDir]
path = "INSTALLDIR/docs"
```

```

[dir.SamplesDir]
path = "INSTALLDIR/samples"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"
name = "Read me.txt"
vital = false

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"
keep = true

[files.DocFiles]
dir = "DocsDir"
glob = "dist/docs/**"

[files.SampleFiles]
dir = "SamplesDir"
glob = "dist/samples/**/*.txt"

[folder.Data]
dir = "INSTALLDIR"
name = "data"
keep = true

[remove.Logs]
dir = "INSTALLDIR"
name = "*.log"
on = "uninstall"

[copy.ReadmeInDocs]
source = "file:Readme"
dir = "DocsDir"
name = "readme.txt"

```

Build it and look at the files:

```

C:\work\hello> rubrapack build hello.toml -o hello.msi
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\Read me.txt 8 Readme
[ProgramFiles64Folder]\Hello\docs\guide.txt 6 F_6d1508f52f6615459567
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello
[ProgramFiles64Folder]\Hello\samples\sample1.txt 7 F_1dc2b5e85973ee91d955
[ProgramFiles64Folder]\Hello\samples\sub\sample2.txt 7 F_c9c5c4da73214a6abf37
[ProgramFiles64Folder]\Hello\settings.ini 23 Settings

```

(The output has more columns; only the path, the size and the ID are shown here.)

4.3 Folders: [dir.ID]

A dir's path starts from a *known folder* or from another dir, then names the folders below it:

path	On the user's computer (typical)
ProgramFiles/Hello	C:\Program Files\Hello (for an x86 package: C:\Program Files (x86)\Hello)
INSTALLDIR/docs	C:\Program Files\Hello\docs - below the dir INSTALLDIR, wherever the user put it
CommonAppData/Hello	C:\ProgramData\Hello - data shared by all users
LocalAppData/Hello	C:\Users\<name>\AppData\Local\Hello

Building on INSTALLDIR rather than repeating ProgramFiles/Hello matters once users may choose the install folder (chapter 6): docs then follows the folder they chose. The full list of known folders is in the Reference part (Tables).

4.4 One file: [file.ID]

- name installs the file under another name - here readme.txt becomes Read me.txt. Names may use any language's letters; what Windows forbids in file names (< > : " / \ | ? *) is refused.
- vital = false: if this file cannot be written, the installation goes on without it. By default every file is vital.
- keep = true: the file stays when the product is removed, and a repair or a later version does not overwrite it once the user has changed it. Use it for settings the user edits.

4.5 Many files: [files.ID]

glob is a path with wildcards: * is any characters within one folder name, ? one character, ** any number of folders. Every file that matches is installed, and **the folders below the first wildcard are recreated** under dir:

glob	dist\samples\sub\sample2.txt goes to
dist/samples/**/*.* into SamplesDir	...\Hello\samples\sub\sample2.txt
dist/samples/*.* into SamplesDir	not installed: * does not go into sub

The matches are sorted by name, so the package is the same whatever order the file system lists them in. A glob that matches nothing stops the build (RP1503) - usually a typo in the path. rubrapack gives each matched file an ID of its own (F_ and a number derived from its path).

4.6 An empty folder: [folder.ID]

[folder.Data] creates data inside INSTALLDIR although no file goes there - a place for the program to write. keep = true leaves it (and whatever the program wrote in it) when the product is removed.

4.7 Cleaning up: [remove.ID]

Files the program creates itself - logs, caches - are not the installer's, so Windows Installer would leave them, and with them the folder. [remove.Logs] deletes *.log in INSTALLDIR when the product is removed (on = "uninstall"). on = "install" deletes at installation (files an earlier version left behind), on = "both" at either. Without name, the folder itself is removed if it is empty. Folders that existed before the installation are never removed, and a failed installation puts removed files back.

4.8 A second copy: [copy.ID]

[copy.ReadmeInDocs] installs another copy of the file Readme into DocsDir as readme.txt. The copy comes and goes with its source.

4.9 Two more keys you may meet

- `any-arch = true` on a file: rubrapack checks that every program file (.exe, .dll) is built for the package's arch, because an x64 installer that copies a 32-bit DLL by mistake installs a broken program. When that is intended - a 32-bit helper in an x64 product - say so with `any-arch = true`.
- `component-guid = "{...}"` on a file: rubrapack derives every component's GUID from the product and the file's place, so it stays the same from version to version, which upgrades rely on. Give one by hand only to continue a component that an earlier package made with a different GUID.

4.10 What happened inside

Windows Installer installs *components*: small groups of resources that are installed and removed together, each with a GUID and a *key path* (the thing whose presence means "installed"). rubrapack makes one component per file, which is the safe rule, and one more for each folder, removal and copy that needs one:

```
C:\work\hello> rubrapack inspect hello.msi Component
...
C_74a883a037bc227f9189 {51A68DD4-96FA-83C0-86AA-F20D51339238} INSTALLDIR 272 Settings
C_cec3a9b89b2e391393d0 {615660C7-986E-8FB8-87AF-0616B544A6F9} Data 272
...
```

The 272 in the attributes column is $256 + 16$: 256 means a 64-bit component, 16 "permanent" - `keep = true`. Part IV describes these tables; `RemoveFile`, `CreateFolder` and `DuplicateFile` hold the removal, the empty folder and the copy.

5 Shortcuts, icons and the Installed apps entry

In this chapter

5.1	The source	15
5.2	Shortcuts: [shortcut.ID]	16
5.3	The Installed apps entry: [arp]	16
5.4	A name in another script	16
5.5	Try it	16
5.6	What happened inside	17

Goal: a Start menu folder "Example Software" with Hello and its readme, a desktop shortcut with its own icon, and an icon and links for Hello in Installed apps.

You need an icon file, `hello.ico`, next to the source (any `.ico` will do; many image editors save one).

5.1 The source

```
# tutorial 05: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.3.0"

[arp]
icon = "hello.ico"
help = "https://example.com/hello/help"
about = "https://example.com/hello"
no-repair = true

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.MenuFolder]
path = "Programs/Example Software"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"

[shortcut.StartMenu]
dir = "MenuFolder"
name = "Hello"
target = "file:Hello"
description = "Says hello"
working-dir = "INSTALLDIR"
```

```
[shortcut.ReadmeLink]
dir = "MenuFolder"
name = "Hello - read me"
target = "file:Readme"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
args = "--greet \"[ProductName] user\""
icon = "hello.ico"
```

5.2 Shortcuts: [shortcut.ID]

- `dir` is where the shortcut goes: a dir of your own, or directly one of Programs (the Start menu's program list), StartMenu, Desktop, Startup (started at every sign-in). Here the dir MenuFolder is a folder "Example Software" in the Start menu's programs, so the Start menu shows "Example Software" with two entries.
- `name` is the text under the shortcut.
- `target` is the file it opens: `file:` and the ID of a `[file.*]` of this package.
- `description` is the tool tip; `working-dir` (a dir ID) the folder the program starts in.
- `args` are the program's arguments. They are an MSI *formatted string*: `[ProductName]` becomes "Hello" when the shortcut is made. A double quote inside a TOML string is written `\`.
- `icon` gives the shortcut its own `.ico`; without it Windows shows the program's own icon.

The Start menu folder rubrapack makes for shortcuts is removed again with the product, as are the shortcuts.

5.3 The Installed apps entry: [arp]

("ARP" is the old name of the list: Add or Remove Programs.)

Key	Effect
<code>icon = "hello.ico"</code>	the icon in the list
<code>help = "https://..."</code>	a support link Windows shows with the product
<code>about = "https://..."</code>	the product's web page
<code>no-repair = true</code>	no Repair for this product
<code>no-modify = true</code>	no Modify/Change (useful when the package has nothing to change: chapter 8 adds something)

5.4 A name in another script

`name` may be in any language (`name = "안녕"`). The file properties of the `.msi` itself (its Details tab) can only show plain ASCII text, so for such a name give an ASCII form as well: `summary-name = "Annyeong"` in `[package]`. Without it rubrapack warns (RP1203) and the properties read "rubrapack package".

5.5 Try it

Build, install, and open the Start menu: "Example Software" holds "Hello" and "Hello - read me", and the desktop has "Hello" with the icon you gave. Installed apps shows Hello with its icon. Remove it: the shortcuts and the Start menu folder go too.

5.6 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi Shortcut
Shortcut Directory_ Name Component_ Target Arguments Description Hotkey
...
Desktop DesktopFolder Hello C_185f8db32271fe25f561 [#Hello] --greet "[ProductName] user"
ReadmeLink MenuFolder HELLO~1|Hello - read me C_44ff26388205705fb806 [#Readme]
StartMenu MenuFolder Hello C_185f8db32271fe25f561 [#Hello] Says hello
```

- A shortcut belongs to its target file's component: it is installed and removed with the file.
- [#Hello] is Windows Installer's way to say "the installed path of the file Hello".
- HELLO~1|Hello - read me holds two names: an old-style short name (8 characters, a dot, 3 characters - from the days of MS-DOS) and the long one. Windows Installer wants both for every name; rubrapack makes the short ones up.

The icons are stored inside the package (the Icon table), and the Property table carries the Installed apps settings:

```
ARPHHELPLINK https://example.com/hello/help
ARPNOREPAIR 1
ARPPRODUCTICON RpIcon1.ico
ARPURLINFOABOUT https://example.com/hello
```

6 Dialogs, a license, and starting the program

In this chapter

6.1	The source	18
6.2	Choosing a set of pages: ui	19
6.3	The license: license	19
6.4	The install folder	19
6.5	A picture at the top: banner	19
6.6	Your own words: [ui-text.ID]	19
6.7	Starting the program at the end: launch	20
6.8	Try it	20
6.9	What happened inside	20

Goal: an installer with pages - welcome, a license the user must accept, a choice of folder, progress, finished - that offers to start Hello at the end.

6.1 The source

LICENSE.txt and banner.bmp (a picture about 493 x 58 pixels) are next to the source.

```
# tutorial 06: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"
license = "LICENSE.txt"

[define]
VERSION = "1.4.0"

[ui]
banner = "banner.bmp"
launch = "file:Hello"
launch-args = "--first-run"
launch-checked = true

[ui-text.WelcomeText]
text = "This will install [ProductName] [ProductVersion]. Close Hello if it is running."

[ui-text.ExitText]
text = "[ProductName] is ready. Thank you for installing it."

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
```

```
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

6.2 Choosing a set of pages: ui

ui in [package] picks one of the built-in dialog sets:

ui	When installing	When run again after installation
(none)	Windows Installer's own small progress window only	the same
basic	progress, then finished (or an error)	progress, finished
minimal	welcome, the license (if any), progress, finished	repair or remove
installdir	welcome, license, install folder (with a folder browser), ready, progress, finished	repair or remove
features	as installdir, plus a tree of features and the disk space they need (chapter 8)	repair, change or remove

Every set also has the pages Windows needs when something happens: "Are you sure you want to cancel?", an error message, "these programs use files that must be replaced" (files in use), and "not enough disk space". A package with dialogs still installs silently with /qn: every page only collects values that have defaults.

6.3 The license: license

license = "LICENSE.txt" adds a license page after the welcome page. **Next stays disabled until "I accept the terms of the license agreement" is ticked.** The file may be:

- .txt or .md - shown as plain text; Korean, other scripts and emoji are kept;
- .rtf - shown as it is, with its fonts and formatting (make it in WordPad or Word).

The license text is stored in the package when it is built; the user does not need the file.

6.4 The install folder

With installdir or features, a page shows the install folder with a Change button that opens a folder browser. The folder the user picks becomes INSTALLDIR, and every dir built on INSTALLDIR moves with it. To let the user change another dir, name it: install-dir = "AppDir" in [ui]. From the command line the folder is given as a property:

```
msiexec /i hello.msi INSTALLDIR="D:\Tools\Hello"
```

6.5 A picture at the top: banner

banner = "banner.bmp" puts a picture in the strip at the top of every inner page (Windows Installer draws a BMP; about 493 x 58 pixels). Without it the strip is plain white.

6.6 Your own words: [ui-text.ID]

Every text on the built-in pages has an ID; [ui-text.ID] replaces one. The IDs are listed in the Reference part (Dialogs); WelcomeText and ExitText are the sentences on the first and the last page. Texts are formatted strings: [ProductName], [ProductVersion] and [Manufacturer] are replaced, and a literal square bracket is written \[]. In a button's text, & marks the letter that works with Alt: &Next is Alt+N.

6.7 Starting the program at the end: launch

`launch = "file:Hello"` puts a tick box "Launch Hello" on the finished page (ticked, unless `launch-checked = false`); `launch-args` are the program's arguments. If it is ticked, Finish starts Hello - with the rights of the user who ran the installer, not the installer's administrator rights - after a first installation or an upgrade, but not after a repair or a removal, and never in a silent installation.

6.8 Try it

Double-click `hello.msi: welcome` (with your text and the banner), the license (Next is grey until you tick the box), the install folder, ready, progress, finished with "Launch Hello" ticked. Run it again after installing: a page offers Repair and Remove.

6.9 What happened inside

A package with dialogs carries them as tables - `Dialog`, `Control`, `ControlEvent` and more - that describe every window, button and text box with its position and what happens when it is pressed. rubrapack writes them from its built-in sets:

```
C:\work\hello> rubrapack inspect hello.msi Dialog
...
RpBrowseDlg RpCancelDlg RpErrorDlg RpExitDlg RpFatalDlg RpInstallDirDlg RpLicenseDlg
RpMaintenanceDlg RpOutOfDiskDlg RpProgressDlg RpReadyDlg RpUserExitDlg RpWelcomeDlg FilesInUse
```

(Here only the first column is shown.) The `InstallUISequence` table says when a page appears:

```
RpWelcomeDlg NOT Installed 1230
RpMaintenanceDlg Installed AND NOT RESUME AND NOT Preselected 1240
RpProgressDlg 1280
RpExitDlg -1
RpUserExitDlg -2
RpFatalDlg -3
```

The middle column is a *condition*: the welcome page only when the product is not installed yet, the maintenance page when it is. The negative numbers are special: the page shown at the end after success (-1), after the user cancelled (-2), after a failure (-3). Chapter 9 uses conditions of your own.

7 Several languages

In this chapter

7.1	The source	21
7.2	Adding a language: languages	22
7.3	A license per language: license-xx	22
7.4	A text per language: text-xx	22
7.5	Languages other than English and Korean	22
7.6	What is not translated	22
7.7	Try it	22
7.8	What happened inside	23

Goal: one package whose pages speak English or Korean, with a license in each language, Korean chosen in advance on Korean Windows.

7.1 The source

LICENSE-ko.txt, the Korean license, is next to LICENSE.txt.

```
# tutorial 07: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"
license = "LICENSE.txt"
downgrade-message = "A newer Hello is already installed."

[define]
VERSION = "1.5.0"

[ui]
languages = ["ko"]
license-ko = "LICENSE-ko.txt"
launch = "file:Hello"

[ui-text.WelcomeText]
text = "This will install [ProductName] [ProductVersion]."
```

```
text-ko = "[ProductName] [ProductVersion]을(를) 설치합니다."
```

```
[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"
```

7.2 Adding a language: languages

The pages are always in English. `languages = ["ko"]` in `[ui]` adds Korean to the same package, and then:

- the first page asks which language to use - English or 한국어;
- on a Windows whose regional format (or, failing that, system language) is Korean, 한국어 is already selected; elsewhere English;
- every page after it - welcome, license, folder, ready, progress, finished, and the cancel, error, files-in-use, disk-space and maintenance pages - speaks the language chosen.

With English alone there is no language page. A silent installation (`/qn`) shows no pages and needs no choice; `RPLANGUAGE=ko` on the command line chooses without the page:

```
msiexec /i hello.msi RPLANGUAGE=ko
```

7.3 A license per language: license-xx

`license-ko = "LICENSE-ko.txt"` is the license shown when Korean is chosen; English (and any added language without its own) shows `[package] license`. The same `.txt` / `.md` / `.rtf` rules apply.

7.4 A text per language: text-xx

In `[ui-text.ID]`, `text` is the text for every language and `text-ko` the Korean one. Every built-in text already exists in English and Korean; give `text-ko` only where you give `text`, so both languages say the same thing.

7.5 Languages other than English and Korean

rubrapack's built-in pages are written in English and Korean. For another language - `ja`, `de`, `fr`, ... - you provide every text as `text-xx` yourself; the build lists the ones missing:

```
hello.toml:16:1: error[RP1202]: language 'ja' has no built-in texts: give [ui-text.ID] text-ja
for all of them (64 missing, the first is Back)
```

For such a language, three more keys in `[ui]` may help: `name-ja` (its name on the language page), `font-ja` (the typeface of its pages) and `langid-ja` (the Windows language numbers that select it in advance - one number or a list). For common languages (`ja`, `zh`, `de`, `fr`, `es`, `it`, `pt`, `nl`, `pl`, `ru`, `uk`, `tr`, `vi`, `th`) these three are built in. Korean pages use the Malgun Gothic typeface, English ones Segoe UI.

7.6 What is not translated

- Windows Installer's own words - the sizes and the menu in the feature tree, "time remaining", its error messages - come from Windows, not from the package.
- The downgrade message (chapter 3) and the removal message of `refuse-upgrade-below` are one text each; write them in both languages if you need both ("A newer Hello is already installed. / 더 새 Hello 가 설치되어 있습니다.").
- `[package] language = "ko-KR"` only sets the package's own language code; it does not make the pages Korean - `languages` does.

7.7 Try it

On a Korean Windows, double-click `hello.msi`: the language page has 한국어 selected; every page after it is Korean, with the Korean license. Choose English: the same pages in English with `LICENSE.txt`. Try `msiexec /i hello.msi RPLANGUAGE=ko` on an English Windows.

7.8 What happened inside

A Windows Installer package has no built-in notion of "several languages": each text on a page is fixed in its table. rubrapack therefore makes every text a *property* - a named value - and shows the property on the page:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
RpT_WelcomeText This will install Hello 1.5.0.
RpT_WelcomeText_en This will install Hello 1.5.0.
RpT_WelcomeText_ko Hello 1.5.0을(를) 설치합니다.
...
```

Before the first page, small actions copy the chosen language's texts into the shown properties; each runs only when its condition is true:

```
C:\work\hello> rubrapack inspect hello.msi InstallUISequence
...
RpL_ko_26 RPLANGUAGE = "ko" 21
...
C:\work\hello> rubrapack inspect hello.msi CustomAction
...
RpL_ko_26 51 RpT_WelcomeText [RpT_WelcomeText_ko]
```

Type 51 is "set a property": RpT_WelcomeText becomes the value of RpT_WelcomeText_ko. The text is stored as UTF-8 in the package (code page 65001); Text in Part III explains text encodings, and the MSI database in Part IV how the strings are stored.

8 Optional parts: features and the user's choices

In this chapter

8.1	Features	24
8.2	The source	24
8.3	Declaring features: [feature.ID]	25
8.4	Putting files in features: feature	26
8.5	What the user sees	26
8.6	Choosing from the command line	26
8.7	Changing the choice later	27
8.8	What happened inside	27

Goal: let the user choose what to install. The program itself is always installed; the documentation is offered and ticked; the samples are offered but not ticked; a diagnostics part is hidden and installed only on request.

8.1 Features

A *feature* is a part of the product the user can choose: a line in the installer's tree with a title, a description and a tick. Every file belongs to exactly one feature; a feature holds as many files as you like.

8.2 The source

```
# tutorial 08: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "features"
license = "LICENSE.txt"

[define]
VERSION = "1.6.0"

[feature.Main]
title = "Hello"
description = "The program itself."
required = true

[feature.Documentation]
title = "Documentation"
description = "The guide, in the docs folder."

[feature.Samples]
title = "Samples"
description = "Example files to try Hello with."
level = 2
```



```

[feature.MoreSamples]
title = "More samples"
description = "Samples in sub folders."
parent = "Samples"
follow-parent = true

[feature.Diagnostics]
title = "Diagnostics"
hidden = true
level = 3

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
feature = "Main"

[dir.DocsDir]
path = "INSTALLDIR/docs"
feature = "Documentation"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[dir.MoreSamplesDir]
path = "INSTALLDIR/samples/sub"
feature = "MoreSamples"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[files.DocFiles]
dir = "DocsDir"
glob = "dist/docs/**"

[files.SampleFiles]
dir = "SamplesDir"
glob = "dist/samples/*.txt"

[files.MoreSampleFiles]
dir = "MoreSamplesDir"
glob = "dist/samples/sub/*.txt"

[file.Readme]
dir = "INSTALLDIR"
source = "dist/readme.txt"
feature = "Diagnostics"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

```

8.3 Declaring features: [feature.ID]

Key	Meaning
title	the line in the tree
description	the text shown when the line is selected

Key	Meaning
level	1 (default): installed by default. 2 or more: offered, but only installed when chosen
required = true	always installed: the tree does not offer "will be unavailable" for it
parent = "Samples"	a line under another feature in the tree
follow-parent = true	installed exactly when its parent is
hidden = true	not shown in the tree at all

Feature IDs share one name space with dirs and files: Documentation the feature and DocsDir the folder must have different IDs (rubrapack says so with RP1301 otherwise).

8.4 Putting files in features: feature

- On a dir: every file in that folder belongs to the feature - DocsDir puts `guide.txt` in Documentation, SamplesDir puts `sample1.txt` in Samples.
- On a file or a file group: that file (or group) belongs to it, whatever its folder - Readme is in `INSTALLDIR`, a folder of Main, but belongs to Diagnostics.

A dir does not pass its feature on to the dirs below it: MoreSamplesDir names its own. As soon as the source declares one feature, **everything installed needs one**; rubrapack stops with RP1202 naming what has none.

8.5 What the user sees

With `ui = "features"`, after the license and the install folder comes "Choose features": a tree

```
[x] Hello                      (cannot be turned off)
[x] Documentation
[ ] Samples
    [ ] More samples          (follows Samples)
```

Each line has a menu: "Will be installed on local hard drive", "Entire feature will be installed on local hard drive", "Entire feature will be unavailable". Selecting a line shows its description and the disk space it needs; "Disk Usage" shows the space on every drive. (These texts come from Windows and stay English.) Diagnostics does not appear.

A default installation (double-click, Next, Next, ...) installs Hello and the documentation.

8.6 Choosing from the command line

Features are chosen with properties, so a silent installation can choose too:

Command	Installs
<code>msiexec /i hello.msi /qn</code>	the defaults: Main, Documentation
<code>msiexec /i hello.msi /qn ADDLOCAL=Samples</code>	the defaults and the samples (with More samples, which follows)
<code>msiexec /i hello.msi /qn ADDLOCAL=ALL</code>	every feature, hidden ones included
<code>msiexec /i hello.msi /qn INSTALLLEVEL=3</code>	every feature whose level is 3 or less: all of these
<code>msiexec /i hello.msi /qn REMOVE=Documentation</code>	the defaults without the documentation

Feature IDs are case-sensitive and separated by commas: `ADDLOCAL=Samples,Diagnostics`. `required` only binds the tree: `REMOVE=Main` on the command line is still obeyed (that is how Windows Installer works).

8.7 Changing the choice later

After installation, Installed apps offers Modify for Hello; it opens the installer's maintenance page with **Change**, **Repair** and **Remove**. Change shows the same tree with what is installed ticked; the user adds or removes parts. From the command line on an installed product:

```
msiexec /i hello.msi ADDLOCAL=Samples
msiexec /i hello.msi REMOVE=Samples
```

8.8 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi Feature
Feature Feature_Parent Title Description Display Level Directory_ Attributes
...
Diagnostics Diagnostics 0 3 0
Documentation Documentation The guide, in the docs folder. 3 1 0
Main Hello The program itself. 1 1 16
MoreSamples Samples More samples Samples in sub folders. 7 1 2
Samples Samples Example files to try Hello with. 5 2 0
```

- Display orders the tree (odd numbers, in the order of the source); 0 hides the feature.
- Level is the level; Windows installs a feature by default when its level is not above the property INSTALLLEVEL, which is 1.
- Attributes are bit flags: 16 means "may not be made absent" (required), 2 "follow the parent".

The FeatureComponents table ties each component - each file - to its feature:

```
C:\work\hello> rubrapack inspect hello.msi FeatureComponents
...
Documentation C_a10be66379e69940a2e0
Main C_185f8db32271fe25f561
Samples C_a5674982edf0797da30f
...
```

9 Conditions and searches

In this chapter

9.1	Properties	28
9.2	The source	28
9.3	Your own property: [property.NAME]	29
9.4	Installing something only when: when	30
9.5	Refusing to install: [require.ID]	30
9.6	Looking around first: [search.ID]	30
9.7	Remembering the install folder	30
9.8	What happened inside	30

Goal: decide *on the user's computer* what to install. Install only on 64-bit Windows; offer a part only where Notepad exists; make the desktop shortcut something the user can turn off; and install the next version into the folder the user chose for this one.

9.1 Properties

Windows Installer keeps named values called *properties* while it runs: ProductName, INSTALLDIR, VersionNT64 (set on 64-bit Windows), and your own. A property whose name is all capitals is *public*: it can be set on the command line (`msiexec /i hello.msi DESKTOP_SHORTCUT=0`) and by the dialogs. A *condition* is a small expression over properties, true or false:

Condition	True when
VersionNT64	the property is set (not empty): here, on 64-bit Windows
NOT Installed	the product is not installed yet (a first installation)
DESKTOP_SHORTCUT = "1"	the property's value is exactly 1
VersionNT >= 603	the Windows version number is 6.3 or later
A AND (B OR NOT C)	combinations, as you would expect

9.2 The source

```
# tutorial 09: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "features"

[define]
VERSION = "1.7.0"

[property.DESKTOP_SHORTCUT]
value = "1"
secure = true

[require.Windows64]
```

```

condition = "VersionNT64"
message = "[ProductName] needs 64-bit Windows."

[search.Notepad]
property = "NOTEPAD_PATH"
kind = "file"
path = "Windows"
file = "notepad.exe"

[search.PreviousDir]
property = "INSTALLDIR"
kind = "registry"
root = "HKLM"
key = "Software\\Example Software\\Hello"
name = "InstallDir"

[feature.Main]
title = "Hello"
required = true

[feature.NotepadHelper]
title = "Open notes in Notepad"
description = "Only offered where Notepad is installed."
when = "NOTEPAD_PATH"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
feature = "Main"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"
feature = "NotepadHelper"

[registry.RememberDir]
root = "HKLM"
key = "Software\\Example Software\\Hello"
name = "InstallDir"
value = "[INSTALLDIR]"
feature = "Main"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
when = "DESKTOP_SHORTCUT = \\\"1\\\""

```

9.3 Your own property: [property.NAME]

[property.DESKTOP_SHORTCUT] creates the property with the default 1. Names must be all capitals to be public; names Windows or rubrapack use themselves (ALLUSERS, ARP . . . , MSI . . . , ...) are refused.

- `secure = true`: an installation for all users runs in two parts - the user's part (dialogs) and an elevated part that changes the computer. A value set in the first part or on the command line reaches the second only if the property is *secure*. Make every property that a when condition tests secure. (rubrapack does this by itself for the properties of your own dialog pages and for search results.)
- `hidden = true`: the value is not written into installation logs - for passwords and keys.

9.4 Installing something only when: when

when on a feature, a file or file group, a shortcut, a registry value, an environment variable or an INI value installs it only when the condition is true at the first installation:

- `when = "DESKTOP_SHORTCUT = \"1\""` on the desktop shortcut: by default it is made; with `msiexec /i hello.msi DESKTOP_SHORTCUT=0` it is not. Chapter 14 turns the property into a tick box on a page of your own.
- `when = "NOTEPAD_PATH"` on a feature: where the condition is false, the feature is neither installed nor shown in the tree.

A repair keeps what was installed; a major upgrade (the next version) decides again.

9.5 Refusing to install: [require.ID]

A requirement stops a first installation with its message when its condition is false - here, on 32-bit Windows: "Hello needs 64-bit Windows." Repair and removal are never blocked.

9.6 Looking around first: [search.ID]

A search runs before anything else and puts what it found into a property (empty if nothing was found), which conditions can then test:

kind	Finds	Keys
file	the full path of a file	path (a known folder and a relative path), file, min-version (for programs)
dir	a folder	path
registry	a value in the registry	root, key, name, view (32 for the 32-bit view)
component	the key file of another product's component	component-guid

[search.Notepad] looks for `notepad.exe` in the Windows folder and puts its path into `NOTEPAD_PATH`.

9.7 Remembering the install folder

If a user installed Hello into `D:\Tools\Hello`, the next version should go there too, not back to Program Files. Two pieces do it:

1. [registry.RememberDir] writes the folder into the registry when installing: [INSTALLDIR] in a value is replaced by the folder (chapter 11 is about the registry).
2. [search.PreviousDir] reads it back. Its property is the dir ID `INSTALLDIR`, so what it finds becomes that dir's *default* - but only if that folder still exists, and a folder given on the command line still wins.

9.8 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi LaunchCondition
Installed OR (VersionNT64) [ProductName] needs 64-bit Windows.
C:\work\hello> rubrapack inspect hello.msi AppSearch
```

```

NOTEPAD_PATH Notepad
RpFound_PreviousDir PreviousDir
C:\work\hello> rubrapack inspect hello.msi Condition
NotepadHelper 0 NOT Installed AND NOT (NOTEPAD_PATH)

```

(Headers left out.) rubrapack adds Installed OR to requirements, so that an installed product can always be repaired or removed. A feature's when becomes a row in the Condition table that sets its level to 0 - "not installed, not shown" - when the condition is false; NOT Installed keeps that from happening at removal, when properties are gone. The desktop shortcut got a component of its own, whose Condition column holds the when:

```

C_2ea6d5bfd9b4a9073b30 {BCBFF063-...} DesktopFolder 260 DESKTOP_SHORTCUT = "1"
R_7f1795ed8aa7ac1a1c60

```

10 Who it installs for, and a guarded folder

In this chapter

10.1	Three scopes	32
10.2	The source	32
10.3	A dual package	33
10.4	A guarded install folder: guard	33
10.5	Try it	33
10.6	What happened inside	34

Goal: let a user without administrator rights install Hello for themselves, while an administrator can still install it for everyone - and refuse an install folder that someone else prepared.

10.1 Three scopes

So far every package installed *per machine*: into Program Files, for every user of the computer, which needs administrator rights (the User Account Control prompt). scope in [package] changes that:

scope	Installs for	Needs administrator	ProgramFiles/Hello becomes	Start menu, desktop	Registry HKMU
machine (default)	everyone	yes	C:\Program Files\Hello	all users'	HKLM
user	the current user	no	%LOCALAPPDATA%\Programs\Hello	the user's	HKCU
dual	the user by default, or everyone when chosen	only for everyone	either of the above	either	either

%LOCALAPPDATA% is the user's own C:\Users\\AppData\Local. A per-user installation needs no permission and is invisible to other users of the computer - often right for small tools.

Some things only exist per machine and need scope = "machine": services, fonts, permissions, and the folders Windows, System, Fonts and CommonAppData. rubrapack refuses them in a user or dual package.

10.2 The source

```
# tutorial 10: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
scope = "dual"
ui = "installdir"

[define]
VERSION = "1.8.0"

[dir.INSTALLDIR]
```



```

path = "ProgramFiles/Hello"
guard = true

[file.Hello]
dir = "INSTALLEDIR"
source = "dist/hello.exe"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

```

10.3 A dual package

With scope = "dual" and dialogs, a page after the license asks:

```

( ) Just me           (the default)
( ) Everyone on this computer

```

"Everyone" asks for administrator rights; the install folder, the Start menu entry and the Installed apps entry follow the choice. Without dialogs, a dual package installs for the current user; from an administrator's terminal it installs for everyone with

```
msiexec /i hello.msi ALLUSERS=1 MSIINSTALLPERUSER=""
```

(MSIINSTALLPERUSER="" means "empty": the property is cleared.) A user package refuses to install for everyone.

10.4 A guarded install folder: guard

Some programs are loaded by other programs or run with high rights - an input method, a shell extension, a service. For them an install folder that *already exists* is a risk: whoever created it may have left files in it (a planted DLL would be loaded with the program's rights). guard = true on a dir makes a first installation stop, before any file is placed, when that folder

- already exists and is not owned by SYSTEM, Administrators or TrustedInstaller, or
- is reached through a junction or another link (the folder itself or any folder above it).

A folder that does not exist yet passes - the installer creates it. Repair, removal and an upgrade into the folder the earlier version created are not affected. The installation ends with the message DirGuardText (in the language chosen in the dialogs), also in a silent installation, and the log names the owner it found. The check is done by a small helper DLL of rubrapack's that the package then carries.

10.5 Try it

As a normal user (not an administrator), double-click hello.msi and keep "Just me": no permission prompt; Hello lands in %LOCALAPPDATA%\Programs\Hello, and only you see it in the Start menu and in Installed apps. Remove it, and install again choosing "Everyone": Windows asks for administrator rights and installs into Program Files.

To see the guard: as a normal user, create the folder C:\Users\Public\Hello (its owner is then that user). From an administrator's terminal, install for everyone into it:

```
msiexec /i hello.msi ALLUSERS=1 MSIINSTALLPERUSER="" INSTALLEDIR="C:\Users\Public\Hello\"
```

The installation stops with the guard's message and installs nothing. Delete the folder and run the same command again: now the installer creates the folder itself, and the installation goes through.

10.6 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi Property
...
ALLUSERS    2
MSIINSTALLPERUSER 1
RP_GUARD    INSTALLDIR
...
```

ALLUSERS = 2 together with MSIINSTALLPERUSER = 1 is Windows Installer's "single package for both": per user unless asked otherwise. A per-machine package has ALLUSERS = 1. RP_GUARD lists the guarded dirs for the helper DLL. The page that asks is RpScopeDlg in the Dialog table.

11 The registry, environment variables, INI files, file types and links

In this chapter

11.1	The registry in two minutes	35
11.2	The source	35
11.3	Registry values: [registry.ID]	37
11.4	Environment variables: [env.ID]	38
11.5	INI files: [ini.ID]	38
11.6	File types: [assoc.ID]	38
11.7	Links: [protocol.ID]	39
11.8	Try it	39
11.9	What happened inside	39

Goal: give Hello the settings other programs look for - registry values of every type, an environment variable, a line in its INI file - and let Windows open .hello files and hello: links with it.

11.1 The registry in two minutes

The Windows *registry* is a database of settings, organised like folders: *keys* contain other keys and *values*; a value has a name, a type and data. The top-level keys ("roots") are:

Root	Holds
HKLM (HKEY_LOCAL_MACHINE)	settings for the whole computer; writing needs administrator rights
HKCU (HKEY_CURRENT_USER)	the current user's settings
HKCR (HKEY_CLASSES_ROOT)	file types and links (a merged view of both)
HKMU	rubrapack's (and Windows Installer's) "HKLM when installed for everyone, HKCU when for one user"

Run regedit on Windows to look around (and do not change what you do not know).

11.2 The source

```
# tutorial 11: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.9.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"
```

```

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Settings]
dir = "INSTALLDIR"
source = "dist/settings.ini"

[registry.InstallDir]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "InstallDir"
value = "[INSTALLDIR]"

[registry.Greeting]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Greeting"
value = "Hello, world"
with = "file:Hello"

[registry.Runs]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "MaxRuns"
type = "dword"
value = 100

[registry.BigNumber]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Limit"
type = "qword"
value = "0x100000000"

[registry.Colors]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Colors"
type = "multi"
value = ["red", "green", "blue"]

[registry.LogPath]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "LogPath"
type = "expand"
value = "%TEMP%\hello.log"

[registry.Legacy32]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Mode"
value = "compatible"
view = "32"

[registry.OldKey]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello Old'
remove = true

```

```

[registry.UserChoice]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "FirstRun"
type = "dword"
value = 1
keep = true

[env.HelloHome]
name = "HELLO_HOME"
value = "[INSTALLDIR]"

[env.Path]
name = "PATH"
value = "[INSTALLDIR]"
mode = "append"

[ini.InstalledVersion]
dir = "INSTALLDIR"
file = "settings.ini"
section = "hello"
key = "installed"
value = "[ProductVersion]"

[ini.Plugins]
dir = "INSTALLDIR"
file = "settings.ini"
section = "hello"
key = "plugins"
value = "core"
mode = "add"

[assoc.HelloDoc]
extension = ".hello"
prog-id = "Example.HelloDocument"
description = "Hello document"
target = "file:Hello"
icon = "file:Hello"
args = "--open \"%1\""

[protocol.HelloLink]
name = "hello"
description = "Hello link"
target = "file:Hello"

```

TOML note: a string in *single* quotes is taken as it is, so 'SOFTWARE\Example Software\Hello' needs no doubled backslashes; in double quotes a backslash starts an escape ("%TEMP%\hello.log").

11.3 Registry values: [registry.ID]

type	TOML value	Registry type
string (default)	a string	REG_SZ
expand	a string with %VARIABLES%	REG_EXPAND_SZ - Windows expands %TEMP% when the value is read
dword	an integer 0 .. 4294967295	REG_DWORD, 32 bits
qword	an integer, or "0x" and up to 16 hex digits	REG_QWORD, 64 bits

type	TOML value	Registry type
binary	hex digits, "01ff"	REG_BINARY
multi	an array of strings	REG_MULTI_SZ

- name omitted: the key's default value (shown as "(Default)" in regedit).
- value is a formatted string: [INSTALLDIR] becomes the install folder, [#Hello] the path of the file Hello.
- Every value is removed again with the product; keep = true leaves it (FirstRun here, which the program may change).
- with = "file:Hello" ties the value to that file: installed and removed with it (Greeting).
- view = "32": 64-bit Windows keeps a separate registry for 32-bit programs; a 32-bit program reading SOFTWARE\Example Software\Hello sees that one. Values go to the 64-bit view in an x64 package unless view = "32".
- remove = true deletes, during installation, the value named by name - or, without name, the whole key (OldKey: something an older version left behind).

Windows Installer cannot write REG_QWORD by itself; for qword values rubrapack adds its small helper DLL to the package, which writes them and puts the previous values back if the installation fails.

11.4 Environment variables: [env.ID]

A variable for the whole computer (per user in a per-user installation). mode = "set" (default) sets it, append adds ;value to the end of what is there, prepend value; to the front. The product's removal undoes exactly that: HELLO_HOME is deleted; from PATH only ;C:\Program Files\Hello\ is taken out. keep = true leaves it. Programs started after the installation see the change (a terminal that was already open does not).

11.5 INI files: [ini.ID]

An INI file is a text file of [section] and key=value lines. mode = "set" (default) writes key=value in the section, add adds the value to a comma-separated list (plugins=core or, if there was plugins=extra, plugins=extra,core), and remove deletes the key - before the files are installed, so it cleans up an INI file that an older version left. Removal takes out what set and add wrote. file is the file's name in dir; it does not have to be installed by the package.

After installing, settings.ini reads:

```
[hello]
greeting=Hello
installed=1.9.0
plugins=core
```

11.6 File types: [assoc.ID]

[assoc.HelloDoc] makes Windows open .hello files with Hello:

- extension is the file ending, lower case, with the dot;
- prog-id names the kind of document (Company.Kind); several extensions may share one;
- description is what Explorer calls the type ("Hello document");
- target is the program, args its arguments ("%1" is the file's path; the default);
- icon (file:ID) is the file whose first icon the documents show.

If the user has already chosen another program for .hello, Windows keeps that choice.

11.7 Links: [protocol.ID]

[protocol.HelloLink] registers the scheme hello:, so a link hello:world - in a browser, in the Run box - starts Hello with the whole link as its argument.

11.8 Try it

Install, then:

- regedit: HKEY_LOCAL_MACHINE\SOFTWARE\Example Software\Hello holds the values; ... \WOW6432Node\Example Software\Hello holds Mode (the 32-bit view).
- a new terminal: echo %HELLO_HOME% and echo %PATH%.
- C:\Program Files\Hello\settings.ini: the two new lines.
- a file test.hello on the desktop opens Hello; Win+R, hello:world, Enter starts Hello.

Remove Hello: all of it goes, except FirstRun.

11.9 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi Registry
...
Colors 2 SOFTWARE\Example Software\Hello Colors [~]red[~]green[~]blue[~] C_f023...
Greeting 2 SOFTWARE\Example Software\Hello Greeting Hello, world C_185f...
HelloDoc.Ext 0 .hello Example.HelloDocument C_185f...
LogPath 2 SOFTWARE\Example Software\Hello LogPath #%%TEMP%\hello.log C_2e80...
Runs 2 SOFTWARE\Example Software\Hello MaxRuns #100 C_97ef...
...
```

The second column is the root (0 HKCR, 1 HKCU, 2 HKLM, -1 HKMU). The value column carries the type in its first characters, as Windows Installer wants: #100 is a DWORD, %% an expandable string, [~] separates the strings of a multi-string, #x starts binary data. File types and links are nothing but registry values under HKCR, in the program's component. The QWORD value is not in this table: it travels in the property RP_QWORDS for the helper DLL. Environment variables are in the Environment table:

```
HelloHome =-*HELLO_HOME [INSTALLDIR] C_0fa9...
Path =-*PATH [~];[INSTALLDIR] C_82df...
```

In the name, = means "set", - "remove at uninstall", * "a system variable"; [~] in the value stands for what was there before.

12 Services, fonts, permissions

In this chapter

12.1	The source	40
12.2	A service: [service.ID]	41
12.3	A font: [font.ID]	41
12.4	Access rights: [permission.ID]	41
12.5	Restarting: reboot	42
12.6	Try it	42
12.7	What happened inside	42

Goal: install a background service that starts by itself, a font every program can use, and a data folder whose access rights you set - the three things only a per-machine package may do.

12.1 The source

dist\hellosvc.exe is the service program, dist\HelloSans.ttf a font file.

```
# tutorial 12: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
reboot = "suppress"

[define]
VERSION = "1.10.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[dir.DataDir]
path = "CommonAppData/Hello"

[dir.FontsDir]
path = "Fonts"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[file.Service]
dir = "INSTALLDIR"
source = "dist/hellosvc.exe"

[file.Font]
dir = "FontsDir"
source = "dist/HelloSans.ttf"

[service.HelloService]
file = "file:Service"
```



```

name = "HelloService"
display-name = "Hello background service"
description = "Keeps Hello's greetings up to date."
start = "auto"
account = "LocalService"
args = "--service"
start-on-install = true

[font.HelloSans]
file = "file:Font"
title = "Hello Sans"

[permission.DataFolder]
target = "dir:DataDir"
sddl = "D:PAI(A;OICI;FA;;;SY)(A;OICI;FA;;;BA)(A;OICI;0x1301bf;;;BU)"

```

12.2 A service: [service.ID]

A *service* is a program Windows runs in the background, without a window, often before anyone signs in (the Services console, `services.msc`, lists them).

Key	Meaning
file	the service's program: file: and a [file.*] ID
name	the service's internal name (used by <code>sc start HelloService</code>)
display-name, description	what the Services console shows
start	auto (at every start of Windows), demand (when something starts it), disabled
account	who it runs as: LocalSystem (full rights), LocalService (few rights - prefer it), NetworkService
args	its command-line arguments
start-on-install = true	start it at the end of the installation

The installer stops the service before its files are replaced (in an upgrade or a repair) and at removal, and deletes it at removal. The program must really be a service - one that answers Windows' service manager; an ordinary program would make the start fail, and with it the installation.

12.3 A font: [font.ID]

A font is installed by putting its file into the Fonts folder - [dir.FontsDir] with `path = "Fonts"`, the known folder alone - and registering it with [font.ID]. `title` is the name Windows lists; without it Windows reads the name from the font file. Every program sees the font after the installation; removal unregisters and deletes it.

12.4 Access rights: [permission.ID]

Every file, folder and registry key in Windows has an access list saying who may read or change it. [permission.ID] sets that list, written in *SDDL* (Security Descriptor Definition Language), on a folder the package creates (dir:ID), one of its files (file:ID) or a registry value it writes (registry:ID). A folder given a permission is created even without files in it.

The SDDL string above, piece by piece:

Piece	Meaning
D:	the access list (DACL) follows
PAI	protected (do not inherit from the parent folder), and passed on to what is inside
(A;OICI;FA;;;SY)	Allow, to files and sub folders (Object and Container Inherit), Full Access, to SYSTEM
(A;OICI;FA;;;BA)	the same for the Built-in Administrators
(A;OICI;0x1301bf;B;D)	Built-in Users may read, write and delete, but not change the rights

So C:\ProgramData\Hello is writable by every user, but only administrators may change who may. Get a string for a folder you set up by hand with `icaccls` or PowerShell's `(Get-Acl C:\path).Sddl`.

12.5 Restarting: reboot

Sometimes a file cannot be replaced until Windows restarts. `reboot = "suppress"` (the default) never restarts by itself: the installation ends with exit code 3010 ("restart needed") and the user restarts later. `reboot = "allow"` lets Windows Installer ask for, or at /qn perform, a restart when one is needed.

12.6 Try it

Install (as administrator), then: `services.msc` lists "Hello background service", running; a text editor offers the font "Hello Sans"; the Security tab of C:\ProgramData\Hello shows the three entries. Remove Hello: the service, the font and the folder are gone.

12.7 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi ServiceInstall
HelloService HelloService Hello background service 16 2 1 NT AUTHORITY\LocalService
--service C_d677... Keeps Hello's greetings up to date.
C:\work\hello> rubrapack inspect hello.msi ServiceControl
HelloService HelloService 163 1 C_d677...
```

16 is "a service in its own process", 2 "automatic start", 1 "report errors normally". The 163 in `ServiceControl` is a sum of bit flags: 1 (start at install) + 2 (stop at install) + 32 (stop at removal) + 128 (delete at removal). Bit flags in Part III explains how such sums work. A package with permissions declares Windows Installer 5.0 (the summary's schema, 14 500 in `rubrapack inspect hello.msi --summary`), because the `MsiLockPermissionsEx` table is newer than the rest.

13 Running your program during installation

In this chapter

13.1	Why not just run a program?	43
13.2	The source	43
13.3	[action.ID]	43
13.4	What happens when	44
13.5	Try it	44
13.6	What happened inside	44

Goal: after its files are in place, run `hello.exe --register` so that Hello can register itself somewhere (with another program, a plug-in host, the system); before removal, run `hello.exe --unregister`. And if anything fails, undo it all.

13.1 Why not just run a program?

An installation is a transaction: if step 40 fails, Windows Installer undoes steps 1 to 39 and the computer is as before. A program you run is outside that - Windows cannot know how to undo what it did. So rubrapack asks you for *two* commands, one that does and one that undoes, and arranges that the right one runs whichever way things go.

13.2 The source

```
# tutorial 13: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[define]
VERSION = "1.11.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[action.Register]
run = "file:Hello"
do = "--register"
undo = "--unregister"
```

13.3 [action.ID]

Key	Meaning
run	the program: file: and the ID of an .exe this package installs
do	its arguments after installation - and again on a repair

Key	Meaning
undo	its arguments at removal, before the files are removed
check	accepted, but not used by this version of rubrapack

The arguments are passed as written: they are *not* formatted strings, so [INSTALLDIR] would arrive as the text [INSTALLDIR]. The program knows where it is (its own path), which is usually all it needs.

Both commands:

- must finish without asking anything: they run without a window, and nobody is there to answer;
- must be safe to run twice (register when already registered: fine);
- report failure by exiting with a non-zero code - which fails and rolls back the installation.

In a per-machine package they run with the installer's administrator rights; in a per-user package, as the user.

13.4 What happens when

Situation	Runs
first installation	do after the files are installed
repair	do again
removal	undo, then the files go
upgrade to a new version	the old version's undo (its removal), then the new version's do
installation fails after do ran	undo, as part of the rollback
removal fails after undo ran	do, and the files come back

13.5 Try it

A real program for this reads its arguments and, for example, writes a registry value or calls a registration interface. To watch the order, make `hello.exe` write its arguments into a log file, install, repair and remove, and read the log.

13.6 What happened inside

```
C:\work\hello> rubrapack inspect hello.msi CustomAction
RP_Register_Do 3090 Hello --register
RP_Register_DoRollback 3410 Hello --unregister
RP_Register_RedoRollback 3410 Hello --register
RP_Register_Undo 3090 Hello --unregister
RP_Register_UndoRollback 3410 Hello --register
```

Five *custom actions*. The type numbers are bit flags added together: 18 ("run an .exe installed by this package") + 1024 ("deferred": runs in the installation script, in order with the file copying) + 2048 ("no impersonation": with the installer's rights) = 3090; the rollback twins add 256 ("run only while rolling back") and 64 ("ignore the exit code"). The `InstallExecuteSequence` places them around the file actions, each with a condition on the state of `hello.exe`'s component:

```
RP_Register_UndoRollback $C_185f...=2 AND ?C_185f...=3 3400
RP_Register_Undo $C_185f...=2 AND ?C_185f...=3 3401
RemoveFiles 3500
InstallFiles 4000
RP_Register_DoRollback $C_185f...>2 AND ?C_185f...<>3 4001
RP_Register_RedoRollback $C_185f...>2 AND ?C_185f...=3 4002
RP_Register_Do $C_185f...>2 4003
```

$\$C$ is the state the component is going to (2 absent, 3 installed), $?C$ the state it is in now. "Going away while installed" runs undo; "coming" runs do.

14 Your own dialog pages

In this chapter

14.1	The source	46
14.2	A page: [dialog.ID]	48
14.3	Where things go on the page	49
14.4	Controls: [dialog-control.ID]	49
14.5	Every value needs a default: [property.NAME]	49
14.6	Try it	49
14.7	What happened inside	50

Goal: add a page "Options" after the install folder, where the user picks a greeting style, types a name, chooses a language from a list and decides about the desktop shortcut - and store the answers in the registry.

14.1 The source

```
# tutorial 14: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
ui = "installdir"

[define]
VERSION = "1.12.0"

[ui]
languages = ["ko"]

[dialog.Options]
after = "RpInstallDirDlg"
title = "Options"
title-ko = "선택 사항"
description = "Choose how Hello greets you."
description-ko = "Hello 가 인사하는 방식을 고르십시오."

[dialog-control.Modelabel]
dialog = "Options"
type = "text"
x = 20
y = 55
width = 330
height = 12
text = "Greeting &style:"
text-ko = "인사 방식(&S):"

[dialog-control.Mode]
dialog = "Options"
type = "radio"
x = 20
```

```

y = 70
width = 200
height = 42
property = "GREETING_STYLE"
values = ["friendly", "formal", "silent"]
labels = ["&Friendly", "F&ormal", "S&ilent"]
labels-ko = ["친근하게(&F)", "정중하게(&O)", "조용히(&I)"]

```

```

[dialog-control.NameLabel]
dialog = "Options"
type = "text"
x = 20
y = 122
width = 330
height = 12
text = "&Your name:"
text-ko = "이름(&Y):"

```

```

[dialog-control.Name]
dialog = "Options"
type = "edit"
x = 20
y = 136
width = 200
height = 18
property = "GREETING_NAME"

```

```

[dialog-control.LanguageLabel]
dialog = "Options"
type = "text"
x = 20
y = 164
width = 330
height = 12
text = "&Greeting language:"
text-ko = "인사 언어(&G):"

```

```

[dialog-control.Language]
dialog = "Options"
type = "combo"
x = 20
y = 178
width = 200
height = 16
property = "GREETING_LANGUAGE"
values = ["en", "ko"]
labels = ["English", "Korean"]
labels-ko = ["영어", "한국어"]

```

```

[dialog-control.DesktopBox]
dialog = "Options"
type = "checkbox"
x = 20
y = 204
width = 330
height = 16
property = "DESKTOP_SHORTCUT"
text = "Create a &desktop shortcut"
text-ko = "바탕화면 바로가기 만들기(&D)"

```

```

[property.GREETING_STYLE]

```

```

value = "friendly"

[property.GREETING_NAME]
value = "friend"

[property.GREETING_LANGUAGE]
value = "en"

[property.DESKTOP_SHORTCUT]
value = "1"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"

[registry.StyleValue]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Style"
value = "[GREETING_STYLE]"

[registry.NameValue]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Name"
value = "[GREETING_NAME]"

[registry.LanguageValue]
root = "HKLM"
key = 'SOFTWARE\Example Software\Hello'
name = "Language"
value = "[GREETING_LANGUAGE]"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[shortcut.Desktop]
dir = "Desktop"
name = "Hello"
target = "file:Hello"
when = "DESKTOP_SHORTCUT"

```

14.2 A page: [dialog.ID]

Key	Meaning
after	the page it follows: RpWelcomeDlg, RpLicenseDlg (with a license), RpInstallDirDlg (installdir, features), RpCustomizedDlg (features), or another page of yours
title, description	the heading and the line under it in the banner (default title: the product name)
title-xx, description-xx	the same in another language of [ui] languages

The page gets the same banner and Back / Next / Cancel buttons as the built-in ones; you fill its body. Several pages after the same page come in ID order. In minimal the last page's button says

Install; in the other sets the "ready" page always comes last. Pages work with minimal, installldir and features.

14.3 Where things go on the page

Positions and sizes are in *dialog units*, not pixels: Windows scales them to the font, so a page looks the same at any screen size. Every page is 370 x 270 units:

```

x=0                                     x=370
+-----+-----+ y=0
| Title (banner)                               |
| Description                                   |
+-----+-----+ y=44
|                                           | y=45
|   your controls go here: x from 0 to 370, y from 45 to 234   |
|                                           |
+-----+-----+ y=234
|                                           | y=243
|                               [ < Back ] [ Next > ]   [ Cancel ] |
+-----+-----+ y=270

```

Tab moves through your controls from top to bottom, then left to right.

14.4 Controls: [dialog-control.ID]

Every control names its dialog, a type, and x, y, width, height:

type	Shows	Property
text	a label (text)	-
checkbox	a tick box with a label	set to 1 when ticked, removed when not
edit	a line to type in	the text typed
radio	one button per value, one chosen	one of values
combo	a drop-down list	one of values

- values (1 to 32 strings) are what the property receives; labels what the user sees, in the same order (default: the values). labels-xx and text-xx translate them.
- A radio group needs about 12 units of height per value.
- In a label, & marks the Alt key: on a text in front of an edit, Alt+Y jumps to the edit.
- Control IDs Banner, Title, Description, BannerLine, BottomLine, Back, Next, Cancel belong to the frame; page IDs starting with Rp are rubrapack's. At most 64 controls a page.

14.5 Every value needs a default: [property.NAME]

The dialogs only *collect* values. A silent installation (/qn) shows no page and uses the properties as they are - so each property a control sets has a [property.*] with its default, and for radio and combo that default must be one of the values. The same properties can be given on the command line:

```
msiexec /i hello.msi /qn GREETING_STYLE=formal GREETING_NAME=Ada DESKTOP_SHORTCUT=""
```

rubrapack makes the properties of your controls secure by itself, so they reach the elevated part of the installation: [registry.StyleValue] writes [GREETING_STYLE], and the desktop shortcut's when = "DESKTOP_SHORTCUT" sees the tick box.

14.6 Try it

Install: after the folder page comes "Options" with your controls; Back returns to the folder; Next goes to "ready". With Korean chosen on the language page, the page and the labels are Korean. After

installing, regedit shows Style, Name and Language under HKLM\SOFTWARE\Example Software\Hello; the desktop shortcut exists only if the box was ticked.

14.7 What happened inside

The page is rows of the Control table - the frame's and yours - and its buttons are rows of ControlEvent:

```
C:\work\hello> rubrapack inspect hello.msi ControlEvent
...
RpInstallDirDlg Next NewDialog Options 1 1
Options Back NewDialog RpInstallDirDlg 1 1
Options Next NewDialog RpReadyDlg 1 1
RpReadyDlg Back NewDialog Options 1 1
...
```

"When Next on the folder page is pressed, show Options"; Options' Back and Next lead to the neighbours. rubrapack rewires the built-in pages to fit your page in. In Control, the attribute numbers are bit flags again (1 visible, 2 enabled, 65536 transparent, ...).

15 Cabinets, compression, big packages, and several architectures

In this chapter

15.1	The folder	51
15.2	The source	51
15.3	Cabinets and compression	52
15.4	Several architectures: --arch	52
15.5	The same bytes every time: --reproducible	52
15.6	Names from a Mac: --nfc	53
15.7	Variables from the command line: -D	53
15.8	What happened inside	53

Goal: control how the files are packed - compression, one package file or package plus cabinets - build Hello for x64, x86 and Arm64 from one source, and make builds byte-for-byte repeatable.

15.1 The folder

The program now exists in three builds, one per processor type:

```
dist\
  x64\hello.exe
  x86\hello.exe
  arm64\hello.exe
docs\guide.txt
```

15.2 The source

```
# tutorial 15: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
upgrade-code-x86 = "{7D1C2B3A-4E5F-4061-9728-3A4B5C6D7E8F}"
upgrade-code-arm64 = "{0E9F8D7C-6B5A-4948-8372-6150F4E3D2C1}"
compress = "mszip:9"
cab = "external"
cab-max-size = 500

[define]
VERSION = "2.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/${ARCH}/hello.exe"
```

```
[files.DocFiles]
dir = "INSTALLDIR"
glob = "dist/docs/**"
```

15.3 Cabinets and compression

Inside an MSI the files are not stored one by one: they are packed into a *cabinet* (.cab), the compressed archive format Windows has used since the 1990s. Keys in [package]:

Key	Values	Effect
compress	mszip:6 (default), mszip:0 .. mszip:9, mszip (= 6), none	how hard to compress. 9 is smallest and slowest, 1 fastest; none stores the files as they are
cab	embed (default), external	the cabinet inside the .msi, or next to it as hello-x64.cab
cab-max-size	a number of MiB	start a new cabinet after that much data (-1.cab, -2.cab, ...)

For most programs the defaults are right: one .msi holding everything. External cabinets are for very large products: **an MSI of 2 GiB or more cannot be opened by Windows Installer**, so a product that big must use cab = "external" (rubrapack refuses the embedded form with RP1516), and the .msi and its .cab files must then be kept together - in the same folder, on the same share. Without cab-max-size, external cabinets are split below 2 GiB each.

Compression runs on every processor of your computer; --jobs 2 uses at most two. On a 200 MB program with 16 processors, mszip:6 takes about 5 seconds and makes 105 MB.

15.4 Several architectures: --arch

arch = "x64" is the source's own architecture. --arch x86 builds the same source for another one:

```
C:\work\hello> rubrapack build hello.toml -o out-x64\hello-x64.msi --arch x64
C:\work\hello> rubrapack build hello.toml -o out-x86\hello-x86.msi --arch x86
C:\work\hello> rubrapack build hello.toml -o out-arm64\hello-arm64.msi --arch arm64
```

- \$(ARCH) is a built-in variable: x64, x86 or arm64, the architecture being built - here it picks dist/x86/hello.exe for the x86 package. rubrapack checks each program against it.
- ProgramFiles follows: C:\Program Files for x64 and Arm64, C:\Program Files (x86) for x86.
- **Each architecture needs its own upgrade code** - upgrade-code-x86, upgrade-code-arm64 (and upgrade-code-x64 if the source's own arch were another). Windows' upgrade logic cannot tell architectures apart: with one shared code, installing the x86 package would remove the x64 one as "an older version". Without them, --arch is refused.

Each output goes to its own folder here because external cabinets are named after the package and must not collide.

15.5 The same bytes every time: --reproducible

A normal build gives every package a new random *package code* (a GUID in the file's summary), so two builds of the same source differ. --reproducible derives it from the content instead: the same source and files give the same .msi, byte for byte, on any computer, with any number of processors:

```
C:\work\hello> rubrapack build hello.toml -o r1.msi --reproducible --jobs 1
C:\work\hello> rubrapack build hello.toml -o r2\1.msi --reproducible
C:\work\hello> fc /b r1.msi r2\1.msi
FC: no differences encountered
```

Anyone can then check that a published package was built from the published source. The package code is shown by `rubrapack inspect hello.msi --summary` (property 9).

15.6 Names from a Mac: `--nfc`

Unicode can write a character like `한` or `é` in two ways - one code point, or letters plus combining marks - and macOS file systems store names in the second form. `--nfc` converts file and folder names to the first form (NFC), which Windows uses; `lint` warns (RP2105) about names that are not NFC. Part III explains Unicode normalization.

15.7 Variables from the command line: `-D`

`-D NAME=VALUE` sets any [define] variable, as often as needed: `-D VERSION=2.0.1 -D EDITION=Pro. $ $(` writes a literal `$` (.

15.8 What happened inside

```
C:\work\hello> rubrapack inspect out-x64\hello-x64.msi Media
1 2    hello-x64.cab
C:\work\hello> rubrapack inspect out-x86\hello-x86.msi --summary
...
7 Intel;1033
```

The Media table lists the cabinets: disk 1 holds files 1 to 2, in `hello-x64.cab` (an embedded one is written `#cab1.cab`, the `#` meaning "inside this file"). The summary's *template* (property 7) names the platform (x64, Arm64, or Intel for x86) and the language. Part IV shows the cabinet format - CFHEADER, CFFOLDER, CFDATA - and the MSZIP compression byte by byte.

16 Signing and timestamps

In this chapter

16.1	What a signature is	54
16.2	Signing with a key file	54
16.3	Timestamps: signatures that outlive the certificate	55
16.4	A key in the Windows certificate store	55
16.5	A hardware token: PKCS#11	55
16.6	Checking a signature: verify	56
16.7	External cabinets are not covered	56
16.8	What happened inside	56

Goal: sign `hello.msi` so that Windows shows "Example Software" as its publisher instead of "Unknown publisher", and so that nobody can change it unnoticed - with a key file, with a key in the Windows certificate store, or with a hardware token.

16.1 What a signature is

A *digital signature* is a number computed from the file's contents and a secret *private key*. Anyone can check it with the matching *public key*, which is inside a *certificate*: a small document, itself signed by a *certificate authority* (CA), that says "this public key belongs to Example Software". Windows trusts a list of CAs; a certificate from one of them makes your name appear in the User Account Control prompt. If one byte of the package changes, the signature no longer matches. Part III explains hashes and keys, certificates and signatures step by step.

Where the certificate comes from:

- **For real users:** a *code signing certificate* bought from a CA. Since 2023 the private key must live in hardware - a USB token or a cloud service - and never in a file. rubrapack signs through such a token (PKCS#11) or through the Windows certificate store, below.
- **For learning and testing:** a certificate you make yourself. Windows does not trust it (the prompt still says "Unknown publisher" unless you add it to the trusted list on the test computer), but everything else works the same. In PowerShell on Windows:

```
PS> New-SelfSignedCertificate -Type CodeSigningCert -Subject "CN=Example Software" -
CertStoreLocation Cert:\CurrentUser\My
```

It prints the certificate's *thumbprint*, 40 hexadecimal digits that name it.

16.2 Signing with a key file

A key file holds the private key and the certificate, protected by a password: a `.pfx` (also called `.p12`), as Windows exports it (`Export-PfxCertificate` with `-CryptoAlgorithmOption AES256_SHA256`), or a PEM key. The password is never typed on the command line (other programs could read it there): it comes from an environment variable or a file. The options that would take one are there only to say so:

```
C:\work\hello> rubrapack sign hello.msi --key signer.pfx --pass secret
rubrapack: error[RP0004]: a password is never taken on the command line (others can see it);
use --pass-env or --pass-file
```

(--pin for a token's PIN answers the same way.)

```
C:\work\hello> set SIGN_PASS=...the password...
C:\work\hello> rubrapack sign hello.msi --key signer.pfx --pass-env SIGN_PASS -o hello-
signed.msi
rubrapack: warning[RP0011]: 'hello.msi': no --timestamp: the signature stops being valid when
the certificate expires
signed hello-signed.msi
```

Without -o the file is signed in place. A file that is signed already is refused - sign the unsigned file again instead. The certificate must be one for code signing, not a CA's, and valid today. --pass-file pass.txt reads the password from a file instead. --cert chain.pem adds certificates the key file does not hold (the CA's intermediate certificates, which Windows needs to connect your certificate to a trusted root).

Signing while building does the same in one step:

```
C:\work\hello> rubrapack build hello.toml -o hello.msi --key signer.pfx --pass-env SIGN_PASS
```

sign also signs programs (.exe, .dll) - sign hello.exe before you build, so the program itself carries your name too - and MSIX packages (next chapter).

16.3 Timestamps: signatures that outlive the certificate

A certificate is valid for a year or three. The warning above says what happens after that: the signature no longer counts. A *timestamp* fixes it: a timestamp server (run by the CAs, free to use) signs "this signature existed at this time", and the signature then stays valid for good.

```
C:\work\hello> rubrapack sign hello.msi --key signer.pfx --pass-env SIGN_PASS --timestamp
http://timestamp.digicert.com
```

Always timestamp what you publish. Options for unusual networks: --proxy http://host:port, --tls-trust (certificates to trust for an https:// timestamp server), --system-roots (trust the computer's own root certificates for that connection), --tsa-trust (certificates to trust for the timestamp itself).

16.4 A key in the Windows certificate store

The self-made certificate above is in your certificate store. Sign with it by thumbprint - on Windows only:

```
C:\work\hello> rubrapack keys list
CurrentUser\My thumbprint=1a2b... RSA publisher=CN=Example Software until=...
C:\work\hello> rubrapack sign hello.msi --key-store 1a2b...
```

--machine-store looks in the computer's store (LocalMachine) instead of yours. Keys on a USB token whose driver plugs into Windows also appear here.

16.5 A hardware token: PKCS#11

A token's maker supplies a PKCS#11 *module* - a DLL (or .so on Linux) that speaks for the token. rubrapack talks to it directly:

```
C:\work\hello> rubrapack keys list --pkcs11 C:\token\pkcs11.dll --pin-env TOKEN_PIN
C:\work\hello> rubrapack sign hello.msi --pkcs11 C:\token\pkcs11.dll --key-label "Code Signing"
--pin-env TOKEN_PIN --timestamp http://timestamp.digicert.com
```

--key-label picks the key by its label, --token-label the token when several are plugged in, --pin-env/--pin-file give the PIN (never on the command line).

16.6 Checking a signature: verify

```
C:\work\hello> rubrapack verify hello-signed.msi --trust ca.pem
hello-signed.msi
  structure: ok
  digest:    ok
  signature: ok
  chain:     trusted
  revocation: not-checked
  timestamp: not-present
```

- structure: the signature is where it belongs; digest: the contents are unchanged; signature: it was made with the certificate's key;
- chain: the certificate leads to a certificate you trust - given with --trust, or --system-roots for the computer's trusted roots. Without either, verify says it cannot tell (and exits with 4);
- timestamp: present and valid, or not.

An unsigned file says structure: invalid ... the package is not signed.

16.7 External cabinets are not covered

A signature covers the .msi only. With cab = "external" (chapter 15) someone could swap the .cab files unnoticed, so sign refuses such a package:

```
C:\work\hello\out-x64> rubrapack sign hello-x64.msi --key signer.pfx --pass-env SIGN_PASS
rubrapack: error[RP0011]: 'hello-x64.msi': the package uses cabinets outside itself, which a
signature of the .msi does not cover; build it with embedded cabinets, or sign anyway with --
allow-unsigned-cabs
```

--allow-unsigned-cabs signs the .msi anyway, with a warning. Prefer embedded cabinets for signed packages.

16.8 What happened inside

An MSI's signature is a hidden stream, \005DigitalSignature, holding a PKCS#7 structure: the hash of every other stream, your certificate chain, and the signature over it. Part IV (Authenticode) shows how the hash is computed and where each part lies.

17 MSIX packages and bundles

In this chapter

17.1	MSI or MSIX?	57
17.2	The folder	57
17.3	The source	58
17.4	The package's identity: [msix]	59
17.5	The application: [msix-app.ID]	59
17.6	Extras: [msix-extension.ID]	59
17.7	What the other tables become	59
17.8	Building for testing: --unsigned-test	60
17.9	Signing it	60
17.10	All architectures in one file: a bundle	60
17.11	Compression	61
17.12	What happened inside	61

Goal: build Hello as an MSIX package too - the newer package format of Windows - from the same source, then as one bundle that holds all three architectures.

17.1 MSI or MSIX?

	MSI (.msi)	MSIX (.msix)
Installs	anywhere the package says, and changes the computer as told	into a sealed folder Windows owns; files and registry the app writes are kept apart
Removal	as clean as the package makes it	always complete: nothing is left behind
Signature	recommended	required: Windows does not install an unsigned MSIX (except for testing, below)
Can do	everything in this tutorial	files, shortcuts, file types, registry, fonts, a command-line alias, a start-at-sign-in task
Cannot do	-	services, custom actions, environment variables, INI files, permissions, conditions, dialogs

Many products ship both. rubrapack builds either from one source: the output's extension decides.

17.2 The folder

Three logos go next to the source, in `assets\`: PNG pictures of exactly 150x150, 44x44 and 50x50 pixels. The Start menu and the Settings app show them. Without them the package gets plain one-colour logos - give all three or none.

```
C:\work\hello\
  assets\
    Square150x150.png
    Square44x44.png
    StoreLogo.png
  dist\
    x64\hello.exe
```

```

x86\hello.exe
arm64\hello.exe
docs\guide.txt
hello.toml

```

17.3 The source

```

# tutorial 17: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
upgrade-code-x86 = "{7D1C2B3A-4E5F-4061-9728-3A4B5C6D7E8F}"
upgrade-code-arm64 = "{0E9F8D7C-6B5A-4948-8372-6150F4E3D2C1}"

[define]
VERSION = "2.1.0"

[msix]
identity-name = "ExampleSoftware.Hello"
publisher = "CN=Example Software"
publisher-display-name = "Example Software"
min-version = "10.0.17763.0"

[msix-app.Hello]
executable = "Hello"
display-name = "Hello"
description = "Says hello."
logo-150 = "assets/Square150x150.png"
logo-44 = "assets/Square44x44.png"
store-logo = "assets/StoreLogo.png"

[msix-extension.Command]
kind = "alias"
alias = "hello.exe"

[msix-extension.AtSignIn]
kind = "startup-task"
display-name = "Hello"
enabled = false

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/${ARCH}/hello.exe"

[files.DocFiles]
dir = "INSTALLDIR"
glob = "dist/docs/**"

[shortcut.StartMenu]
dir = "Programs"
name = "Hello"
target = "file:Hello"

[assoc.HelloDoc]

```

```

extension = ".hello"
prog-id = "ExampleSoftware.HelloDocument"
description = "Hello document"
target = "file:Hello"

[registry.Greeting]
root = "HKMU"
key = "Software\\Example Software\\Hello"
name = "Greeting"
value = "Hello"

[env.HelloHome]
name = "HELLO_HOME"
value = "[INSTALLDIR]"
msi-only = true

```

17.4 The package's identity: [msix]

Key	Meaning
identity-name	the package's name for Windows, 3-50 characters of A-Z a-z 0-9 . - ; by custom Company.Product. Like the upgrade code of an MSI it must never change: a new version with the same name and publisher replaces the old one.
publisher	the <i>subject</i> of the certificate you will sign with, written as Windows writes it (next section)
publisher-display-name	the publisher's name people see; the default is [package] manufacturer
min-version	the oldest Windows it installs on, as 10.0.<build>.0; the default 10.0.17763.0 is Windows 10 version 1809

The version is [package] version with a fourth part: 2.1.0 becomes 2.1.0.0.

17.5 The application: [msix-app.ID]

An MSIX lists the *applications* it contains - the entries in the Start menu. executable names the [file.*] that starts it; the folder of that file (here INSTALLDIR) becomes the package's own folder. display-name and description default to the package's name. Several [msix-app.*] tables give several entries.

17.6 Extras: [msix-extension.ID]

- kind = "alias": typing hello.exe in a console starts the application, from any folder - the MSIX way of putting a program on the PATH.
- kind = "startup-task": the application starts when the user signs in, once it has been run once. enabled = false leaves it off until the user turns it on in Task Manager's Startup apps, which shows display-name. task-id names the task for Windows; the default is the table's ID (here AtSignIn, as the manifest below shows).

With several [msix-app.*] tables, app = "Hello" says which application an extension belongs to; the default is the first. An MSI build leaves these tables out.

17.7 What the other tables become

- [shortcut.StartMenu] in Programs to the application's executable is the application's Start menu entry. A shortcut on the Desktop also works, from min-version = "10.0.19645.0" on. Shortcuts elsewhere, with arguments or with an icon, are MSI only (RP1613).

- `[assoc.HelloDoc]` becomes a file type association of the application; the application's logo stands for the file type.
- `[registry.Greeting]` goes into the package's *virtual registry*: the application sees the value as if it were in the real registry, while the computer's registry stays untouched. Only keys under `Software` can go there (RP1612).
- `[env.HelloHome]`: an MSIX cannot set environment variables. Without `msi-only = true` the build stops:

```
C:\work\hello> rubrapack build hello.toml -o hello.msix --unsigned-test
hello.toml:65:1: error[RP1605]: [env.HelloHome] cannot go into an MSIX; add msi-only = true to
build the MSIX without it
```

`msi-only = true` keeps the table in the MSI and leaves it out of the MSIX. rubrapack never drops something silently: what an MSIX cannot hold is an error until you say what you want.

Features, properties, dialogs and `[arp]` concern Windows Installer only and are not used. rubrapack `lint hello.toml --target msix` checks a source for MSIX without building it.

17.8 Building for testing: --unsigned-test

```
C:\work\hello> rubrapack build hello.toml -o hello.msix --unsigned-test
C:\work\hello> rubrapack build hello.toml -o hello.msi
```

Windows installs an unsigned MSIX only when the package says it is meant for testing and the installer says it accepts that. `--unsigned-test` adds that mark (it changes the publisher, so the package is a different package from the signed one). Install it in PowerShell, as administrator because it contains a program:

```
PS C:\work\hello> Add-AppxPackage -Path hello.msix -AllowUnsigned
PS C:\work\hello> hello.exe
PS C:\work\hello> Get-AppxPackage ExampleSoftware.Hello | Remove-AppxPackage
```

Such a package is for your own test computer only.

17.9 Signing it

For people to install it, sign it (chapter 16) - the MSIX's publisher must be exactly the certificate's subject. Otherwise signing stops and prints the subject to use:

```
C:\work\hello> rubrapack build hello.toml -o hello.msix --key signer.pfx --pass-env SIGN_PASS
rubrapack: error[RP0011]: 'hello.msix': the package's publisher "CN=Example Software" is not
the certificate's subject "O=Example Software Ltd, CN=Example Software"; set [msix] publisher
to it (without --unsigned-test)
```

Copy that subject into publisher. Windows writes a subject's parts from the last to the first, so it differs from how some tools show it.

17.10 All architectures in one file: a bundle

An output ending in `.msixbundle` builds the source once for each architecture of `--arch`, as in chapter 15, and puts the packages in one file:

```
C:\work\hello> rubrapack build hello.toml -o hello.msixbundle --arch x64,x86,arm64 --unsigned-
test
C:\work\hello> rubrapack inspect hello.msixbundle --files
AppxMetadata\AppxBundleManifest.xml 1616 deflate
ExampleSoftware.Hello_2.1.0.0_x64.msix 11009 stored
```

```
ExampleSoftware.Hello_2.1.0.0_x86.msix 11096 stored
ExampleSoftware.Hello_2.1.0.0_arm64.msix 11012 stored
```

Windows installs the package for its own processor from it. With `--key` the bundle and each package in it are signed.

17.11 Compression

Files are compressed with deflate; pictures and other files that are compressed already are stored as they are. `--msix-compress` store stores everything, which makes the package bigger (here 11009 to 29961 bytes) but quicker to open. An MSIX holds no date or time: the same source gives the same bytes on every computer.

17.12 What happened inside

An MSIX is a ZIP archive. Look at its list of files:

```
C:\work\hello> rubrapack inspect hello.msix --files
hello.exe 17920 deflate
guide.txt 6 deflate
Assets\Square150x150.png 301 stored
Assets\Square44x44.png 111 stored
Assets\StoreLogo.png 117 stored
Registry.dat 8192 deflate
AppxManifest.xml 2537 deflate
```

Registry.dat is the virtual registry - a registry *hive* file. AppxManifest.xml describes the package; `--manifest` prints it:

```
C:\work\hello> rubrapack inspect hello.msix --manifest
...
  <Identity Name="ExampleSoftware.Hello" Publisher="CN=Example Software,
OID.2.25.311729368913984317654407730594956997722=1" Version="2.1.0.0"
ProcessorArchitecture="x64" />
...
  <Application Id="Hello" Executable="hello.exe" EntryPoint="Windows.FullTrustApplication">
...
    <uap3:FileTypeAssociation Name="examplesoftware.hellodocument" Parameters="&quot;
%1&quot;">
...
    <desktop:StartupTask TaskId="AtSignIn" Enabled="false" DisplayName="Hello" />
...
    <desktop:ExecutionAlias Alias="hello.exe" />
...
  <rescap:Capability Name="runFullTrust" />
```

The `OID.2.25...=1` part of the publisher is the test mark `--unsigned-test` added. `runFullTrust` means the program runs like any desktop program, not in the restricted sandbox of store apps. Two more entries are in every package, though `--files` does not list them: `AppxBlockMap.xml`, the hash of every 64 KB block of every file, which is what a signature covers, and `[Content_Types].xml`, the type of each file. A signed package also has `AppxSignature.p7x`, the signature. Part IV shows the ZIP layout and the block map byte by byte.

18 Checking and looking inside

In this chapter

18.1	Checking a source: lint	62
18.2	Checking a package: lint <file>	63
18.3	Looking at the tables: inspect	63
18.4	Files and streams	64
18.5	Unpacking without installing: extract	65
18.6	Signatures: verify	65
18.7	MSIX packages	65
18.8	Faster test builds: --compress none	65
18.9	GUIDs: guid	65
18.10	Help on the command line	66

Goal: catch mistakes before users do - check a source or a finished package, look at every table, file and stream in it, unpack it without installing, and check its signature. These commands also work on packages that other tools made.

This chapter uses the `hello.toml` of chapter 17 and the `hello.msi` built from it.

18.1 Checking a source: lint

`lint` runs every check build runs, and writes nothing:

```
C:\work\hello> rubrapack lint hello.toml
C:\work\hello> rubrapack lint hello.toml --target msix
C:\work\hello> rubrapack lint hello.toml --arch x86 -D VERSION=2.2.0
```

No output and exit code 0 means no problem. `--target msix` checks the source for an MSIX instead of an MSI; `--arch` and `-D` check it as build with the same options would build it.

Warnings do not stop a build: something is probably not what you meant, but the package works. `--strict` makes them fail too, which is what an automatic build wants. Suppose the product's name is Korean and there is no `summary-name` (chapter 5):

```
C:\work\hello> rubrapack lint ko.toml
ko.toml:1:1: warning[RP1203]: name is not ASCII and summary-name is missing; the summary
Subject will read 'rubrapack package'
C:\work\hello> echo %errorlevel%
0
C:\work\hello> rubrapack lint ko.toml --strict
ko.toml:1:1: warning[RP1203]: name is not ASCII and summary-name is missing; the summary
Subject will read 'rubrapack package'
C:\work\hello> echo %errorlevel%
5
```

`%errorlevel%` is how `cmd` shows the exit code of the last command; in PowerShell it is `$LASTEXITCODE`. The exit codes:

Code	Meaning
0	success
1	an error in the source

Code	Meaning
2	a command line rubrapack does not understand
3	a file could not be read or written
4	signing or a signature
5	lint found a problem (or a warning, with --strict)
6	the network (a timestamp server)

18.2 Checking a package: lint <file>

```
C:\work\hello> rubrapack lint hello.msi
hello.msi: 0 errors, 0 warnings
C:\work\hello> rubrapack lint hello.msix
hello.msix: 7 files, 0 errors, 0 warnings
C:\work\hello> rubrapack lint hello.msixbundle
hello.msixbundle: 4 files, 0 errors, 0 warnings
```

For an MSI it checks the tables with the rules Windows Installer applies: what would stop an installation is an error, anything else a warning. A package made by another tool may well have warnings. Here a product name was typed in a program that stores accented letters in two pieces (see Unicode normalization in Part III):

```
C:\work\hello> rubrapack lint cafe.msi
cafe.msi: warning[RP2105]: lint: Feature row 'Main': column Title is not in NFC (1 row); build
--nfc puts folder, file and shortcut names in NFC
cafe.msi: warning[RP2105]: lint: Property row 'ProductName': column Value is not in NFC (1
row); build --nfc puts folder, file and shortcut names in NFC
cafe.msi: 0 errors, 2 warnings
```

For an MSIX it checks what Windows checks before installing: the ZIP archive, the block map, the hash of every block, and that the manifest names files that are there. `lint new.msi --previous old.msi` checks an upgrade (chapter 3).

18.3 Looking at the tables: inspect

An MSI is a small database. `inspect` with a table's name prints that table:

```
C:\work\hello> rubrapack inspect hello.msi File
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
F_6d1508f52f6615459567 C_a10be66379e69940a2e0 guide.txt 6 512 1
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 2
```

This is Windows Installer's own text format for tables (*IDT*), which its tools read too. The first three lines are the header:

1. the column names;
2. each column's type: `s72` a string of up to 72 characters, `l255` a *localizable* string, `i4` a 4-byte integer, `i2` a 2-byte one; a capital letter (`S72`, `I2`) means the column may be empty;
3. the table's name, then its key columns - the ones that make a row unique.

Every row after it is one file. `hello.exe` has a version and a language (1033 is English (United States)) because it is a program with a version resource; `guide.txt` has none. Attributes 512 is the bit "vital" (chapter 4: the installation fails if the file cannot be written). Part IV explains every type and how the rows are stored.

Without a table name, `inspect` lists every table with its row and column counts, and the *summary information*, the few facts Windows shows in a file's properties:

```
C:\work\hello> rubrapack inspect hello.msi
code page: 65001
strings: 156
tables: 19
  AdminExecuteSequence  rows=8 columns=3
  ...
  File  rows=2 columns=8
  ...
summary:
  2 = Installation Database
  3 = Hello
  4 = Example Software
  5 = Installer
  7 = x64;1033
  9 = {7FC806A2-F359-4941-A159-26A77B78F839}
  14 = 200
  15 = 2
  18 = rubrapack 0.7.0
```

The numbers of the summary are property IDs: 2 title, 3 subject, 4 author, 5 keywords, 7 the platform and languages, 9 the *package code* (a new GUID for every build), 14 the Windows Installer version needed (200 = 2.0), 15 word flags (2 = compressed), 18 the program that made it. `--summary` prints only them.

18.4 Files and streams

```
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\guide.txt  6 F_6d1508f52f6615459567  C_a10be66379e69940a2e0
2b6739281f9372442df30e784b7dbdc7
[ProgramFiles64Folder]\Hello\hello.exe  17920 Hello C_185f8db32271fe25f561  1.2.3.4 1033
```

One line per file: where it installs, size, file ID, component, version, language, and the MD5 hash Windows uses to decide whether a file without a version needs replacing (a program is compared by its version instead, so it has none).

```
C:\work\hello> rubrapack inspect hello.msi --streams
table  40 File
stream 6821 cab1.cab
table  14 Media
table  728 _Columns
...
stream 332 !SummaryInformation
```

A *stream* is a file inside the package (Part IV: the package is a file system in a file). Each table is one stream; `cab1.cab` holds the compressed files; `!SummaryInformation` the summary.

A cabinet by itself can be inspected too - the external one of chapter 15:

```
C:\work\hello> rubrapack inspect out-x64\hello-x64.cab
6 F_6d1508f52f6615459567
17920 Hello
```


18.5 Unpacking without installing: extract

```
C:\work\hello> rubrapack extract hello.msi -d unpacked
unpacked: 2 files, 17926 bytes
```

The files come out as they would install, below the names of the standard folders:

```
unpacked\ProgramFiles64Folder\Hello\guide.txt
unpacked\ProgramFiles64Folder\Hello\hello.exe
```

extract works on .msi, .cab, .msix and .msixbundle, and is safe on packages from anywhere: the target folder must be new or empty; every file is checked against its size and hash; names that would write outside the folder (., a drive, CON) are refused; and nothing is written until the whole package has passed. Limits guard against a package made to fill your disk:

```
C:\work\hello> rubrapack extract hello.msi -d small --limit-entries 2
rubrapack: error[RP0008]: 'hello.msi': 4 entries, more than the limit 2 (--limit-entries);
nothing was extracted
C:\work\hello> rubrapack extract hello.msi -d small --limit-bytes 1000
rubrapack: error[RP0008]: 'hello.msi': more than 1000 bytes to write (--limit-bytes); nothing
was extracted
```

The defaults are 100,000 entries and 16 GiB. (The four entries are the two files and the two folders, ProgramFiles64Folder and Hello.)

18.6 Signatures: verify

Chapter 16 showed verify. It is the check to run on what you are about to publish:

```
C:\work\hello> rubrapack verify hello.msi --system-roots
```

18.7 MSIX packages

```
C:\work\hello> rubrapack inspect hello.msix
C:\work\hello> rubrapack inspect hello.msix --files
C:\work\hello> rubrapack inspect hello.msix --manifest
```

The first shows the identity, the executable and the files, after checking every block's hash. Chapter 17 showed the other two.

18.8 Faster test builds: --compress none

Compression takes most of a build's time. While you try things out, leave it out:

```
C:\work\hello> rubrapack build hello.toml -o fast.msi --compress none
```

The package is bigger (here 45056 bytes instead of 32768) and works the same.

18.9 GUIDs: guid

```
C:\work\hello> rubrapack guid
{98AE4FED-BF0B-4C77-B712-E0649EB47178}
C:\work\hello> rubrapack guid --from hello
{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}
```

The first is random - new every time - for an upgrade code. The second is the GUID rubrapack *derives* from a text; it is the same on every computer, every time. rubrapack derives component GUIDs this way, which is why they stay the same from version to version. Part III shows the calculation.

18.10 Help on the command line

```
C:\work\hello> rubrapack help
rubrapack 0.7.0 - build Windows Installer (.msi) and MSIX (.msix) packages

usage: rubrapack <command> [arguments]

commands:
  build    build a package from a source file
  sign     sign a PE file, an MSI package, an MSIX package or bundle
  ...
C:\work\hello> rubrapack help sign
usage: rubrapack sign <file.exe|.dll|.msi|.msix|.msixbundle> (--key <key.pfx|.pem> ...)
C:\work\hello> rubrapack version
rubrapack 0.7.0 (proven_c_lib-v0.1.1)
```

`rubrapack --help` is the same as `rubrapack help`. `help <command>` prints that command's options

- the short form of Part II, which has them all with their meaning.

19 Starting fast and automating

In this chapter

19.1	A source from answers: new	67
19.2	Changing a source: edit	68
19.3	A release script	69
19.4	On a build server	69
19.5	With an AI assistant	69
19.6	Where to go from here	70

Goal: let rubrapack write a first source for you and change it without opening an editor, then turn every step of this tutorial into a script that builds, checks and signs a release by itself - on your computer, on a build server, or through an AI assistant.

19.1 A source from answers: new

In a folder that has the program's files in dist\:

```
C:\work\hello2> rubrapack new hello.toml
rubrapack new: a few questions make the source (Enter takes the value in [ ]).
Product name [hello]: Hello
Manufacturer (shown in Installed apps) [Hello authors]: Example Software
Version [1.0.0]:
Folder with the files to install [dist]:
  2 files and 2 folders there
Main program, for shortcuts (- for none) [hello.exe]:
Architecture (x64, x86, arm64) [x64]:
Folder name under Program Files [Hello]:
Install for (machine, user, dual) [machine]:
  sub folders: docs samples
Optional parts the user may tick (sub folders, commas; - for none) [-]: samples
Dialogs (none, basic, minimal, installdir, features) [features]:
License to accept (.txt, .md, .rtf; - for none) [LICENSE.txt]:
Korean dialogs as well as English? [y/N]: y
Start menu shortcut? [Y/n]:
Desktop shortcut? [y/N]: y
wrote hello.toml
the same without questions:
  rubrapack new "hello.toml" --name "Hello" --manufacturer "Example Software" --version "1.0.0"
--arch "x64" --dist "dist" --main "hello.exe" --install-dir "Hello" --scope "machine" --ui
"features" --license "LICENSE.txt" --languages "ko" --optional "samples" --shortcuts
"start,desktop"
lint: no problems
```

Every question is something a chapter of this tutorial explained, and the value in [] is what Enter takes. The offers follow the answers: after the name Hello the manufacturer offered is Hello authors, and once there are optional parts the dialogs offered are features.

The source it writes is an ordinary source - with comments that say what each line does, a fresh upgrade code, a [file.*] for each file at the top of the folder and a glob for each sub folder, and a [feature.*] for each optional one:

```
[feature.Main]
title = "Hello"
required = true                # always installed

[feature.samples]
title = "samples"
level = 2                     # offered, not ticked by default
...
[files.samples_files]
dir = "samples_dir"
glob = "dist/samples/**"
```

From here on you change it by hand, as in chapters 2-18, or with `edit`. `new` never replaces a file that is there: `new hello.toml` a second time says to use `edit`.

Three other forms:

- The line `new` prints last - `rubrapack new hello.toml --name ... --shortcuts ...` - gives the same answers without asking, for scripts. A left-out option takes the default the question would offer.
- `rubrapack new hello` (a name without `.toml`, and no options) writes a short starter source without asking anything, for filling in by hand.
- `rubrapack new hello -i` (or `--interactive`) asks the questions. They go to the error output and the answers are read one per line, so a file of answers can be piped in: `rubrapack new hello.toml < answers.txt`. If the answers run out before the last question, nothing is written.

19.2 Changing a source: `edit`

`edit` without options shows a menu:

```
C:\work\hello2> rubrapack edit hello.toml
hello.toml: "Hello" 1.0.0, x64, machine, dialogs features
  1 name, manufacturer, version, architecture    2 install folder, who it installs for
  3 dialogs, license, languages                  4 optional parts
  5 shortcuts                                    6 files: match the program folder
  7 any key                                     v view  l lint  s save  q quit
Choose [q]:
```

Each choice asks its questions again, offering the current values. 7 changes any key of any table. `v` shows the source, `l` checks it, `s` saves and checks, `q` leaves (and asks first if something changed). Only the values you changed are rewritten: your comments, your order, and the tables you wrote by hand stay as they are.

With options, `edit` asks nothing:

```
C:\work\hello2> rubrapack edit hello.toml --set define.VERSION=1.1.0 --set package.summary-
name=Hello --unset package.license
wrote hello.toml
lint: no problems
```

- `--set table.key=value` sets a key. A value written as TOML - "text", a number, true, ["ko"] - is used as it is; anything else is taken as text. A table that is not there is added.
- `--unset table.key` removes a key.

After the program changed - here `readme.txt` is gone and `changes.txt` is new - `--sync` matches the file list to the folder:

```
C:\work\hello2> rubrapack edit hello.toml --sync
removed [file.readme_txt]
```

```
added [file.changes_txt]
wrote hello.toml
lint: no problems
```

19.3 A release script

Everything so far is a command with no state outside the source, so a release is a short script. A PowerShell script, `release.ps1`, next to `hello.toml`:

```
param([Parameter(Mandatory)] [string] $Version)
$ErrorActionPreference = "Stop"
function Run { & rubrapack @args; if ($LASTEXITCODE -ne 0) { throw "rubrapack $args failed
($LASTEXITCODE)" } }

Run lint hello.toml --strict -D VERSION=$Version
foreach ($arch in "x64", "x86", "arm64") {
    Run build hello.toml -o "out\hello-$Version-$arch.msi" -D VERSION=$Version --arch $arch --
reproducible `
        --key-store $env:SIGN_THUMBPRINT --timestamp http://timestamp.digicert.com
    Run lint "out\hello-$Version-$arch.msi" --strict --previous "released\hello-$arch.msi"
    Run verify "out\hello-$Version-$arch.msi" --system-roots
}
Run build hello.toml -o "out\hello-$Version.msixbundle" -D VERSION=$Version --arch
x64,x86,arm64 `
    --key-store $env:SIGN_THUMBPRINT --timestamp http://timestamp.digicert.com
```

```
PS C:\work\hello> .\release.ps1 -Version 2.2.0
```

`released\` holds the packages of the version before, one per architecture, so `--previous` checks each upgrade (chapter 3). The script stops at the first command that fails, because `rubrapack`'s exit code is not 0 (chapter 18). The thumbprint comes from an environment variable, so the script holds no secret and can be kept with the source.

19.4 On a build server

`rubrapack` runs on Linux too, and builds the same bytes there with `--reproducible`, so a build server (GitHub Actions, GitLab CI, Jenkins) does not need Windows. On a Linux runner:

```
V=0.7.0
curl -sLo rubrapack "https://github.com/rubidus-api/rubrapack/releases/download/v$V/rubrapack-$
V-linux-x86_64"
chmod +x rubrapack
./rubrapack lint hello.toml --strict
./rubrapack build hello.toml -o "hello-$VERSION.msi" -D VERSION="$VERSION" --reproducible \
    --key signer.pfx --pass-env SIGN_PASS --timestamp http://timestamp.digicert.com
```

The key and its password come from the server's secret store (for example the repository's *secrets* in GitHub Actions) into a file and an environment variable for this step only. Never put them in the repository. A key on a hardware token or in a cloud signing service is used with `--pkcs11` instead (chapter 16).

19.5 With an AI assistant

The source is short plain text and every error message says where and why, so an AI coding assistant can do all of the above: give it this manual (the book, <https://rubidus-api.github.io/rubrapack/>) and say what to install. It can write the source, run `rubrapack lint` until it is clean, build, check the result with `inspect` and `extract`, and write the release script. Two things stay yours:

- keep the upgrade code of the first version in every later one (chapter 3);
- install the package on a real Windows once before you ship it, and try the upgrade from the previous version.

19.6 Where to go from here

You have used every table and every option of rubrapack. From now on:

- **Part II, Reference**, lists every key of every table and every option in one place - the page to keep open while writing a source.
- **Part III, Background knowledge**, explains the ideas under all this - bits and bytes, text encodings, GUIDs, hashes, compression, signatures - from the beginning.
- **Part IV, File formats**, opens the files you built and shows every byte: the MSI's file system, its tables and strings, the cabinet, the MSIX's ZIP and block map, and the signature.

Part II

Reference

20 Source files and the rubrapack command

In this chapter

20.1	Getting started	73
	A first package	73
	A license page and optional parts	74
	Where the rest is explained	75
20.2	Example	75
20.3	The TOML subset	76
20.4	Tables	76
	MSIX packages	77
	Registering with an installed program: [action.ID]	78
	Registry values: [registry.ID]	78
	Shortcuts: [shortcut.ID]	79
	File types and links: [assoc.ID], [protocol.ID]	79
	MSIX only: [msix-extension.ID]	79
	Removing and copying: [remove.ID], [copy.ID]	80
	Environment variables: [env.ID]	80
	INI files: [ini.ID]	80
	Searching and requiring: [search.ID], [require.ID]	80
	Services, fonts and permissions	81
	Per-user and dual packages: scope	81
	Cabinets	81
	Installed apps entry and properties	81
	Installing from the command line	81
	Installing without a window	82
	Dialogs: ui	82
	Several languages	83
	Icon, the finished page, and the scope page	83
	Your own dialog pages: [dialog.ID], [dialog-control.ID]	84
	Architectures and upgrade families	84
	Versions that must be removed first	85
	Wildcards: [files.ID]	85
	Empty folders: [folder.ID]	85
	Features	85
	Conditions: when	85
	Variables	86
	Program files	86
	Leaving a file at removal: keep	86
	Guarding the install folder	86
	Paths	86
20.5	Command line	87
20.6	Diagnostic codes	90

A source file describes a package. It is a strict subset of [TOML 1.0](#) and is named *.toml, so editors, GitHub and AI assistants treat it as TOML (the older *.rpk name is read the same way). Every source is valid TOML, but rubrapack refuses TOML features outside the subset instead of ignoring them.

20.1 Getting started

rubrapack is a single program file. Download `rubrapack-<version>-windows-x64.exe` (rename it `rubrapack.exe` if you like) or `rubrapack-<version>-linux-x86_64` (`chmod +x` it) from the releases page and run it where it lies, or put it on the `PATH`. There is nothing else to install: no runtime, no SDK, no libraries. The same program builds the same packages on Windows and on Linux.

This section is the short version, for readers who know installers. **Part I, the tutorial**, teaches the same from the beginning, one step at a time, and goes on to every table and option: start with Before you start.

A first package

Put what you ship in a folder `dist/`: the program, and whatever it needs, in `dist/files/` (subfolders are kept). Then let rubrapack write a starting source:

```
rubrapack new app.toml      # asks, then writes app.toml and checks it
```

It asks for the product name, the version, the folder with the files, the main program (its architecture is read from the file), the folder under Program Files, who it installs for, which sub folders are optional parts, the dialogs, a license (a `LICENSE.txt`, `.md` or `.rtf` next to it is offered), Korean dialogs, and the shortcuts. Enter takes the value in brackets. At the end it prints the same answers as one command (`rubrapack new app.toml --dist dist --ui features ...`), which a script or an AI assistant can run without questions. `rubrapack new` without a name asks for the file name too, and `rubrapack new app` (no extension) writes a fixed starter instead.

Either way, `app.toml` is plain text to read and change - in any text editor, or with `rubrapack edit app.toml`, a menu of the same questions with the current values as defaults (version, install folder, dialogs and license, optional parts, shortcuts, and matching the file list to the program folder after it changed). It changes only those values and keeps every other line, comment and table as it was. For scripts: `rubrapack edit app.toml --set define.VERSION=1.1.0 --sync`. This one installs the program and its files into Program Files\My App, puts it in the Start menu, and shows a dialog that lets the user change the folder:

```
[package]
name = "My App"
manufacturer = "My Company"          # shown in Settings > Installed apps
version = "${VERSION}"
arch = "x64"                          # x64, arm64 or x86
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}" # yours from `new`; keep it forever
ui = "installdir"

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/My App"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[shortcut.StartMenu]
dir = "Programs"
```

```
name = "My App"
target = "file:App"
```

```
rubrapack build app.toml -o app-1.0.0.msi
rubrapack build app.toml -o app-1.0.1.msi -D VERSION=1.0.1    # the next version
```

That is a complete installer. It appears in Installed apps, repairs itself, and removes everything it installed. A higher version replaces the one installed; the same or an older one is refused with a message. The component GUIDs, file keys, cabinet and tables are derived from the source, so nothing but the upgrade code needs to be remembered from one version to the next.

A license page and optional parts

Two things most installers want: the user accepts a license before installing, and some parts are optional. `license` adds the license page (Next stays off until "I accept" is ticked), and `ui = "features"` adds a tree in which the user picks the features to install. Here the program is always installed and the samples are offered but not selected:

```
[package]
name = "My App"
manufacturer = "My Company"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}"
ui = "features"                # welcome, license, folder, feature tree, ready
license = "LICENSE.txt"        # Next stays off until "I accept" is ticked

[define]
VERSION = "1.0.0"

[feature.Main]
title = "My App"
description = "The program itself."
required = true                # always installed: the tree does not offer to leave it
out

[feature.Samples]
title = "Samples"
description = "Example documents to try the program with."
level = 2                      # offered in the tree, not selected by default

[dir.INSTALLDIR]
path = "ProgramFiles/My App"
feature = "Main"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[files.Samples]
dir = "SamplesDir"
```

```
glob = "dist/samples/**"

[shortcut.StartMenu]
dir = "Programs"
name = "My App"
target = "file:App"
```

Put the license text in LICENSE.txt (.txt, .md or .rtf) and the samples in dist/samples/. A feature with level = 2 is not installed unless the user ticks it; required keeps the tree from offering to leave a feature out. Later the user changes the choice from Installed apps (Change), and the command line does the same without dialogs: msiexec /i app.msi /qn ADDLOCAL=Samples adds the samples, REMOVE=Samples takes them out (required binds only the tree, not the command line). Features and Conditions have the rest.

Where the rest is explained

To	Read
install, upgrade, remove and log with msiexec	A first installer, Versions and upgrades
understand an error and look inside a package (lint, inspect, extract, exit codes)	Checking and looking inside, Diagnostic codes
sign a package	Signing and timestamps
build in a script or CI job, or with an AI assistant	Starting fast and automating

20.2 Example

```
[define]
VERSION = "1.4.0"

[package]
name = "Example App"
manufacturer = "Example"
version = "${VERSION}"
arch = "x64" # x64, arm64 or x86 - no default
upgrade-code = "{0B9A6C1E-3D2F-4A5B-8C7D-6E5F4A3B2C1D}" # generate your own once, keep it forever
language = "en-US" # or "ko-KR"

[dir.INSTALLDIR]
path = "ProgramFiles/Example App"

[dir.Docs]
path = "INSTALLDIR/docs"

[file.MainExe]
dir = "INSTALLDIR"
source = "dist/app.txt"

[file.Guide]
dir = "Docs"
source = "dist/guide.txt"
name = "User guide.txt"
```

```
rubrapack build example.toml -o example.msi -D VERSION=1.4.1
rubrapack inspect example.msi File
```

20.3 The TOML subset

Accepted: tables [kind] and [kind.ID], bare keys (A-Z a-z 0-9 _ -), basic strings "...", with TOML escapes, literal strings '...' (no escapes - use them for backslashes and quotes: 'SOFTWARE\Example'), decimal and 0x integers, true/false, one-type arrays, # comments. UTF-8 with or without a BOM, or UTF-16LE with a BOM; LF or CRLF.

Refused with an error: multi-line strings, inline tables, arrays of tables, dotted and quoted keys, table names with more than two parts, floats, dates, _ in numbers, octal/binary numbers, empty or mixed arrays, keys before the first table. Table and key order never matters.

20.4 Tables

Table	Keys (required in bold)
[package]	name , manufacturer , version (a.b.c or a.b.c.d), arch , upgrade-code , upgrade-code-x64 / -arm64 / -x86, product-code, summary-name (ASCII), language, scope (machine, user, dual), ui (none, basic, minimal, installdir, features), license (.txt, .md, .rtf), reboot (suppress/allow), downgrade-message, compress (none, mszip, mszip:0..mszip:9; default mszip:6), cab (embed or external), cab-max-size (MiB), refuse-upgrade-below, refuse-upgrade-message
[define]	variables: NAME = "value"
[feature.ID]	title , description, level (1-32767), hidden, parent, required, follow-parent, when
[dir.ID]	path = Base/relative/path, feature, guard (true: see Guarding the install folder)
[file.ID]	dir , source , name, vital (default true), any-arch, feature, component-guid, keep, when
[files.ID]	dir , glob , vital, any-arch, feature, keep, when
[folder.ID]	dir , name , keep, feature
[arp]	no-modify, no-repair, help (URL), about (URL), icon (.ico) - how the product shows in Installed apps
[property.ID]	value , secure, hidden - an upper-case public property
[action.ID]	run (file:ID of an .exe in this package), do , undo , check
[registry.ID]	root (HKLM, HKCU, HKCR, HKMU), key , name, value, type, remove, keep, view, with, feature, when
[remove.ID]	dir , name (* and ?; omitted = the folder itself), on (install, uninstall, both), feature
[ini.ID]	dir , file , section , key , value, mode (set, add, remove), feature, when
[require.ID]	condition , message
[search.ID]	property (or a dir ID), kind (registry: root, key, name, view; file: path, file, min-version; dir: path; component: component-guid)
[service.ID]	file (file:ID of an .exe), name , display-name, description, start (auto, demand, disabled), account (LocalSystem, LocalService, NetworkService), args, start-on-install
[assoc.ID]	extension (.ext, lower case), prog-id , target (file:ID of an .exe), description, icon (file:ID), args (default "%1")
[protocol.ID]	name (the scheme, lower case), target (file:ID of an .exe), description, args (default "%1")
[font.ID]	file (file:ID of a file in a dir with path = "Fonts"), title
[permission.ID]	target (dir:ID, file:ID, registry:ID), sddl
[env.ID]	name , value , mode (set, append, prepend), keep, feature, when
[copy.ID]	source (file:ID), dir , name (default: the source's name)

Table	Keys (required in bold)
[ui]	install-dir (a dir ID; default INSTALLDIR), banner (.bmp), launch (file: ID), launch-args, launch-checked, languages (added to English, e.g. ["ko"]), license-xx, name-xx, font-xx, langid-xx - see Several languages
[ui-text.ID]	text or text-xx - replaces one built-in dialog text
[dialog.ID]	after (a built-in page or another [dialog.*]), title, description, title-xx, description-xx
[dialog-control.ID]	dialog , type (text, checkbox, edit, radio, combo), x , y , width , height , text, property, values, labels, text-xx, labels-xx
[shortcut.ID]	dir (a dir ID, or Programs, Desktop, StartMenu, Startup), name , target (file: ID), args, description, working-dir (a dir ID), icon (.ico), when
[msix]	identity-name , publisher , publisher-display-name, min-version - see MSIX packages
[msix-app.ID]	executable (a [file.*] ID), display-name, description, logo-150, logo-44, store-logo
[msix-extension.ID]	kind (alias: alias ; startup-task: task-id, display-name, enabled), app (an [msix-app.*] ID; default the first) - MSIX only

Base in a dir path is another dir ID or one of: ProgramFiles (64-bit for x64/arm64, 32-bit for x86), ProgramFiles32, CommonFiles, AppData, LocalAppData, CommonAppData, StartMenu, Programs, Desktop, Startup, Windows, System, Fonts, Temp. A path may also be a known folder alone (path = "Fonts") for files that go into that folder itself.

IDs are [A-Za-z_] [A-Za-z0-9_]* (at most 72 characters, 38 for features) and must differ across dirs, files and features.

MSIX packages

The same source builds an MSIX package when the output ends in .msix: one desktop application that runs with full trust, for one architecture. Two more tables say what only MSIX needs:

```
[msix]
identity-name = "Example.App"           # 3-50 characters: A-Z a-z 0-9 . -
publisher = "C=KR, O=Example, CN=Example" # the signing certificate's subject, last part
first
publisher-display-name = "Example"       # default: [package] manufacturer
min-version = "10.0.17763.0"            # the oldest Windows it installs on (this is the
default)

[msix-app.Main]
executable = "MainExe"                  # the [file.*] that starts the app
display-name = "Example App"             # default: [package] name
description = "An example"               # default: the display name
logo-150 = "assets/Square150x150.png"    # PNG, 150x150
logo-44 = "assets/Square44x44.png"       # PNG, 44x44
store-logo = "assets/StoreLogo.png"      # PNG, 50x50
```

- The version is [package] version with four parts (1.2.3 becomes 1.2.3.0), the architecture [package] arch, the language [package] language.
- The package's own folder is that of the first [msix-app.*]'s executable - the dir anchored in a known location, such as ProgramFiles/Example App. Files in other known locations go into the package's virtual file system, where the app sees them at their usual place: Program Files (VFS\ProgramFilesX64, or X86 for an x86 package and for ProgramFiles32), CommonFiles, System, Windows and CommonAppData (ProgramData). There is none for the user's AppData or LocalAppData, nor for Temp: files there are an error (RP1609). A font in Fonts goes in through its [font.*]; the Start menu, Programs and Desktop folders take shortcuts ([shortcut.*]), and Startup a startup task ([msix-extension.*]).

- Several [msix-app.*] tables make several entries of one package (at most 100); the first one gives the package its logo.
- [registry.*] values go into the package's virtual registry, which the app sees merged into the real one while the machine's registry stays untouched: HKLM (and HKMU) under Software into Registry.dat (with view = "32" in the 32-bit view), HKCU under Software into User.dat. What an MSIX cannot hold is an error (RP1612): HKCR (file types and protocols are [assoc.*] and [protocol.*]), keys outside Software, remove and keep, and values with a part Windows Installer fills in at install time ([INSTALLDIR], [#File], ...; the escapes [\[] and [\]] are fine).
- Give the three logos or none: without them the package gets plain one-colour logos. A logo must have the exact size (RP1608).
- What an MSIX cannot do is an error, not something left out quietly (RP1605): custom actions, services, environment variables, INI files, permissions, launch conditions and searches, files removed or copied at install, empty folders. Add msi-only = true to such a table (or to a [file.*]/[files.*]) and the MSI keeps it while the MSIX is built without it. Features, properties, dialogs and [arp] concern the Windows Installer only and are not used for an MSIX.
- --unsigned-test adds the attribute Windows needs to install an unsigned package for testing (Add-AppxPackage -AllowUnsigned, as administrator when it contains a program); such a package is not for distribution and its identity differs from the signed one. --key signs the package (or the bundle and its packages) instead; see sign below.
- Files are compressed (--msix-compress store turns it off); pictures, archives and other already compressed files are stored. An MSIX holds no time: the same source gives the same bytes on Linux and Windows.
- An output ending in .msixbundle is a bundle: the source built once for each architecture of --arch (a list, --arch x64,x86,arm64; without it the source's own), each package named <identity-name>_<version>_<arch>.msix inside. Windows installs the package for its own architecture from it (an x64 machine takes x64 before x86). \$(ARCH) gives each build its own programs: source = "bin/\$(ARCH)/app.exe". The bundle's version is the packages' version.

Registering with an installed program: [action.ID]

```
[action.Tip]
run = "file:MainExe"      # an .exe this package installs
do = "--register"         # run after the files are installed, and again on repair
undo = "--unregister"     # run on removal, before the files are removed
```

rubrapack turns the pair into deferred, elevated actions with their rollback twins, so removal and upgrade are all-or-nothing: if anything fails later (or do/undo itself exits non-zero), the files come back and the other command restores the registration. Both commands must be safe to run twice and must finish without asking anything - they run without a window, and nothing waits for a user. The arguments are passed as written (they are not formatted strings). check is accepted but not used by this version.

Registry values: [registry.ID]

```
[registry.InstallDir]
root = "HKLM"
key = 'SOFTWARE\Example'    # literal strings keep the backslashes
name = "InstallDir"        # omit it for the key's default value
value = "[INSTALLDIR]"     # an MSI formatted string: [PROPERTY], [#FileID], [\[] for
"["
```

type is string (default), expand, dword (an integer, 0 to 0xFFFFFFFF), qword (an integer, or "0x" and up to 16 hex digits), binary (hex digits) or multi (an array of strings). Windows Installer cannot write REG_QWORD itself; rubrapack adds its small helper DLL to the package for it, which also restores

the previous value if the installation fails. Each value is its own component, removed at uninstall (keep = true leaves it); with = "file:ID" puts it in that file's component instead. In a 64-bit package values go to the 64-bit registry view; view = "32" writes to the 32-bit view. remove = true (without value) deletes the named value - or the whole key when name is omitted - during installation.

Shortcuts: [shortcut.ID]

```
[dir.Menu]
path = "Programs/Example"      # Start menu > Example

[shortcut.Settings]
dir = "Menu"
name = "Example settings"      # ".lnk" is added
target = "file:MainExe"
args = "--settings \"[INSTALLDIR]\""
```

A shortcut belongs to its target file (and its feature); folders created for it are removed at uninstall. In a per-machine package Programs and Desktop are the all-users Start menu and the Public Desktop.

In an MSIX a shortcut starts an application (its target must be an [msix-app.*] executable): one in Programs or StartMenu is that application's own Start menu entry (no args); one on the Desktop is written into the manifest and needs min-version = "10.0.19645.0" or later (RP1614). Other folders, working-dir and [...] in args are errors there (RP1613); for Startup use a startup task.

File types and links: [assoc.ID], [protocol.ID]

```
[assoc.Doc]
extension = ".exdoc"
prog-id = "Example.Document"   # tables sharing it describe one kind of document
description = "Example document"
target = "file:MainExe"
args = "--open \"%1\""        # the default is "%1"

[protocol.Link]
name = "example"               # example:... opens the program
target = "file:MainExe"
```

In an MSI they are registry values under HKEY_CLASSES_ROOT, in the program's component: the extension's default value is the prog-id, the prog-id has the description, DefaultIcon (icon, or the program's first icon) and shell\open\command; a scheme gets URL Protocol. HKEY_CLASSES_ROOT follows the installation: per machine they land in HKLM\Software\Classes, per user in HKCU\Software\Classes, and removal takes them away. A type that only this program claims opens in it directly; where the user has chosen another program, Windows keeps that choice.

In an MSIX they go into the manifest of the application whose executable is the target (a file type association, a protocol), and args must be plain text. icon is not used there: the application's logo stands for the file type.

MSIX only: [msix-extension.ID]

```
[msix-extension.Cli]
kind = "alias"
alias = "example.exe"          # typed in a console, it starts the application

[msix-extension.Boot]
kind = "startup-task"          # starts with Windows once the application has run once
display-name = "Example"       # the name in Task Manager; task-id defaults to the table ID
enabled = true
```

An MSI build leaves these out. Fonts ([font.*]) in an MSIX are shared with other applications (uap4:SharedFonts) from the package's Fonts folder; title is not used there.

Removing and copying: [remove.ID], [copy.ID]

[remove.ID] deletes files matching name in dir - for example *.log an older version left, at on = "install", or files the program writes at run time, at on = "uninstall". Without name it removes the folder itself when it is empty. A failed installation puts removed files back. [copy.ID] installs a second copy of a file of this package into another folder; it goes and comes with its source. Folders that existed before the installation are never removed.

Environment variables: [env.ID]

A system (machine-wide) variable. mode = "set" (default) replaces it, append/prepend add ;value to the end or value; to the front of what is there. Uninstall undoes exactly that: a set variable is deleted, an appended part is taken out and the rest kept. keep = true leaves it.

INI files: [ini.ID]

mode = "set" (default) writes key=value in [section] of the file, add appends the value to a comma-separated list (a becomes a,b), remove deletes the key during installation. Uninstall takes out what set and add wrote. remove runs before files are installed, so it is for INI files an older version left behind.

Searching and requiring: [search.ID], [require.ID]

A search runs before anything else and puts what it found into a public property (empty when nothing is found): a registry value's data, the full path of a file (path = a known folder and a relative path, like System or ProgramFiles/Example; min-version for program files), a folder, or the key file of another product's component. A requirement stops a first installation with its message when its condition is false (repair and removal are never blocked); conditions use Windows Installer's syntax (VersionNT >= 603, FOUND_TOOL, NOT OLDSETTING) and may test search results.

```
[search.Tool]
property = "FOUND_TOOL"
kind = "file"
path = "System"
file = "tool.exe"

[require.Tool]
condition = "FOUND_TOOL"
message = "[ProductName] needs tool.exe."
```

A registry or dir search may name a dir instead of a property: the folder it finds becomes that dir's default. This is how a package remembers the folder the user chose: write the folder down, and look it up in the next version. A registry value is used only when that folder still exists; a folder given on the command line (msiexec /i app.msi INSTALLDIR=D:\Apps\Example\) still wins.

```
[registry.RememberDir]
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
value = "[INSTALLDIR]"

[search.PreviousDir]
property = "INSTALLDIR"           # a dir: its default, when the registry holds an existing
folder
kind = "registry"
root = "HKLM"
```



```
key = "Software\\Example"
name = "InstallDir"
```

Services, fonts and permissions

[service.ID] installs a service run by an .exe of the package: it is stopped before its files change and at uninstall, deleted at uninstall, and started after installation with `start-on-install = true`. [font.ID] registers a font file that the package installs into the Fonts folder; without `title` Windows reads the name from the TrueType/OpenType file. [permission.ID] sets an SDDL security descriptor on a folder the package creates, one of its files, or a registry value it writes (this raises the package to Windows Installer 5.0).

Per-user and dual packages: scope

`scope = "machine"` (default) installs for everyone and needs administrator rights. `scope = "user"` installs for the current user without elevation: `ProgramFiles` becomes `%LOCALAPPDATA%\Programs`, `Programs` and `Desktop` are the user's, registry values go to `HKCU` (or `HKMU`), environment variables are the user's, and do/undo actions run as the user; asking for a per-machine installation is refused. `scope = "dual"` installs per user by default and per machine from an elevated prompt with `msiexec /i x.msi ALLUSERS=1 MSIINSTALLPERUSER=""`; its registry values use `HKMU`, which is `HKLM` or `HKCU` as installed. Services, fonts, permissions and the machine folders (`Windows`, `System`, `Fonts`, `CommonAppData`) need `scope = "machine"`.

Cabinets

Files are compressed into one cabinet embedded in the package. `cab-max-size = N` starts a new cabinet after `N` MiB of files; `cab = "external"` writes the cabinets next to the package as `<name>.cab` (or `<name>-1.cab`, `<name>-2.cab`, ...), which must travel with it; without `cab-max-size` external cabinets are split before 2 GiB each. Windows Installer cannot open a package of 2 GiB or more, so an embedded cabinet that large is an error (RP1516) that asks for `cab = "external"`. rubrapack never overwrites an existing cabinet and writes the package last: it is built in `<out>.rp-map` (the package's bytes go straight into that file, not into memory) and renamed to its name only when everything succeeded, so a failed build leaves no package. Compression runs on every processor (`--jobs N` to use fewer); the bytes are the same either way. Every package also carries the administrative (`msiexec /a`, an uncompressed network image) and advertisement (`msiexec /jm`) sequences.

Only these fields are MSI formatted strings: registry value (and multi items), shortcut args, environment value, INI value, requirement message, service args. Everywhere else rubrapack writes the text exactly as given.

Installed apps entry and properties

[arp] sets how the product appears in Settings > Installed apps (`no-modify`, `no-repair`, `help`, `about`). [property.NAME] adds a public property; `secure = true` lets it reach the elevated part of the installation, `hidden = true` keeps its value out of logs. Names the installer or rubrapack set themselves (`ARP*`, `MSI*`, `RP_*`, `ALLUSERS`, `REBOOT`, ...) are refused.

Installing from the command line

Everything the dialogs choose can be given to `msiexec` instead:

Property	What it does
INSTALLDIR=D:\Apps\Example\	the install folder (any dir with an upper-case ID)
ADDLOCAL=Core,Extra	install these features (ADDLOCAL=ALL: every feature)
REMOVE=Extra	remove these features from an installed product (REMOVE=ALL: everything)

Property	What it does
INSTALLLEVEL=3	install every feature whose level is at most 3
RPLANGUAGE=ko	the dialogs' language (with [ui] languages)
ALLUSERS=1 MSIINSTALLPERUSER=""	a dual package for everyone (default: just the current user)
DESK=1, APP_MODE=server	your own properties: when conditions, dialog values

Installing without a window

Every package rubrapack builds installs, repairs, upgrades and uninstalls from the command line with no user interface:

```
msiexec /i example.msi /qn /l*v install.log
msiexec /x {ProductCode} /qn
```

Exit code 0 is success, 3010 success with a restart needed (rubrapack never restarts the machine itself), anything else a failure after which the machine is as before.

Dialogs: ui

ui in [package] picks one of the built-in dialog sets. Without it (none) the package shows only Windows Installer's own progress bar.

ui	Installing	Already installed
basic	progress, then finished (or error)	progress, finished
minimal	welcome, the license if there is one, progress, finished	repair or remove
installdir	welcome, license, install folder (with a folder browser), ready, progress, finished	repair or remove
features	as installdir, plus a feature tree and the disk space it needs	repair or remove

Every set also has the cancel question, the error dialog, the files-in-use list and the out-of-disk-space warning. The dialogs only collect values that all have defaults, so /qn still installs without a window.

```
[package]
ui = "installdir"
license = "LICENSE.txt"           # the Install/Next button stays disabled until it is accepted

[ui]
install-dir = "APPDIR"           # the dir the user may change (default INSTALLDIR)
banner = "banner.bmp"           # the strip at the top; about 493 x 58 pixels

[ui-text.WelcomeText]
text = "This will install [ProductName]. Close other programs first."
```

A .txt or .md license is shown as plain text (Korean, emoji and any other characters are kept); an .rtf license is used as it is. Without banner the strip is plain white. [ui-text.ID] replaces one text; the texts are MSI formatted strings, so [ProductName] is replaced and a literal [is written [\]. The IDs are: Back, Next, Cancel, Install, Finish, OK, Yes, No, Retry, Ignore, Abort, Exit, Browse, WelcomeTitle, WelcomeText, LicenseTitle, LicenseText, LicenseAccept, DirTitle, DirText, DirLabel, BrowseTitle, BrowseText, BrowseLookIn, BrowseFolder, BrowseUp, BrowseNew, CustomizeTitle, CustomizeText, Reset, DiskCost, DiskCostTitle, DiskCostText, ReadyTitle, ReadyText, ProgressTitle, ProgressText, ProgressStatus, ExitTitle, ExitText, UserExitTitle, UserExitText, FatalTitle, FatalText, CancelText, FilesInUseTitle, FilesInUseText, OutOfDiskTitle, OutOfDiskText, MaintTitle, MaintText, Repair, RepairText,

Remove, RemoveText, LanguageTitle, LanguageText, DirGuardText (the guard's message below). In button texts & marks the access key (&Next is Alt+N).

Several languages

The dialogs are in English. [ui] languages adds other languages to the same package; then the first page asks for the language, and every page after it - welcome, license, folder, your own pages, ready, progress, finished, and the cancel, error, files-in-use, disk-space and maintenance pages - speaks the one chosen. The choice made for the user in advance is the first added language whose LANGIDs hold the user's regional format (UserLanguageID), then the system locale (SystemLanguageID), else English. RPLANGUAGE=ko on the command line chooses directly; a silent installation (/qn) shows nothing and needs no choice. With English alone there is no language page.

```
[ui]
languages = ["ko"]           # English is always there, and the default
license-ko = "LICENSE-ko.txt" # a license per language (default: [package] license)

[ui-text.WelcomeText]
text-ko = "[ProductName]을(를) 설치합니다." # text = every language, text-xx = one

[dialog.Options]
after = "RpInstallDirDlg"
title = "Options"
title-ko = "선택 사항"

[dialog-control.Mode]
# ...
labels = ["&Typical", "&Portable"]
labels-ko = ["표준(&T)", "휴대용(&P)"]
```

- The built-in texts exist in English and Korean (ko). Another language (ja, de, ...) gives every built-in text as text-xx (lint names the ones missing), and may give name-xx (its name on the language page), font-xx (the face of its dialogs) and langid-xx (the LANGIDs that choose it: one number or an array). Common languages have these built in: ja, zh, de, fr, es, it, pt, nl, pl, ru, uk, tr, vi, th.
- Korean dialogs use Malgun Gothic, English ones Segoe UI.
- A text that uses a property other than [ProductName], [Manufacturer] and [ProductVersion] is formatted when the language is chosen, and must fit 255 characters.
- Windows Installer's own texts - the feature tree's menu, sizes, the time left, and its error messages - are not the package's: the first stay English, the error messages follow Windows.
- [package] language sets only the package's language (ProductLanguage, the summary). Since 0.2 it no longer makes the dialogs Korean: add languages = ["ko"] (lint warns, RP1317).

Icon, the finished page, and the scope page

- [arp] icon = "app.ico" is the product's icon in Installed apps; [shortcut] icon gives a shortcut its own .ico (without it, the program's own icon). Both are read when the package is built and stored in it.
- [ui] launch = "file:App" puts "Launch [ProductName]" on the finished page (ticked unless launch-checked = false; launch-args are its arguments). Finish starts the program after a first installation or an upgrade, as the user who ran the setup, not with the installer's rights; not after a repair or removal, and never at /qn.
- scope = "dual" with dialogs adds a page after the license: "Just me" (the default) or "Everyone on this computer", which needs administrator rights; the install folder moves to %LOCALAPPDATA%\Programs or Program Files accordingly.

Your own dialog pages: [dialog.ID], [dialog-control.ID]

With minimal, installdir or features you can add pages to the built-in flow. A page gets the same banner and Back / Next / Cancel buttons as the others; you place the controls in its body.

```
[dialog.Options]
title = "Options"                # the banner heading; [ProductName] when omitted
description = "Choose how to install."
after = "RpInstallDirDlg"        # shown right after this page

[dialog-control.ModelLabel]
dialog = "Options"
type = "text"
x = 20
y = 55
width = 330
height = 12
text = "Installation &mode:"

[dialog-control.Mode]
dialog = "Options"
type = "radio"
x = 20
y = 70
width = 200
height = 42                      # at least 12 per value
property = "APP_MODE"
values = ["typical", "portable", "server"]
labels = ["&Typical", "&Portable", "&Server"]

[property.APP_MODE]
value = "typical"                # the default, also for a silent installation
```

- after is a page of the chosen set - RpWelcomeDlg, RpLicenseDlg (with a license), RpInstallDirDlg (installdir, features), RpCustomizeDlg (features) - or another of your pages. Several pages after the same page follow in ID order. In minimal the last page's button is Install; in the other sets the ready page always comes last.
- Coordinates are dialog units on a 370 x 270 page; controls go in the body, between y = 45 and y = 234. The Tab key moves through the controls from top to bottom, then left to right.
- text shows a label (& marks an access key; on a text control it moves to the next control). checkbox sets its property to 1 when ticked and removes it when clear. edit lets the user type the property's value. radio and combo choose one of values (1 to 32 strings of at most 64 bytes), shown as labels (default: the values themselves), in the order written.
- A radio or combo needs a [property.*] whose value is one of its values: the dialogs only collect values, and a silent installation (/qn) uses the defaults. Any property can also be set on the command line: msiexec /i example.msi /qn APP_MODE=server.
- The properties are public names of your own (upper case), one control each, and reach the elevated part of the installation, so [registry.*], [ini.*], [env.*], conditions and the rest can use them as [APP_MODE].
- Page IDs starting with Rp and the control IDs of the frame (Banner, Title, Description, BannerLine, BottomLine, Back, Next, Cancel) are reserved; a page holds up to 64 controls.

Architectures and upgrade families

arch is the source's own architecture and upgrade-code its upgrade family. To build the same source for another architecture (--arch), give that architecture its own family with upgrade-code-x86, upgrade-code-arm64 or upgrade-code-x64; without it the build is refused. Windows Installer's upgrade detection cannot tell architectures apart, so sharing one code would make installing one

architecture remove the other. An MSIX has no upgrade code, so .msix and .msixbundle builds do not need these.

Versions that must be removed first

refuse-upgrade-below = "1.0.0" refuses to upgrade an installed version below 1.0.0 and tells the user to remove it first; the message (refuse-upgrade-message, or a default one) always ends with the command that does it: `msiexec /x {ProductCode} /qn MSIRESTARTMANAGERCONTROL=Disable`. Use it when older versions were built by another tool without `MSIRESTARTMANAGERCONTROL=Disable`: removing such a version inside an upgrade would try to close every program that has its files loaded (see `formats/msi-package.md`, "Files in use").

Wildcards: [files.ID]

glob is a source path with * (any characters within one folder), ? (one character) and ** (any number of folders). Matches are sorted by name, so the result never depends on the file system. Folders below the first wildcard are recreated under dir: `glob = "dist/layouts/**/* .jmt"` installs `dist/layouts/de/x.jmt` as `<dir>/de/x.jmt`. No match, a symbolic link, or the output file itself among the matches is an error.

Empty folders: [folder.ID]

Creates the folder name inside dir even when no file goes there. With `keep = true` the folder stays after uninstall (for data the program writes).

Features

Without any [feature.*] table everything goes into one hidden feature. As soon as one feature is declared, every file needs one: its own feature key, or the feature of its dir. A feature with a level above 1 is not installed by default. With `ui = "features"` the user picks features in a tree, and changes them later from Installed apps (Change on the maintenance page).

- `required = true`: the tree offers no "will be unavailable" for it.
- `follow-parent = true`: installed wherever its parent is (on this computer, or not at all).
- `when = "<condition>"`: the feature is off - not installed and not shown - unless the condition holds (see below).

Conditions: when

when takes a Windows Installer condition (`VersionNT64`, `DESK = "1"`, `NOT OLDVERSION`) on a feature, a file or file group, a registry value (not one with a file: put it on the file), a shortcut, an environment variable or an INI value. What it guards is installed only when the condition holds; properties set in your own dialog pages, on the command line or by a search can be tested. The condition is evaluated when the item is first installed, a major upgrade included (a repair keeps what is there); an MSIX installs everything and refuses when.

```
[dialog-control.Desk]                                # a check box in your own page (see below)
dialog = "Options"
type = "checkbox"
x = 20
y = 60
width = 300
height = 16
text = "Create a &desktop shortcut"
property = "DESK"

[shortcut.Desk]
dir = "Desktop"
name = "Example"
```

```
target = "file:App"
when = "DESK"
```

Variables

\$(NAME) inside a string value is replaced once, from -D NAME=value on the command line first, then [define]. The result is not read again (a value containing \$(X) stays literal). \$\$ is a literal \$. An undefined name is an error. \$(ARCH) is built in, unless -D or [define] defines it: the architecture being built (x64, arm64 or x86) - so one source can name each architecture's program, as source = "bin/\$(ARCH)/app.exe".

Program files

Files that are Portable Executables (.exe, .dll, ...) are checked: their machine type must match arch (an x86 helper in an x64 package needs any-arch = true), and their version resource becomes the file's version in the package, which is how Windows Installer decides whether to replace an installed file.

Leaving a file at removal: keep

keep = true on a file (or a file group) leaves it in place when the product is removed - for settings the user may have changed. A repair or a later version does not overwrite such a file once it has changed (Windows Installer's rule for files without a version). An MSIX removes all its files and refuses keep.

Guarding the install folder

A program whose files are loaded by other programs - an input method, a shell extension, a service - must not be installed into a folder that someone else prepared: files already there stay, and a DLL planted beside the program would be loaded with its rights. guard = true on a dir makes a first installation stop before any file is placed when that folder

- already exists and its owner is not SYSTEM, Administrators or TrustedInstaller, or
- is reached through a junction or another reparse point (the folder itself or any folder above it).

A folder that does not exist yet passes (the installer creates it; [permission.*] can lock it). Repair, upgrade-in-place and removal are not checked; a major upgrade is a first installation of the new version and is checked like one (the folder the earlier version created passes). The installation ends with 1603 after the message DirGuardText ([1] is the folder; in the language chosen in the dialogs), also at /qn, and the log names the owner found. The check is done by rubrapack's helper DLL, which the package then carries (as for REG_QWORD values). What is checked is the owner: a folder owned by Administrators whose permissions let anyone write is not refused (lock it with [permission.*]).

```
[dir.INSTALLDIR]
path = "ProgramFiles/Example"
guard = true
```

Paths

Source paths are relative to the source file and use /. Absolute paths, \, symbolic links and missing files are errors. Target names may use any Unicode text except what Windows forbids (< > : " / \ | ? *, control characters, trailing dot or space, device names such as CON), and two names in one folder may not differ only by letter case. Windows Installer stores a name together with its 8.3 short name in 255 UTF-16 units, so a long name may use only what the short name leaves: 242 units for name.ext with a three-letter extension, 246 for a name without one (NTFS would accept 255). A longer name is refused with RP1514.

20.5 Command line

```

rubrapack build <src.toml> -o <out.msi|out.msix|out.msixbundle> [-D NAME=VALUE]...
    [--arch x64|arm64|x86 | --arch <list> (.msixbundle)]
    [--compress none|mszip|mszip:N] [--jobs N] [--nfc] [--reproducible]
    [<key> [--cert <chain.pem>]
    [--timestamp <URL> [--tsa-trust <certificates>] [--tls-trust <certificates>]
    [--system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs]
    [--unsigned-test] [--msix-compress deflate|store]      (.msix, .msixbundle)
rubrapack inspect <file.msi> [table | --summary | --files | --streams]
rubrapack inspect <file.msix|file.msixbundle> [--files | --manifest]
rubrapack inspect <file.cab>
rubrapack new [msi] <name>
rubrapack new [<file>.toml] [-i]
rubrapack new <name> [--dist <folder>] [--name ...] [--ui ...] [--optional ...] ...
rubrapack edit <file>.toml [--set <table>.<key>=<value>]... [--unset <table>.<key>]... [--sync]
rubrapack guid [--from <text>]
rubrapack lint <src.toml> [-D NAME=VALUE]... [--arch x64|arm64|x86] [--target msi|msix] [--nfc]
    [--strict]
rubrapack lint <file.msi> [--previous <old.msi>] [--strict]
rubrapack lint <file.msix|file.msixbundle> [--strict]
rubrapack extract <file.msi|file.msix|file.msixbundle|file.cab> -d <new dir> [--limit-entries
N] [--limit-bytes N]
rubrapack sign <file.exe|.dll|.msi|.msix|.msixbundle> <key> [--cert <chain.pem>]
    [--timestamp <URL> [--tsa-trust <certificates>] [--tls-trust <certificates>] [--
system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs] [-o <out>]
rubrapack keys list [--pkcs11 <module>] [--token-label <label>] [--pin-env VAR | --pin-file
FILE]]
rubrapack verify <file.exe|.dll|.msi|.msix|.msixbundle> [--trust <certificates>]... [--system-
roots] [--tsa-trust <certificates>]...
rubrapack version | help [command]

<key> is one of:
    --key <key.pfx|.pem> [--pass-env VAR | --pass-file FILE]                a key file
    --pkcs11 <module> --key-label <label> [--token-label <label>]
        [--pin-env VAR | --pin-file FILE]                                a key in a
PKCS#11 token
    --key-store <SHA-1 thumbprint> [--machine-store]                      a key in the
Windows store

```

- Command-line options override the source.
- `--reproducible` derives the package code from the content: the same source gives the same bytes, on Linux and on Windows, from any folder, with the options in any order. Without it the package code is random, as Windows Installer expects for different package files. An MSI holds no time at all (no creation or save dates in its summary; cabinet entries are dated 1980-01-01), so `SOURCE_DATE_EPOCH` does not change it; nor does it hold the source's path or the name of the machine or user that built it. Every source file is read once, and those bytes are hashed, versioned and packed; a file that changes while the package is being built stops the build (RP1515).
- `--nfc` puts the names the package gives to folders, files and shortcuts in Unicode NFC (composed). A file from macOS often has a decomposed name - a Hangul syllable such as U+D55C as the three jamo U+1112 U+1161 U+11AB - and Windows installs names exactly as they are, so the same word can become two different files. The source files keep their names; texts and registry values stay as written. `lint` warns (RP2105) about any text that is not in NFC, and a name that becomes the same as another one is refused (RP1511). The normalization is rubrapack's own, from Unicode 17.0 data.

- `inspect <file.msi> <table>` prints the table in Windows Installer's IDT format. `--files` prints one tab-separated line per file: installed path, size, File key, component, version, language, MD5 (from `MsiFileHash`; empty when the package has none). `--streams` lists the streams and their sizes. `inspect <file.cab>` lists the files in a cabinet. `inspect <file.msix>` shows the identity, the executable and the files (after checking every block's hash); `--files` lists path, size and whether each file is compressed, `--manifest` prints `AppxManifest.xml`. For a bundle, the identity and the packages (each opened and checked as a package), and its `AppxBundleManifest.xml`; `extract` writes the packages out.
- `new <name>` writes `<name>.toml`, a source that builds as soon as the program's files are in `dist/`, with a fresh upgrade-code. `new <file>.toml`, `new` without a name (which asks for the file name too), or `new <name> -i` asks the questions of A first package instead (on `stderr`; answers from `stdin`, one per line, so they can be piped; the input ending before the last answer writes nothing), and `new <name>` with options takes the same answers without asking: `--name`, `--manufacturer`, `--version`, `--dist` (default `dist`), `--main <file>|-`, `--arch`, `--install-dir`, `--scope`, `--optional <folder,...>|-`, `--ui`, `--license <file>|-`, `--languages ko|-`, `--shortcuts start,desktop|none`; what is left out takes the default the question would offer. The source lists the folder's top-level files one by one (a shortcut names a file, and a glob cannot leave one out) and each sub folder that holds files as a glob; optional sub folders become features at `level = 2` next to a required `Main`. It is checked like `lint` after it is written (exit 1 if that finds a problem). It never replaces an existing file; `new app.toml` over an existing `app.toml` says to use `edit`.
- `edit <file>.toml` changes a source in place. Without options it shows a menu: 1 name, manufacturer, version (in `[define] VERSION` when `version = "${(VERSION)}"`) and architecture; 2 the folder under Program Files and the scope; 3 dialogs, license and Korean dialogs; 4 which sub folders are optional parts (the first one adds a required `Main` feature and gives it to everything that then needs a feature); 5 the Start menu and desktop shortcuts and the program they open; 6 the file list against the program folder - files and sub folders that are gone are removed, new ones added; 7 any `table.key`; `v` shows the text, `l` checks it like `lint`, `s` saves and checks, `q` leaves (asking first when something changed). Each question offers the current value. Only the values changed are rewritten - comments, order and tables written by hand stay; a change that would not parse is not made. With options it asks nothing, applies them in order and saves: `--set package.version=1.2.0` (a value that is TOML - "text", a number, true, `["ko"]` - is used as it is, anything else as a string; a table that is not there is added), `--unset package.license`, `--sync` (6 without questions). Sources in UTF-16 are refused: save them as UTF-8 first.
- `guid` prints a random GUID (version 4). `guid --from <text>` prints the GUID rubrapack derives from a text, the same way on every machine: SHA-256 over the 32-bit little-endian length of `guid` and those 4 bytes, the 32-bit little-endian number 1, and the 32-bit little-endian length of the text and its UTF-8 bytes; the first 16 bytes of the hash, with the version nibble set to 8 and the variant bits to 10 (RFC 9562 UUIDv8), written as `{XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX}` in upper case. For example `guid --from hello` gives `{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}`.
- Diagnostics look like `example.toml:12:3: error[RP1201]: unknown key 'nmae' in [package] (did you mean 'name'?)`.
- Before writing, `build` checks the finished tables (RP20xx diagnostics, exit code 5); these checks guard rubrapack itself, so a source that passes the RP1xxx checks should never meet them.
- `lint <src.toml>` runs every check `build` runs and writes nothing. It checks the source for an MSI; `--target msix` checks it for an MSIX instead (the `[msix]` tables, and what an MSIX cannot carry, such as `keep` or `when`). `lint <file.msi>` checks a package made by any tool with the same table rules: what stops an installation is an error (exit code 5) - a value that does not fit its column, a missing referenced row, a duplicate key, a broken dialog tab order (RP2101), a dialog's first/default/cancel control that is not there (RP2102), a files-in-use dialog without a `ListBox` table (RP2103), an error dialog without `ErrorText/ErrorIcon` (RP2104) - and everything else is a warning. `--strict`

makes warnings fail too. A package in another code page than 65001 gets a note (RP2100): its text cannot be checked as UTF-8. The last line on stdout counts errors and warnings. `lint <file.msix>` checks the package the way Windows reads it - the ZIP, the block map, every block's hash - and that the manifest has an identity and names files that are in the package (RP2201, RP2202). For a bundle: its manifest's block map, and for each package where the manifest says it lies, its size, its identity and the package itself; one package per architecture.

- `lint new.msi --previous old.msi` also checks that `new.msi` upgrades `old.msi` cleanly. Errors: another UpgradeCode (RP2301: the new package does not replace the old one), a version that is not higher in its first three fields (RP2302: Windows compares only those). Warnings: the same ProductCode (RP2303: an upgrade needs a new one), a component whose GUID stays but whose key path (file, folder or registry value) changed (RP2304), or whose 64-bit flag changed (RP2305), a component that is gone (RP2306: its resources are removed with the old version), a feature that is gone (RP2307: a patch or a change of the installed set loses it). The last line names the previous package.
- `extract` unpacks a package the way it installs: folders by their long names under the Directory tree (a standard folder such as `ProgramFiles64Folder` keeps its name), files from the embedded or external cabinets, or from the source folders next to an uncompressed package. Every file is checked against its size and `MsiFileHash`. The target must be new or empty, and nothing is written until the whole package has passed: names with `..`, `/`, `\`, `:`, a drive, a reserved device name (`CON`, `COM1`, ...), a trailing dot or space, or two paths that differ only in case are refused. The defaults allow 100,000 entries and 16 GiB. A `.cab` unpacks by the names inside it. An `.msix` unpacks its files (not the ZIP's own `[Content_Types].xml` and `AppxBlockMap.xml`) after every block has matched its hash.
- `sign` adds an Authenticode signature (SHA-256; RSA, or ECDSA P-256/P-384) to a PE file, an MSI package, an MSIX package or a bundle (whose packages are signed first), in place or to `-o`; `build --key` signs the package as it is built (the same code), and a signed `--reproducible` build still gives the same bytes every time (the signature holds no time) - unless it is timestamped: a timestamp carries the server's time and serial number, so it differs on every run, and only the package inside the signature stays the same. An MSI whose cabinets lie outside it is refused: its signature would not cover them - use embedded cabinets, or `--allow-unsigned-cabs` to sign the `.msi` alone with a warning. The key file is a PKCS#12 file (`.pfx/.p12`: AES with PBKDF2-HMAC-SHA256, SHA-256 MAC - what current Windows and OpenSSL export) or a PKCS#8 PEM/DER key, plain or encrypted with PBES2; `--cert` adds certificates the key file does not hold. The certificate must be for code signing, must not be a CA and must be valid now. The password comes from an environment variable or a file, never from the command line. A file that is signed already is refused. An MSIX's `[msix] publisher` must be the certificate's subject as Windows writes it - its parts from last to first, as `O=Example Ltd, CN=Example` - or signing stops and prints the subject to use; `--unsigned-test` and a key do not go together.
- A key that must not leave its hardware - required for publicly trusted code-signing certificates since 2023 - stays there: `rubrapack` computes the hashes, the CMS structure and the timestamp request itself and asks only for the one signature. `--pkcs11 <module>` loads the token's PKCS#11 library (the only library `rubrapack` ever loads, and only this one you name), logs in with the PIN from `--pin-env` or `--pin-file` (never the command line), and uses the private key labelled `--key-label` with the certificate stored under the same ID on the token (`--cert` supplies it, or the rest of the chain, when the token has none); with several tokens present, `--token-label` picks one. On Windows, `--key-store <thumbprint>` uses a certificate in the current user's personal store (`--machine-store`: the local machine's) whose private key is reachable through NCrypt - a smart card, a token or TPM through its key storage provider, or a software key - and takes the chain Windows builds for it. RSA and ECDSA keys both work. Every signature is checked with the certificate before it is written.

- `keys list` shows the keys that can sign: with `--pkcs11`, each private key on the token(s) with its label, ID and certificate; on Windows without it, the certificates with a private key in the personal stores, with their thumbprints. For each it prints the MSIX publisher that certificate signs for and when it expires; nothing secret.
- `--timestamp <URL>` asks an RFC 3161 time-stamping server to countersign the signature, so that it stays valid after the certificate expires; without it `sign` and `build` `--key` warn that it will not. There is no default server and nothing goes over the network unless you name one. Public servers include `http://timestamp.digicert.com`, `http://timestamp.sectigo.com` and `http://time.certum.pl`; http is fine, because the answer is itself signed and checked (the nonce, the hash of the signature, the server's signature and its time-stamping certificate). `--tsa-trust` also requires the server's certificate path to end in one of the given certificates. For an https server rubrapack speaks TLS 1.3 itself (AES-GCM, x25519 or P-256, RSA-PSS or ECDSA server keys; not TLS 1.2) and checks the server's certificate - its path to a root given with `--tls-trust` or found in the operating system's store with `--system-roots`, its dates, that it names the server (subjectAltName; no wildcard across dots) and that it is for TLS servers; nothing turns these checks off. The three trusts stay apart: `--tls-trust` and `--system-roots` say whom to believe about the connection, `--tsa-trust` whom to believe about the time; neither is used for the other. On Linux `--system-roots` reads `$SSL_CERT_FILE` or the distribution's CA bundle; on Windows the ROOT store, which Windows fills with some roots only when something first needs them, so a root may be missing there until then. `--proxy http://host:port` sends the request through a proxy (an https server through CONNECT, so the proxy sees only the server's name); without it `https_proxy` or `HTTPS_PROXY` (for an https server) or `http_proxy` (for http; the upper-case `HTTP_PROXY` is not read, since a CGI environment can set it from a request header) are used unless `no_proxy` or `NO_PROXY` - names, each also covering the names under it, or `*` - lists the server. A proxy that asks for a password is not supported (a clear error), nor an `https://` proxy. When the timestamp fails - no answer, a refusal, a bad answer, a time outside the signing certificate's validity - nothing is signed and nothing is written (exit code 6); the signature never quietly goes out without it. The limits: 10 s to connect, 60 s in all, a 4 MiB answer, 3 redirects, 2 retries after a network error or a 5xx answer.
- `verify` prints what it checked - structure, digest, signature, the path to a certificate given with `--trust`, revocation (never looked up: not-checked) and timestamp - and succeeds only when all of them hold and the path ends in a trusted certificate. Without `--trust` it does not call anything trusted (exit code 4); `--system-roots` adds the operating system's roots to `--trust` for the signer's path (not for the timestamp's). It says nothing about Windows' own reputation checks. The timestamp line is not-present, `<time>`, TSA not-checked (give `--tsa-trust`), `<time>`, TSA trusted, `<time>`, TSA untrusted, invalid or unsupported (the older Authenticode timestamp that Set-AuthenticodeSignature -TimestampServer writes, which rubrapack does not check); the last three fail. Only a timestamp whose server is trusted through `--tsa-trust` - `--trust` does not count for it - moves the time at which the signer's certificate must have been valid from now to the stamped time. (For tests, the environment variable `RUBRAPACK_TEST_NOW` - seconds since 1970 - replaces "now" in `verify`'s checks, so that an expired certificate can be checked without changing a clock; it widens nothing that a clock set to that time would not.)
- Exit codes: 0 success, 1 error in the source, 2 usage, 3 input/output, 4 signing, 5 lint, 6 network.

20.6 Diagnostic codes

Every problem rubrapack reports has a code, `error[RPnnnn]` or `warning[RPnnnn]`, after the file, line and column it is about. The first two digits say what kind of problem it is:

Codes	What went wrong	Where to look
RP00xx	the command line: an unknown command or option, a file that cannot be read or written	rubrapack help <command>

Codes	What went wrong	Where to look
RP10xx	the source file's encoding: not UTF-8 (or UTF-16 with a BOM), stray carriage returns	save the file as UTF-8
RP11xx	TOML outside the subset rubrapack reads: multi-line strings, inline tables, a table defined twice	The TOML subset
RP12xx	tables and keys: an unknown table or key (with a suggestion), a required key or table missing, something without a feature once features exist	Tables
RP13xx	values: IDs (unique across all tables, not reserved), GUIDs, versions, numbers out of range, references to things that do not exist	the table's section
RP14xx	variables: \$(NAME) without a value, \$(not closed	Variables
RP15xx	the files to install: not found, a directory, a link, a glob without a match, a program for another architecture, names too long, a file that changed during the build, a package too large	Paths, Program files
RP16xx	MSIX: what an MSIX package needs, and what it cannot carry	MSIX packages
RP19xx	a feature this rubrapack does not provide	-
RP20xx	the finished tables break a Windows Installer rule. A source that passes the RP1xxx checks should never meet these: please report it	-
RP21xx	lint of a package: dialogs, code page, text normalisation	lint under Command line
RP22xx	lint of an MSIX package or bundle	lint under Command line
RP23xx	lint --previous: this package would not upgrade the previous one cleanly	Versions and upgrades

The message says what to change; the codes above are the ones to search the manual for.

Part III

Background knowledge

21 Bits, bytes and hexadecimal

In this chapter

21.1	Bits	93
21.2	Binary numbers	93
21.3	Bytes	94
21.4	Hexadecimal	94
21.5	Reading a hex dump	94
21.6	Units: KB, MB and KiB, MiB	95
21.7	Where this is used	95

This part explains the ideas the rest of the manual stands on, from the beginning. Nothing is assumed beyond using a computer. Each chapter ends with the places in the tutorial and in Part IV where its ideas are used.

21.1 Bits

Everything a computer stores - a program, a photo, this manual, an installer - is in the end a long row of **bits**. A bit has two values, written 0 and 1 (in the hardware: no charge or charge, one magnetic direction or the other). One bit can answer one yes/no question.

Several bits together can tell more values apart. Each bit added doubles the count:

Bits	Different values	All of them
1	2	0 1
2	4	00 01 10 11
3	8	000 001 010 011 100 101 110 111
8	256	00000000 ... 11111111
16	65,536	
32	4,294,967,296	

With n bits there are $2 \times 2 \times \dots \times 2$ (n times) = 2^n values.

21.2 Binary numbers

A row of bits can be read as a number, the same way we read decimal digits. In decimal, each place is worth ten times the place to its right: 305 is $3 \times 100 + 0 \times 10 + 5 \times 1$. In **binary** (base 2), each place is worth two times the place to its right:

bit:	1	0	1	1	0	0	1	0
worth:	128	64	32	16	8	4	2	1
	---		--	--			-	
	128	+	32	+	16	+	2	= 178

So 10110010 in binary is 178 in decimal. The rightmost bit, worth 1, is called bit 0; the leftmost of these eight, worth 128, is bit 7. Bit k is worth 2^k .

21.3 Bytes

Bits are grouped by eight into **bytes**. A byte is the smallest piece a file is made of: a file of 28,672 bytes - the size of the first `hello.msi` of the tutorial - is $28,672 \times 8 = 229,376$ bits. A byte holds one of 256 values, 0 to 255.

What a byte *means* depends entirely on how a program reads it: the same byte `01001000` is the number 72, or the letter H in a text file (chapter 3), or part of a larger number (chapter 2), or part of a machine instruction in a program. The file does not say; the file's *format* does. Part IV of this manual is the description of such formats.

21.4 Hexadecimal

Binary is long to write and hard to read. Decimal does not line up with bytes. So bytes are usually written in **hexadecimal** (base 16, "hex"): digits 0-9 and then A-F for ten to fifteen. One hex digit is exactly four bits, so one byte is exactly two hex digits:

Decimal	Binary	Hex	Decimal	Binary	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

To turn a byte into hex, split its eight bits into two halves of four and look each up:

```
10110010
1011 0010
  B    2    -> B2
```

And back: hex B2 is $B \times 16 + 2 = 11 \times 16 + 2 = 178$. The same number three ways: 178, 10110010, B2.

To make clear that a number is hex, programmers write `0x` before it (`0xB2`), or `h` after it (`B2h`), or just say so. This manual writes bytes as two hex digits with spaces between them, `B2 00 FF`, and numbers in hex with `0x`: `0x1000` is 4096.

Windows' Calculator has a *Programmer* mode that converts between decimal, hex and binary: type a number in one and read the others.

21.5 Reading a hex dump

A **hex dump** shows a file's bytes in hex, sixteen to a line. Here are the first 64 bytes of the first `hello.msi` of the tutorial:

```
offset  bytes                                     as text
00000000 d0 cf 11 e0 a1 b1 1a e1 00 00 00 00 00 00 00 00 .....
00000010 00 00 00 00 00 00 00 00 3e 00 04 00 fe ff 0c 00 .....>.....
00000020 06 00 00 00 00 00 00 00 01 00 00 00 01 00 00 00 .....
00000030 01 00 00 00 00 00 00 00 00 00 10 00 00 02 00 00 .....
```

- The **offset** on the left is the position of the line's first byte, counted from 0, in hex: `00000010` is byte 16, `00000030` byte 48. The byte at offset `0x1C` is the thirteenth of the second line: `fe`.
- The **bytes** follow, two hex digits each.

- The last column shows each byte as a letter where it is one (32 to 126, chapter 3) and . otherwise. Here there is hardly any text; in a text file the column would be readable.

The first eight bytes, `d0 cf 11 e0 a1 b1 1a e1`, are the *signature* of the Compound File format that every .msi uses - read with some imagination, "DOCFILE". Every file format begins with such a fixed pattern, a **magic number**, so programs can tell what a file is: a program (.exe) begins with `4d 5a` ("MZ"), a ZIP archive and so an MSIX with `50 4b 03 04` ("PK"), a cabinet with `4d 53 43 46` ("MSCF"). The file's name can lie; its first bytes rarely do.

To look at a file this way yourself, in PowerShell:

```
PS C:\work\hello> Format-Hex hello.msi | Select-Object -First 4
```

(Its layout differs a little from the one above; the bytes are the same.)

21.6 Units: KB, MB and KiB, MiB

Sizes are counted in bytes, with prefixes for large counts. There are two sets of prefixes, and confusing them is common:

Decimal (SI)	Bytes	Binary (IEC)	Bytes
1 kB (kilobyte)	1,000	1 KiB (kibibyte)	$1,024 = 2^{10}$
1 MB (megabyte)	1,000,000	1 MiB (mebibyte)	$1,048,576 = 2^{20}$
1 GB (gigabyte)	1,000,000,000	1 GiB (gibibyte)	$1,073,741,824 = 2^{30}$
1 TB (terabyte)	10^{12}	1 TiB (tebibyte)	2^{40}

Binary sizes appear because memory and file formats work in powers of two. The dump above shows one: at offset `0x1E` the Compound File header stores `0c 00`, the number 12, and its sectors are $2^{12} = 4096$ bytes (4 KiB, $0x1000$) long - the format stores the power, not the size. An MSIX block map entry covers 64 KiB. Windows Explorer says "KB" and "MB" but means KiB and MiB; disk makers mean the decimal ones, which is why a "1 TB" disk shows as 931 GB.

rubrapack uses the binary units where a limit is a power of two: `cab-max-size` is in MiB, and Windows Installer's limit on an .msi is 2 GiB (2,147,483,648 bytes). When this manual writes "MB" for a program's size (as in "a 200 MB program"), it is a rough size, and the difference does not matter.

21.7 Where this is used

- Every chapter of Part IV shows file contents as hex dumps and offsets, starting with Compound File Binary.
- Tutorial chapter 15: `cab-max-size` in MiB and the 2 GiB limit.
- Tutorial chapter 18: file sizes in `inspect --files` are bytes.

22 Numbers in files, and bit flags

In this chapter

22.1	One, two, four, eight bytes	96
22.2	Byte order: little-endian	96
22.3	Negative numbers: two's complement	97
22.4	Bit flags	97
22.5	Testing and setting a bit: AND, OR	98
22.6	Where this is used	98

A file format stores numbers - sizes, counts, positions, versions - as a fixed number of bytes. This chapter shows how, and how one number can carry many yes/no settings at once.

22.1 One, two, four, eight bytes

A number stored in a file takes a fixed number of bytes, chosen by the format:

Size	Bits	Unsigned range	Common names
1 byte	8	0 to 255	byte, u8, uint8
2 bytes	16	0 to 65,535	word, u16, uint16, short
4 bytes	32	0 to 4,294,967,295	double word (DWORD), u32, uint32, int
8 bytes	64	0 to about 1.8×10^{19}	quad word (QWORD), u64, uint64

Unsigned means no negative numbers. A number too big for its size cannot be stored - which is why an MSI table column declared i2 (2 bytes) cannot hold 100,000, and why a Compound File, whose positions are 4-byte numbers of 4 KiB sectors, has an upper size limit.

22.2 Byte order: little-endian

A number of several bytes is split into bytes. Take the size of `hello.exe` in the tutorial, 17,920 bytes. In hex it is `0x00004600` - as four bytes, `00 00 46 00`, the most significant first. But files for Windows almost always store the bytes the other way round, **least significant first**:

```
17920 = 0x00004600

most significant first (big-endian):  00 00 46 00
least significant first (little-endian): 00 46 00 00

      |  |  |  |
      |  |  |  +-- x 16,777,216 (256^3) = 0
      |  |  +----- x 65,536      (256^2) = 0
      |  +----- x 256              = 17,920  (0x46 =
70)
      +----- x 1                  = 0
```

This order is called **little-endian** ("the little end first"); the other is **big-endian**. Intel and Arm processors in Windows computers work little-endian, and so do the MSI, CAB, ZIP and program file formats. Network protocols and certificates (chapter 8) use big-endian. When a Part IV table says "u32 LE", it means "an unsigned 4-byte number, little-endian".

To read a little-endian number from a dump: take its bytes, reverse them, read the result as hex. From the Compound File header of chapter 1:

Offset	Bytes	Reversed	Value	Meaning
0x18	3e 00	00 3e	62	minor version
0x1A	04 00	00 04	4	major version: 4 KiB sectors
0x1C	fe ff	ff fe	0xFFFFE	the byte order mark: a reader that gets 0xFEFF knows it read the wrong way round
0x1E	0c 00	00 0c	12	sector size is $2^{12} = 4096$
0x38	00 10 00 00	00 00 10 00	4096	streams smaller than this go into the mini stream

22.3 Negative numbers: two's complement

Where a format needs negative numbers, it uses **two's complement**: the same bits, with the top half of the range read as negative. In one byte, 0 to 127 are themselves and 128 to 255 stand for -128 to -1:

Byte	Unsigned	Signed
00	0	0
7f	127	127
80	128	-128
fe	254	-2
ff	255	-1

The rule: a signed value is negative when its top bit is 1, and its value is the unsigned value minus 2^{bits} . In four bytes, ff ff ff ff is -1 and fe ff ff ff is -2. Formats like to use such "all ones" values as markers: in a Compound File, the sector number 0xFFFFFFF (2) means "end of the chain" and 0xFFFFFFF (-1) "free".

22.4 Bit flags

Often a format needs many yes/no settings for one thing. Rather than a byte for each, it gives each setting one **bit** of a number, and the number is the sum of the bits that are on. Each bit is worth a power of two - 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, ... - so every sum has exactly one way to be made, and the settings can be read back from it.

The tutorial met several such numbers. Here they are taken apart - write the number in binary, and each 1 is one setting:

Number	Binary	Bits on	Meaning (tutorial chapter)
258	1 0000 0010	256 + 2	Upgrade row: minimum version included, only detect (3)
272	1 0001 0000	256 + 16	Component: 64-bit, permanent (4)
512	10 0000 0000	512	File: vital (2, 18)
163	1010 0011	128 + 32 + 2 + 1	ServiceControl: delete at removal, stop at removal, stop at install, start at install (12)
3090	1100 0001 0010	2048 + 1024 + 16 + 2	Custom action: no impersonation, deferred, source is a File, an .exe (13)
3410	1101 0101 0010	2048 + 1024 + 256 + 64 + 16 + 2	the same, plus rollback and ignore the exit code (13)

To take a number apart by hand, subtract the largest power of two that fits, and repeat: $3090 - 2048 = 1042$; $1042 - 1024 = 18$; $18 - 16 = 2$; $2 - 2 = 0$. So $3090 = 2048 + 1024 + 16 + 2$. (Microsoft's documentation calls the custom action's $2 + 16$ together "type 18": run an .exe that the package installs.)

Hex makes flags easy to see, because each hex digit is four bits: 3090 = 0xC12, and C = 1100 means 2048 and 1024, 1 = 0001 means 16, 2 = 0010 means 2.

22.5 Testing and setting a bit: AND, OR

Programs test a flag with **AND**: keep only the bits that are 1 in both numbers.

```
3090 = 1100 0001 0010
AND 1024 = 0100 0000 0000
        = 0100 0000 0000    not zero: "deferred" is on
```

They set one with **OR** (a bit is 1 if it is 1 in either) and clear one with AND of the opposite. Windows Installer conditions (tutorial chapter 9) have the same test as the operator ><: MYFLAGS >< 2 is true when the bit worth 2 is on in the property MYFLAGS. You will not need these for rubrapack - it computes all these numbers - but they explain why the numbers look the way they do.

22.6 Where this is used

- Every "u16", "u32 LE" in Part IV, starting with the header and sector chains of Compound File Binary.
- Tutorial chapters 2, 3, 4, 12, 13: the numbers in the tables.

23 Text: characters, code points and encodings

In this chapter

23.1	Characters and code points	99
23.2	ASCII	99
23.3	Code pages: one byte, 256 characters	100
23.4	UTF-8	100
23.5	UTF-16	101
23.6	The byte order mark	101
23.7	Unicode normalization	101
23.8	When text looks wrong	102
23.9	Where this is used	102

A file holds bytes, and a byte is a number (chapter 1). Text - a product name, a file name, a license - is stored by giving every character a number and writing the numbers as bytes. There are several ways to do both, and a package that uses the wrong one shows garbage. This chapter explains the ways rubrapack and Windows use.

23.1 Characters and code points

Unicode is the world's catalogue of characters: letters of every script, digits, punctuation, symbols, emoji - about 160,000 of them. Each has a number, its **code point**, written U+ and at least four hex digits:

Character	Code point	Name in the catalogue
H	U+0048	LATIN CAPITAL LETTER H
é	U+00E9	LATIN SMALL LETTER E WITH ACUTE
€	U+20AC	EURO SIGN
한	U+D55C	HANGUL SYLLABLE HAN
😊	U+1F600	GRINNING FACE

Code points go up to U+10FFFF. A code point says *which* character; it does not yet say which bytes to write. That is the job of an **encoding**.

23.2 ASCII

The oldest encoding still everywhere is **ASCII** (1963): 128 characters, code points 0 to 127, one byte each. It has the English letters, digits, punctuation and some control characters:

Bytes	Characters
00-1f	control characters: 09 tab, 0a line feed (new line), 0d carriage return
20	space
30-39	0-9
41-5a	A-Z
61-7a	a-z
2e 2f 3a 5c	. / : \

The tutorial's `dist\docs\guide.txt` holds six bytes:

```
47 75 69 64 65 0a
G u i d e (new line)
```

The first 128 code points of Unicode are ASCII's, and every encoding below writes them as the same single byte. That is why English text looks the same in almost any encoding - and why problems show only with other letters.

Windows ends a line with two bytes, 0d 0a (carriage return, line feed); Linux and macOS with 0a alone. Most programs read both.

23.3 Code pages: one byte, 256 characters

ASCII leaves the values 128 to 255 of a byte unused. For decades each language region filled them differently, and each filling is a **code page**, numbered by Microsoft:

Code page	Region	é	€	한
1252	Western Europe	e9	80	-
949	Korean	-	a2 e6	c7 d1
65001	all: this is UTF-8	c3 a9	e2 82 ac	ed 95 9c

A code page can hold only its own region's characters (949 uses two bytes for Korean). And bytes mean different things in different code pages: the byte e9 is é in 1252 but half of a Korean character in 949. Text written in one and read in another comes out as garbage, known by the Japanese word *mojibake*.

Every MSI package has one code page for all its text. rubrapack always writes 65001, UTF-8, so any language fits into one package; tutorial chapter 7 shows inspect reporting it. Part IV shows a trap: the summary information has a code page of its own.

23.4 UTF-8

UTF-8 writes every Unicode code point in one to four bytes. ASCII stays one byte, so an ASCII file is already UTF-8. Larger code points use more bytes, following a fixed bit pattern:

Code points	Bytes	Bit pattern (x = bits of the code point)
U+0000 - U+007F	1	0xxxxxxx
U+0080 - U+07FF	2	110xxxxx 10xxxxxx
U+0800 - U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 - U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Worked out for 한, U+D55C:

```
D55C in binary:    1101 0101 0101 1100      (16 bits)
split 4 + 6 + 6:   1101   010101   011100
into the pattern:  1110 1101  10 010101  10 011100
in hex:           ED      95      9C
```

So 한 is ed 95 9c in UTF-8. The pattern makes UTF-8 easy to check: a byte starting 10 is never the first byte of a character, so a reader can always find where characters begin, and bytes of another encoding almost never form valid UTF-8 by accident. rubrapack stops at the first byte of a source that breaks the pattern (RP1001: not valid UTF-8) rather than guessing.

23.5 UTF-16

UTF-16 writes code points up to U+FFFF as one 2-byte unit, and the rest as two units (a *surrogate pair*). Windows uses it inside: file names, the registry, and every text in a Compound File's directory are UTF-16, little-endian (chapter 2):

H	e	l	l	o		
48 00	65 00	6c 00	6c 00	6f 00		"Hello" in UTF-16LE: ASCII letters get a 00 after them
한						
5c d5						U+D55C: the two bytes of D55C, little end first
😊						
3d d8 00 de						U+1F600: too big for one unit, so the pair D83D DE00

A text in a dump that looks like letters with dots between them - H.e.l.l.o. - is UTF-16.

23.6 The byte order mark

A file may start with the code point U+FEFF, the **byte order mark** (BOM), to say which encoding it is: ef bb bf for UTF-8, ff fe for UTF-16LE. Windows Notepad used to add one to UTF-8 files. rubrapack build and lint read a source in UTF-8, with or without a BOM, and in UTF-16 with its BOM; edit changes UTF-8 sources only.

23.7 Unicode normalization

Some characters can be written in Unicode in more than one way. é is one code point, U+00E9 - or two: the letter e, U+0065, followed by U+0301, a *combining acute accent* that sits on the letter before it. Both look exactly the same on screen:

Form	Code points	UTF-8 bytes
composed (NFC)	U+00E9	c3 a9
decomposed (NFD)	U+0065 U+0301	65 cc 81

Korean is the largest case. Every modern Hangul syllable has a composed code point, and can also be written as its letters (*jamo*):

Form	Code points	UTF-8 bytes
NFC	U+D55C 한	ed 95 9c
NFD	U+1112 ㅎ U+1161 ㅏ U+11AB ㄴ	e1 84 92 e1 85 a1 e1 86 ab

(The composed code point is computed from the letters: $0xAC00 + (18 \times 21 + 0) \times 28 + 4 = 0xD55C$, where 18, 0 and 4 are the numbers of ㅎ, ㅏ and ㄴ in the Unicode tables.)

Normalization turns text into one agreed form. NFC ("composed") is the form Windows, the web and almost every keyboard produce. NFD ("decomposed") is what macOS file systems store file names in - so a file copied from a Mac may have a name whose bytes differ from the same name typed on Windows. For Windows the two are *different names*: a package could then install 한.txt twice, or a shortcut could miss its target.

That is why rubrapack has build --nfc (tutorial chapter 15), which turns file, folder and shortcut names into NFC, and why lint warns (RP2105) about text that is not in NFC (tutorial chapter 18). rubrapack's normalization follows the Unicode 17.0 tables.

23.8 When text looks wrong

You see	Likely cause
Ã© instead of é	UTF-8 bytes c3 a9 read as code page 1252
◆뵆 or ? instead of 한	UTF-8 bytes read as code page 949, or the other way round
H.e.l.l.o. in a dump	UTF-16 read as bytes
two names that look the same but are not	one NFC, one NFD

23.9 Where this is used

- Tutorial chapter 7: texts stored as UTF-8, code page 65001.
- Tutorial chapters 15 and 18: --nfc and RP2105.
- Part IV: the string pool of an MSI database, the UTF-16 names in a Compound File, the summary information's code page.

24 GUIDs and hashes

In this chapter

24.1	GUIDs	103
	Version and variant	103
	In text and in bytes	104
24.2	Hashes	104
	Hashes in the tutorial's packages	104
24.3	GUIDs from hashes: <code>guid --from</code>	105
24.4	Where this is used	105

Two kinds of 16-to-32-byte numbers appear everywhere in packages: **GUIDs**, which name things so that no two names ever collide, and **hashes**, which sum up a file's contents so that any change shows. `rubrapack` also joins the two: it makes GUIDs *from* hashes.

24.1 GUIDs

A **GUID** (globally unique identifier; the standards call it a UUID) is a 128-bit number - 16 bytes - written as 32 hex digits in five groups, usually in braces:

```
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}
8 digits  4      4      4      12 digits
```

Windows uses GUIDs to name things that must never be confused with anything else, anywhere: a product (the product code), a product family across versions (the upgrade code), a component, a package build (the package code), a kind of COM object. There is no registry of GUIDs; nobody hands them out. They are unique because there are so many: 2^{122} possible random ones, about 5×10^{36} . A computer making a billion random GUIDs every second would need about 86 years before the chance that any two of them were equal reached one half.

Version and variant

Not all 128 bits are free. Two places in the text say what kind of GUID it is:

```
{98AE4FED-BF0B-4C77-B712-E0649EB47178}    rubrapack guid        (random)
      ^      ^
      |      variant: 8, 9, A or B (the bits 10xx)
      |      version: 4 = random

{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}    rubrapack guid --from hello  (derived)
      ^      ^
      |      variant: 9
      |      version: 8 = made by a method of the maker's own choosing
```

A **version 4** GUID is 122 random bits: `rubrapack guid` makes one for your upgrade code. A **version 8** GUID is 122 bits computed in a way its maker defines - `rubrapack` computes them from names, below.

In text and in bytes

In text, a GUID is its 32 digits in order. In a binary structure (a Compound File's directory, a program file), Windows stores the first three groups as little-endian numbers (chapter 2) and the last two as plain bytes:

```
text:   B52FE8EF - B68D - 84B5 - 91AF - E6E01BEC2773
bytes:  ef e8 2f b5  8d b6  b5 84  91 af  e6 e0 1b ec 27 73
        (reversed) (rev.) (rev.) (as written)
```

MSI tables keep GUIDs as text, in upper case, with braces. The mixed order matters only in Part IV.

24.2 Hashes

A **hash function** reads any amount of data and produces a short, fixed-size number from it, the **hash** (or *digest*). A good one has three properties:

1. **Same input, same hash** - on every computer, every time.
2. **Any change changes everything**. Change one bit of the input and about half the bits of the hash change, with no visible pattern.
3. **One-way**. From a hash you cannot find an input that gives it, except by trying inputs forever.

Take the tutorial's `guide.txt`, six bytes `47 75 69 64 65 0a` ("Guide" and a new line), and the same with a lower-case `g`, one bit different (`47 = 0100 0111`, `67 = 0110 0111`):

Input	MD5 (16 bytes)
Guide\n	2b 67 39 28 1f 93 72 44 2d f3 0e 78 4b 7d bd c7
guide\n	e5 24 11 d5 57 3e b8 9b 5c 31 26 97 79 50 d3 ef

Input	SHA-256 (32 bytes)
Guide\n	32 74 fc ad 88 6c de 4e 2c a8 6b 11 d3 0f d7 c4 48 58 ea df 1c 43 7a 95 83 f3 1e 78 15 db 1a f6
guide\n	90 c3 90 ec 1d e8 06 bf 94 58 85 cd 0a f5 1e 90 c3 cd 8c da 0d 0f f6 76 05 1a 56 c2 08 48 c9 0f

Try it in PowerShell: `Get-FileHash -Algorithm MD5 dist\docs\guide.txt`.

The common hash functions:

Name	Size	Use today
MD5	128 bits	only to notice changes; it is broken for security (different inputs with the same MD5 can be made on purpose)
SHA-1	160 bits	the same; still used as a certificate's <i>thumbprint</i> (chapter 8)
SHA-256	256 bits	the standard for signatures, and what rubrapack uses to derive GUIDs

Hashes in the tutorial's packages

MsiFileHash: for each file without a version, Windows Installer keeps its MD5 to decide at a repair or upgrade whether the file on disk still is the one the package installed. `inspect --files` prints it as hex (tutorial chapter 18). The table itself stores it as four 4-byte numbers - the sixteen bytes read four at a time, little-endian and signed (chapter 2):

```
C:\work\hello> rubrapack inspect hello.msi MsiFileHash
File_ Options HashPart1 HashPart2 HashPart3 HashPart4
...
F_6d1508f52f6615459567  0 674850603 1148359455 2014245677 -943882933

2b 67 39 28 -> 0x2839672B = 674850603
```



```
1f 93 72 44 -> 0x4472931F = 1148359455
2d f3 0e 78 -> 0x780EF32D = 2014245677
4b 7d bd c7 -> 0xC7BD7D4B = 3351084363, top bit set, minus 2^32 = -943882933
```

- **MSIX block map** (tutorial chapter 17): the SHA-256 of every 64 KiB block of every file.
- **Signatures** (tutorial chapter 16): what is signed is a hash of the package, not the package.
- **--reproducible** (tutorial chapter 15): the package code is derived from a hash of the content.

24.3 GUIDs from hashes: guid --from

A component's GUID must stay the same from one version to the next - the upgrade relies on it - but nobody should have to write hundreds of them down. So rubrapack *derives* each one: it hashes a text that names the thing (the product and the file's place) and turns the hash into a GUID. The same text always gives the same GUID; different texts give different GUIDs.

rubrapack guid --from <text> shows the calculation's result for any text. Step by step, for hello:

1. Build the input bytes.

Each part is its length as a 4-byte little-endian number, then its bytes: the word guid, the number 1 (the method's version), and the text in UTF-8.

```
04 00 00 00 67 75 69 64 01 00 00 00 05 00 00 00 68 65 6c 6c 6f
length 4      "guid"      1      length 5      "hello"
```

Writing the lengths makes the parts unambiguous: guid + hello cannot be confused with guidh + ello.

2. Hash them with SHA-256:

```
b5 2f e8 ef b6 8d f4 b5 d1 af e6 e0 1b ec 27 73 a2 8a 56 80 0f 9f 53 1b cd dc ee 7c f5 c8 59 90
```

3. Keep the first 16 bytes:

```
b5 2f e8 ef b6 8d f4 b5 d1 af e6 e0 1b ec 27 73.
```

4. Mark the version and variant.

The top four bits of byte 6 become 1000 (version 8): f4 becomes 84. The top two bits of byte 8 become 10 (the variant): d1 = 1101 0001 becomes 1001 0001 = 91.

```
b5 2f e8 ef b6 8d 84 b5 91 af e6 e0 1b ec 27 73
      ^^      ^^
```

5. Write it as a GUID

, the bytes in order, upper case:

```
C:\work\hello> rubrapack guid --from hello
{B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}
```

Any program can repeat this and get the same GUID; Part IV, Deterministic identities, lists the exact texts rubrapack hashes for each kind of thing.

24.4 Where this is used

- Tutorial chapter 2: the upgrade code, made with rubrapack guid.
- Tutorial chapters 4 and 15: component GUIDs and the package code, derived.
- Tutorial chapters 16, 17, 18: signatures, block maps, file hashes.
- Part IV: Deterministic identities, Authenticode, MSIX.

25 Compression

In this chapter

25.1	Why data can be made smaller	106
25.2	LZ77: "copy from earlier"	106
25.3	Huffman coding: short codes for common symbols	107
25.4	Deflate, bit by bit	107
25.5	MSZIP: deflate in a cabinet	108
25.6	Deflate in an MSIX	108
25.7	Where this is used	108

An MSI's files travel in a cabinet, an MSIX's in a ZIP archive, and in both they are usually *compressed*: written in fewer bytes, from which the exact original comes back. This chapter shows how the method both use, **deflate**, does that - down to the bits of a real example.

25.1 Why data can be made smaller

Compression without loss works because real data is not random. Two things repeat:

1. **Sequences.** A program's machine code repeats instruction patterns; a text repeats words; a picture of a blue sky repeats the same colour.
2. **Symbols, unevenly.** In English text e is far more common than z; in a program the byte 00 is very common.

Deflate uses both, one after the other: **LZ77** replaces repeated sequences by short references, then **Huffman coding** writes common symbols in fewer bits.

Data with neither - random numbers, or data that is compressed already, such as PNG and JPEG pictures, ZIP files, video - cannot be made smaller. Trying costs time and adds a few bytes.

25.2 LZ77: "copy from earlier"

LZ77 (Lempel and Ziv, 1977) reads the data from the start and, wherever the next bytes already appeared recently, writes a **back-reference** instead: "go back *distance* bytes and copy *length* bytes from there". Everything else is written as **literals**, byte by byte.

Take the 12 bytes `abcbabcabc`. Deflate writes them as four literals and one reference:

```
literal a
literal b
literal c
literal a
copy: distance 3, length 8
```

The copy may overlap what it produces: going back 3 bytes lands on `bca`, and copying byte by byte keeps reading bytes that were just written:

```
written so far:  a b c a
                  ^ start 3 back
copy 8:          a b c a b c a b c
                  \_____/ 8 copied bytes
```

So a run of any length is one short reference. Deflate allows distances up to 32,768 bytes (the *window*) and lengths from 3 to 258.

25.3 Huffman coding: short codes for common symbols

After LZ77 the data is a series of symbols: literal bytes, lengths and distances. A normal byte takes 8 bits whatever it is. A **Huffman code** gives each symbol its own number of bits - fewer for frequent ones - such that no code is the beginning of another, so a reader always knows where one ends.

A small example with four symbols:

Symbol	Frequency	Fixed 2-bit code	Huffman code
A	50%	00	0
B	25%	01	10
C	12.5%	10	110
D	12.5%	11	111

On average this takes $0.5 \times 1 + 0.25 \times 2 + 0.125 \times 3 + 0.125 \times 3 = 1.75$ bits per symbol instead of 2. ABAD becomes 0 10 0 111, and reading 0100111 from the left can only mean A, B, A, D.

Deflate has two kinds of code tables: a **fixed** table defined once in the standard (good for small data), and **dynamic** tables, built for each block from its own symbol counts and stored at the block's start (better for large data).

25.4 Deflate, bit by bit

Deflate (RFC 1951) writes blocks. Each starts with a three-bit header:

Bits	Meaning
1 bit: BFINAL	1 = this is the last block
2 bits: BTYPE	0 = stored (bytes as they are), 1 = fixed Huffman codes, 2 = dynamic Huffman codes

In the fixed table, the symbols are:

Symbols	Meaning	Code length	Codes
0-143	literal byte 0-143	8 bits	00110000 + symbol
144-255	literal byte 144-255	9 bits	110010000 + (symbol - 144)
256	end of block	7 bits	0000000
257-279	a length (plus extra bits)	7 bits	0000001 ...
280-287	a length (plus extra bits)	8 bits	11000000 ...

After a length comes a distance code of 5 bits, and some codes add a few *extra bits* to pick an exact value within a range.

Here is abcabcbcabcb compressed by deflate - seven bytes instead of twelve:

```
4b 4c 4a 4e 84 21 00
```

Deflate fills each byte starting with its lowest bit (bit 0, chapter 2). Written out that way, the bit stream reads:

```
byte 4b = 01001011 -> bits in order: 1 1 0 1 0 0 1 0
byte 4c = 01001100 -> bits in order: 0 0 1 1 0 0 1 0
...
```

and decodes as:

Bits (in order)	Meaning
1	BFINAL = 1: the last block
1 0	BTYPE = 1 (the two bits read as a number, low bit first): fixed codes
10010001	literal a (0x61 = 97; 00110000 + 97 = 10010001)
10010010	literal b
10010011	literal c
10010001	literal a
0000110	symbol 262: length 8
00010	distance code 2: distance 3
0000000	symbol 256: end of block

That is $3 + 4 \times 8 + 7 + 5 + 7 = 54$ bits; the last two bits of the seventh byte are unused. (Huffman codes are the one thing deflate writes from the code's *highest* bit first; numbers such as BTYPE and the extra bits go lowest bit first. Part IV spells out every table.)

25.5 MSZIP: deflate in a cabinet

A cabinet (.cab) splits the files' data into blocks of at most 32 KiB. With **MSZIP** each block is the two letters CK (43 4b) followed by one deflate stream, and each block may refer back into the previous one. `compress = "mszip:6"` in a source is deflate at effort 6 of 9: a higher number searches longer for matches - smaller output, more time - and the result unpacks the same way.

25.6 Deflate in an MSIX

An MSIX is a ZIP archive, and ZIP stores each file either as it is ("stored", method 0) or compressed with deflate (method 8). The tutorial's `hello.msix` (chapter 17):

File	Size	In the package	
<code>hello.exe</code>	17,920	6,706	deflate: 37%
<code>Registry.dat</code>	8,192	430	deflate: mostly zero bytes, 5%
<code>AppxManifest.xml</code>	2,537	900	deflate: text, 35%
<code>Assets\Square150x150.png</code>	301	301	stored: PNG is compressed already
<code>guide.txt</code>	6	14	deflate: too small to gain

The last line shows the cost of compressing tiny data: a deflate block has a few bits of header and an end code, and six bytes contain nothing to repeat.

25.7 Where this is used

- Tutorial chapter 15: `compress`, `cab`, `cab-max-size`.
- Tutorial chapter 17: `--msix-compress`.
- Part IV: Cabinets, MSZIP and deflate, MSIX.

26 Files inside a file

In this chapter

26.1	How a disk stores files	109
26.2	An MSI is a file system in a file	109
	Small streams: the mini stream	110
	The names	110
26.3	An MSIX is an archive	110
26.4	Why two designs?	111
26.5	Where this is used	111

A package is one file that holds many: an `.msi` holds its tables, the summary and the cabinet; an `.msix` holds the program files, the logos and the manifest. There are two classic ways to put files inside a file - as a small **file system** or as an **archive** - and the two package formats use one each. This chapter explains both with the tutorial's own packages.

26.1 How a disk stores files

A disk is divided into equal pieces, **sectors** (or *clusters*) - say 4096 bytes each - numbered 0, 1, 2, ... A file rarely fits in one, and the sectors it gets need not be next to each other. So a file system keeps two things besides the data:

- a **directory**: for each file its name, its size, and its *first* sector;
- an **allocation table**: for each sector, the number of the *next* sector of the same file, or a marker for "last sector" or "free".

To read a file, look up its first sector in the directory, then follow the table from sector to sector - a **chain** - until the "last" marker:

```
directory:  report.txt    size 10,000    first sector 4

allocation table:
sector:  0    1    2    3    4    5    6    7
next:    -    -    -    -    5    7    free end

report.txt = sector 4 -> sector 5 -> sector 7 -> end    (3 x 4096 bytes, the last one partly
used)
```

This is how the FAT file system of USB sticks works (FAT: *file allocation table*). Deleting a file only marks its sectors free; growing one only links more sectors to its chain.

26.2 An MSI is a file system in a file

An `.msi` is a **Compound File** (Microsoft's "structured storage"): exactly the scheme above, inside one file. Here is the first `hello.msi` of the tutorial, 28,672 bytes, taken apart:

```
offset 0x0000  header          (the 512-byte header, padded to one 4096-byte sector)
offset 0x1000  sector 0         the allocation table (FAT)
offset 0x2000  sector 1         the directory
offset 0x3000  sector 2         the mini FAT (see below)
offset 0x4000  sector 3         the mini stream (see below)
```

offset 0x5000	sector 4	cab1.cab, first part
offset 0x6000	sector 5	cab1.cab, second part

Sector n starts at byte $(n + 1) \times 4096$, because the header takes the first 4096 bytes. The header (chapter 1) says where the rest is: 01 00 00 00 at offset 0x30 - the directory starts at sector 1. The allocation table in sector 0 reads, as 4-byte little-endian numbers (chapter 2):

Sector	Entry	Meaning
0	0xFFFFFFFF	this sector holds the allocation table itself
1	0xFFFFFFFF	end of chain: the directory is one sector
2	0xFFFFFFFF	end of chain: the mini FAT is one sector
3	0xFFFFFFFF	end of chain: the mini stream is one sector
4	5	next is sector 5
5	0xFFFFFFFF	end of chain: cab1.cab is sectors 4 and 5
6 ...	0xFFFFFFFF	free (there are no more sectors)

A file inside is called a **stream**. The directory has one 128-byte entry per stream: its name in UTF-16 (chapter 3), its type, its first sector and its size. The first entry is always the root:

```
52 00 6f 00 6f 00 74 00 20 00 45 00 6e 00 74 00 72 00 79 00 00 00 ...
R   o   o   t           E   n   t   r   y   (end)
```

Small streams: the mini stream

A 4096-byte sector would waste most of its space on a 20-byte table. So streams smaller than 4096 bytes (the number at offset 0x38 of the header) are kept together in one stream, the **mini stream**, cut into **mini sectors** of 64 bytes, with their own table, the **mini FAT**. In this package, nineteen streams - eighteen tables of 4 to 1,392 bytes and the 332-byte summary - share the 3,840-byte mini stream in sector 3; only the 6,776-byte cabinet is big enough for ordinary sectors. It is the same scheme again, one level down.

The names

The stream names of an MSI look strange - 羶甄縻桃獬菴踰 - because Windows Installer packs each table name into characters of the CJK range to fit more letters in the 31 characters a name may have. rubrapack inspect hello.msi --streams (tutorial chapter 18) shows them decoded: File, Component, cab1.cab, ... One name is plain: \005SummaryInformation, the summary, which every Compound File tool can read.

26.3 An MSIX is an archive

An **archive** is simpler than a file system: the files are written one after the other, each after a small header, and a list at the end says where each one is. It cannot grow a file in place, but a package never needs to. The ZIP format, which MSIX uses, looks like this:

```
"PK 03 04" local header: name, sizes, method + the file's (compressed) bytes hello.exe
"PK 03 04" local header + bytes guide.txt
...
files)
"PK 01 02" central directory: one entry per file, with where its local header is
"PK 06 06" end record: how many files, where the central directory starts
```

The tutorial's hello.msix (11,009 bytes) begins with 50 4b 03 04 - "PK", the initials of Phil Katz, who made ZIP - followed by the name hello.exe. Its central directory starts at byte 10,098 and lists nine

entries; a reader goes to the end of the file first, finds the end record, and from there the list. (MSIX uses the ZIP64 form of the end record, PK 06 06, made for archives above 4 GiB.)

Each entry says its method (chapter 5): 8 for deflate, 0 for stored - here 0 for the three PNG logos, 8 for everything else.

26.4 Why two designs?

	Compound File (MSI)	ZIP (MSIX)
Made for	documents edited in place (Word, Excel of the 1990s)	files packed once, read many times
Changing a stream	rewrite its chain	rewrite the archive
Small parts	cheap, in the mini stream	each has a header of 30+ bytes
Reading one part	follow its chain	look it up in the central directory

Windows Installer was built in 1999 on the storage Office used; MSIX in 2018 on the packaging standard (OPC) that .docx files use - which is a ZIP. Both are "files inside a file".

26.5 Where this is used

- Tutorial chapter 18: `inspect --streams`, `extract`.
- Part IV: Compound File Binary, the MSI database (stream names), MSIX (ZIP layout).

27 Tables, keys and references

In this chapter

27.1	Tables, rows and columns	112
27.2	Primary keys: what makes a row unique	112
27.3	Foreign keys: rows that point at rows	113
27.4	Keys you give, keys rubrapack makes	113
27.5	Storing strings once: the string pool	113
27.6	Tables about tables	114
27.7	The IDT text format	114
27.8	Where this is used	114

Inside an .msi (chapter 6) the streams are mostly **tables**: the package is a small **database**. Windows Installer does not follow a script of steps written by you; it reads the tables and works out what to do. This chapter explains the few database ideas needed to read them.

27.1 Tables, rows and columns

A **table** is a grid. Each **column** has a name and a type; each **row** is one thing - one file, one shortcut, one registry value. Here is the File table of the tutorial's first package (chapter 2 of the tutorial), as rubrapack inspect hello.msi File prints it:

```
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 1255 i4 S72 S20 I2 i4
File File
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 1
```

Laid out as a grid:

File	Component_	FileName	FileSize	Version	Language	Attributes	Sequence
Hello	C_185f8db32271fe25f561	hello.exe	17920	1.2.3.4	1033	512	1

The second line of the text gives the **type** of each column:

Type	Meaning
s72	a string of at most 72 characters
1255	a <i>localizable</i> string (one a translation may change) of at most 255 characters
i2, i4	an integer of 2 or 4 bytes (chapter 2)
capital S, L, I	the same, but the cell may be empty ("null")

A column's type is fixed by Windows Installer for each standard table, and rubrapack checks every value against it (lint, tutorial chapter 18): a 100,000 in an i2 column would not fit.

27.2 Primary keys: what makes a row unique

The third line, File File, names the table and its **primary key** - the column (or columns) whose value is different in every row. In File it is the column File: Hello names this row and no other row may use that name. This is why rubrapack's IDs must be unique (tutorial chapter 2): the ID you write in [file.Hello] becomes this key.

Some tables need two columns together. FeatureComponents says which component belongs to which feature; one feature has many components and one component may be in several features, so only the pair is unique:

```
Feature_ Component_
s38 s72
FeatureComponents Feature_ Component_
Main C_185f8db32271fe25f561
```

27.3 Foreign keys: rows that point at rows

A column whose name ends in `_` holds the key of a row in *another* table - a **foreign key**, or reference. Following them connects the tables into one picture. Start at the file and follow the references:

```
File          Hello
Component_ --> Component C_185f8db32271fe25f561
                ComponentId {B80BEC59-F582-8F10-8EB1-638C6687B899}
                Directory_ --> Directory INSTALLDIR
                                DefaultDir Hello
                                Directory_Parent --> Directory ProgramFiles64Folder
                                                DefaultDir .
                                                Directory_Parent --> TARGETDIR
                KeyPath --> File Hello (back to the start)
FeatureComponents Main + C_185f8db32271fe25f561
Feature_ --> Feature Main
                Level 1 (installed by default)
```

Read aloud: the file `hello.exe` belongs to a component, which installs into the folder `Hello` inside `ProgramFiles64Folder` (Program Files), and which the feature `Main` installs. The component's *key path* is the file itself: if `hello.exe` is there, the component counts as installed (tutorial chapter 4).

A reference to a row that does not exist is an error Windows Installer would stop on - one of the things `rubrapack lint` checks in any `.msi`, from any tool.

27.4 Keys you give, keys rubrapack makes

In the tables, the IDs from your source appear as they are - `Hello`, `INSTALLDIR`, `Main`. Where Windows Installer needs a row that has no table of its own in the source, `rubrapack` makes a key: a component per file is named `C_` and 20 hex digits, a file found by a glob `F_` and 20 hex digits. The digits come from a hash of the thing's place (chapter 4), so they stay the same from build to build.

27.5 Storing strings once: the string pool

Tables are full of repeated strings - `INSTALLDIR`, a component key, `Hello` - and an MSI keeps each distinct string only once. All strings of the package live in one list, the **string pool** (two streams, `_StringPool` and `_StringData`), and a string cell in a table holds just the string's *number* in that list, in 2 bytes (or 3 in very large packages). The tutorial's first package has 111 distinct strings for all 16 tables:

```
C:\work\hello> rubrapack inspect hello.msi
code page: 65001
strings: 111
tables: 16
```

The pool also counts how many cells use each string. Part IV shows its bytes.

27.6 Tables about tables

A database describes itself. Two tables list the others: `_Tables` has one row per table, and `_Columns` one row per column of every table, with its type. That is where `inspect` finds the column names and types it prints on the first two lines. The *sequence* tables (`InstallExecuteSequence`, `InstallUISequence` and three more for administrative and advertised installations) list the installation's steps in order, with a condition each (tutorial chapters 9 and 13) - even the procedure is a table.

27.7 The IDT text format

The text `inspect` prints is Windows Installer's own **IDT** format: the three header lines, then one line per row, columns separated by tabs. The tools in Microsoft's Windows SDK import and export tables in this format, so a table from `rubrapack inspect` can be compared with one exported by another tool.

27.8 Where this is used

- Tutorial chapters 2 and 4: IDs, components, key paths.
- Tutorial chapter 18: `inspect` and `lint`.
- Part IV: the MSI database (the string pool and table encoding), the tables that install.

28 Keys, certificates and signatures

In this chapter

28.1	Two keys instead of one	115
28.2	Signing and checking	115
28.3	Certificates: whose key is this?	116
	Names: CN, O, C	116
28.4	Chains and trust	116
	Revocation	117
28.5	Timestamps	117
28.6	Where keys live: files and tokens	117
	The thumbprint	117
28.7	Where this is used	117

Tutorial chapter 16 signs `hello.msi`. This chapter explains what that means: how a signature proves who made a file and that nobody changed it, what a certificate is, and why a timestamp keeps a signature valid after the certificate has expired.

28.1 Two keys instead of one

The oldest kind of secret writing uses one key for both directions: whoever can lock can also unlock. **Public-key cryptography** (1970s) uses a **key pair** instead: two numbers that belong together, made at the same time.

- The **private key** stays secret with its owner - in a file protected by a password, or better inside a hardware token that never lets it out.
- The **public key** is given to everybody.

What one key does, only the other can check. For signatures that means: only the private key can *make* a signature, and anybody with the public key can *check* it. Knowing the public key does not help to find the private one - for today's sizes, not in the lifetime of the universe.

Two methods are in use for signing code:

Method	Typical key size	Notes
RSA	2048 or 3072 bits	the classic; rubrapack's test certificate uses RSA 2048
ECDSA	256 or 384 bits (P-256, P-384)	elliptic curves: much smaller keys for the same strength

28.2 Signing and checking

A signature is not made over the whole file - that would be slow for a 200 MB package - but over its **hash** (chapter 4):

```

signing (the publisher):
  file ---hash---> digest ---private key---> signature      (stored in the file)

checking (any user's computer):
  file ---hash---> digest'
  signature ---public key---> digest
  digest' = digest ?  yes: unchanged and signed by that key.  no: changed, or another key.
```

Because the hash changes completely when one bit of the file changes, no edit survives the check; and because only the private key makes signatures that the public key accepts, nobody else can sign a changed file in the publisher's name.

28.3 Certificates: whose key is this?

A signature checks out against a public key - but whose key? Anyone can make a key pair and call it "Example Software". A **certificate** answers that: a small document that states "this public key belongs to *this* name", signed by a **certificate authority** (CA) that checked the name.

Here is a test certificate made for rubrapack's own tests, as `openssl x509 -text` shows it:

Field	Value
Subject (who)	CN=rubrapack test signer, O=rubrapack tests
Issuer (who vouches)	CN=rubrapack test root
Valid from / until	30 Sep 2026 / 27 Sep 2036
Serial number	71AB698C19EACB47D6A00D71A711CF7A0ACDB236
Public key	RSA, 2048 bits
Basic constraints	CA:FALSE - it may not vouch for others
Key usage	Digital Signature
Extended key usage	Code Signing
Signature	sha256WithRSAEncryption, made with the issuer's private key

rubrapack checks these before signing: the certificate must be for code signing, must not be a CA's, and must be valid today (tutorial chapter 16).

Names: CN, O, C

A subject is a list of parts, each a type and a value: CN (common name), O (organization), OU (unit), L (city), S (state), C (country, two letters). The certificate stores them in an order, and different programs write them in different orders. This one is stored CN first, then O; Windows writes the parts from the last to the first:

```
as stored:      CN=rubrapack test signer, O=rubrapack tests
as Windows writes: O=rubrapack tests, CN=rubrapack test signer
```

An MSIX's publisher must be the Windows form, exactly (tutorial chapter 17) - which is why rubrapack prints it for you to copy when they differ.

28.4 Chains and trust

The test certificate was issued by "rubrapack test root", whose own certificate is signed by itself - it is a **root**. Real certificates form a **chain**:

```
your code signing certificate   issued by -> an intermediate CA certificate
the intermediate certificate    issued by -> a root CA certificate
the root certificate           issued by -> itself
```

Windows keeps a list of root certificates it trusts, the **trusted root store**, maintained by Microsoft. A signature is trusted when its chain ends at one of them. The intermediate certificates travel inside the signature (`--cert` adds them), so Windows can build the chain without asking anyone.

`rubrapack verify --trust ca.pem` checks against a root you name; `--system-roots` against the computer's own list. A self-made certificate (tutorial chapter 16) has a chain that ends nowhere Windows trusts - which is why it is fine for tests and useless for users.

Revocation

If a private key is stolen, its CA **revokes** the certificate: it publishes the serial number in a list (CRL) or answers queries about it (OCSP). Windows checks this when it can reach the network. rubrapack verify reports revocation: not-checked: it does not go online by itself.

28.5 Timestamps

A certificate expires - code signing certificates today after at most 39 months. What about all the files signed with it? Without help, their signatures stop counting on that day.

A **timestamp** fixes this. At signing, rubrapack sends the signature's hash to a **timestamp authority** (TSA), a server run by a CA. The TSA signs "this hash existed at this time" with its own certificate and sends that back; it goes into the file next to the signature (RFC 3161). Later, Windows checks: was the signing certificate valid *at the time the timestamp shows*? If yes, the signature stays valid for good, long after the certificate expired.

```
signed 2026-10-01, certificate valid until 2029-10-01
checked 2031-05-12:
  without a timestamp: certificate expired -> signature not valid
  with a timestamp:    valid on 2026-10-01 (the TSA says so) -> signature valid
```

The TSA never sees the file, only its hash.

28.6 Where keys live: files and tokens

Form	What it is
.pfx / .p12 (PKCS #12)	one file with the private key, the certificate and often the chain, protected by a password. What Windows exports.
.pem	text: base64 of the binary form between lines -----BEGIN CERTIFICATE----- / -----END ...----- (or PRIVATE KEY). What Linux tools use.
.der, .cer	the same, in binary
Windows certificate store	keys kept by Windows (certmgr.msc), used by thumbprint (--key-store)
hardware token, cloud HSM	the private key never leaves the device; programs ask the device to sign (PKCS #11, --pkcs11)

Since 2023 a CA issues code signing certificates only for keys in hardware, so for publishing you will use the last two rows.

A certificate in binary (DER) is itself a nest of length-prefixed fields. The test certificate begins 30 82 03 d8: 30 means "a sequence follows", 82 "its length takes the next 2 bytes", and 03 d8 is that length, 984 - big-endian this time (chapter 2). 984 bytes plus these four make the whole 988-byte certificate.

The thumbprint

A certificate's **thumbprint** is the SHA-1 hash (chapter 4) of its binary form - here 2E AB FE 60 19 0A F6 6C 52 E4 29 31 AF A6 01 BE C0 EF 86 0D. It is not a secret; it is a short name for one certificate, which is how Windows' store and rubrapack keys list name them.

28.7 Where this is used

- Tutorial chapter 16: sign, verify, keys, timestamps.
- Tutorial chapter 17: the MSIX publisher.
- Part IV: Authenticode (what exactly is hashed and signed, and the structure Windows accepts), checking against Windows.

Part IV

File formats

29 Package formats for implementers

In this chapter

29.1	How to read the facts	120
29.2	Conventions	120
29.3	The pipeline, in one picture	120

These pages describe what a program has to write to produce a Windows Installer package (.msi) and an MSIX package or bundle that Windows accepts, and how to sign them. They are written from public specifications and from experiments against Windows itself, so that you can build your own tool from them without reading rubrapack's code - or check rubrapack's code against them.

Page	Covers
Compound File Binary (CFB)	Compound File Binary: the container every .msi is
The MSI database inside the compound file	Streams, string pool, system tables, table encoding, IDT export
Summary information	The summary information stream and its code page trap
The smallest package that installs, repairs, upgrades and uninstalls	The tables that install, upgrade, repair and uninstall: files, registry, shortcuts, services, fonts, dialogs, per-user packages, sequences
Program files: machine type and version	Program files: machine type and version resource
Cabinets, MSZIP and deflate	Cabinet files, MSZIP blocks, the deflate encoder
Deterministic identities and reproducible packages	Deterministic GUIDs and keys, reproducible builds
Authenticode signatures	Authenticode signatures for PE files and MSI packages: the digests, the CMS structure Windows accepts, ECDSA, RFC 3161 timestamps
Checking your output against Windows	How to check your output with Windows' own components
MSIX packages	MSIX: the ZIP layout, block map, manifest, virtual registry and file system, extensions, bundles, signatures

Page	Covers
Registry hive files (REGF)	Registry hive files (REGF), as MSIX Registry.dat and User.dat hold them

29.1 How to read the facts

Every statement is tagged with where it comes from:

- **[spec]** - a public specification: [MS-CFB], [MS-OLEPS], [MS-CAB], [MS-MCI], RFC 1951, RFC 1321, RFC 3161, RFC 5652, RFC 9562, the PKWARE ZIP APPNOTE, ECMA-376 Part 2 (OPC), the Authenticode PE specification, or the Windows Installer and MSIX references on Microsoft Learn.
- **[observed]** - measured on Windows 11: the MSI pages on 25H2 (build 26200) with Windows Installer 5.0.10011, by creating databases through `msi.dll` and reading them back byte for byte, and by installing packages; the MSIX, registry hive and signature pages on build 26100, against Windows' own packaging API, Offline Registry Library and signer, and by installing and running packages. Observations are facts about those builds; where Microsoft does not document a format (the MSI table encoding, the registry hive, the MSIX signature's record of hashes are not published), observation is the only source.

No text or code here comes from other MSI, MSIX, cabinet or registry implementations.

29.2 Conventions

- All integers are little-endian unless stated otherwise.
- `u16`, `u32`, `u64` are unsigned; `i16`, `i32` signed two's complement.
- Offsets are in bytes from the start of the structure named.
- Hex byte strings are written in file order: `D0 CF 11 E0`.

29.3 The pipeline, in one picture

```
source description --> tables (rows of strings/integers/streams)
                    --> MSI database encoding (string pool + one stream per table)
                    --> files packed into a cabinet (stored or MSZIP) --> one more stream
                    --> summary information stream
                    --> all streams written into one compound file = the .msi
```

An MSIX package is simpler: the payload files (each deflated in independent 64 KiB parts), `AppxManifest.xml`, `AppxBlockMap.xml` (a hash per part) and `[Content_Types].xml` in one ZIP64 archive, with `Registry.dat/User.dat` hives for registry values; a bundle stores packages side by side; a signature adds `AppxSignature.p7x` (MSIX packages, Registry hive files (REGF)).

Each arrow is a page above. None of it needs Windows to produce: rubrapack builds the same bytes on Linux and on Windows.

30 Compound File Binary (CFB)

In this chapter

30.1	Sectors	121
30.2	Header (first 512 bytes) [spec]	121
30.3	FAT, DIFAT, mini FAT [spec]	122
30.4	Directory [spec]	122
30.5	A simple, deterministic writer	123
30.6	Reading safely	123
30.7	Worked example: the tutorial's hello.msi	123

An `.msi` file is a Compound File Binary file ("structured storage"): a small file system of named streams inside one file. The format is specified in [MS-CFB]; this page is the part an MSI writer needs, plus what Windows' own writer does.

30.1 Sectors

- The file is a sequence of fixed-size sectors. **Version 3** uses 512-byte sectors (sector shift 9), **version 4** uses 4096-byte sectors (shift 12). [spec]
- The header is at offset 0. Sector number N starts at file offset $(N + 1) * \text{sector_size}$ in both versions; in version 4 the header therefore occupies a whole 4096-byte sector whose bytes after the header are zero. [spec]
- `msi.dll` writes **version 4** for every new database, whatever its size (20 KB to 10 MB were tried), with minor version `0x003E`. Either version is accepted when installing; rubrapack writes version 4 by default and can write version 3. [observed]

Special sector numbers in chains and tables: [spec]

Value	Meaning
FFFFFFFFA	largest regular sector number
FFFFFFFC	DIFAT sector (in the FAT)
FFFFFFFD	FAT sector (in the FAT)
FFFFFFFE	end of chain
FFFFFFF	free sector / no stream

30.2 Header (first 512 bytes) [spec]

Offset	Size	Field	Value written
0	8	signature	D0 CF 11 E0 A1 B1 1A E1
8	16	CLSID	zero
24	2	minor version	003E
26	2	major version	3 or 4
28	2	byte order	FFFE (bytes FE FF)
30	2	sector shift	9 or 12
32	2	mini sector shift	6 (64-byte mini sectors)
34	6	reserved	zero

Offset	Size	Field	Value written
40	4	number of directory sectors	0 in version 3; the count in version 4
44	4	number of FAT sectors	
48	4	first directory sector	
52	4	transaction signature	0
56	4	mini stream cutoff	4096
60	4	first mini FAT sector	FFFFFFFF if none
64	4	number of mini FAT sectors	
68	4	first DIFAT sector	FFFFFFFF if none
72	4	number of DIFAT sectors	
76	436	first 109 FAT sector numbers (DIFAT)	unused entries FFFFFFFF

30.3 FAT, DIFAT, mini FAT [spec]

- The **FAT** is an array of u32, one entry per sector, giving the next sector of the chain the sector belongs to. FAT sectors themselves are marked FFFFFFFD.
- The FAT's own sector numbers are listed in the **DIFAT**: the first 109 in the header, the rest in DIFAT sectors, each holding $\text{sector_size} / 4 - 1$ numbers and, in its last u32, the next DIFAT sector (FFFFFFFF at the end). DIFAT sectors are marked FFFFFFFC in the FAT.
- With 4096-byte sectors, 109 FAT sectors already cover about 457 MB, so real MSI files in version 4 have no DIFAT sectors. With 512-byte sectors DIFAT starts at about 7 MB.
- Streams smaller than the **cutoff (4096 bytes)** live in the **mini stream**, in 64-byte mini sectors chained through the **mini FAT** (same layout as the FAT). The mini stream is itself stored as the root directory entry's own regular stream.

The FAT must cover its own sectors and the DIFAT's; a writer computes the counts by iterating until they stop changing.

30.4 Directory [spec]

The directory is a chain of sectors holding 128-byte entries. Entry 0 is the root.

Offset	Size	Field
0	64	name, UTF-16LE, NUL-terminated (at most 31 characters + NUL)
64	2	name length in bytes, including the NUL
66	1	type: 0 unused, 1 storage, 2 stream, 5 root
67	1	colour: 0 red, 1 black
68	4	left sibling (FFFFFFFF = none)
72	4	right sibling
76	4	child (storages and root only)
80	16	CLSID
96	4	state bits
100	8	creation time
108	8	modification time
116	4	starting sector (regular or mini, by size)
120	8	stream size (in version 3 only the low 32 bits count)

- The root entry is named `Root Entry`; its starting sector and size describe the mini stream.

- The root entry of an MSI carries the CLSID {000C1084-0000-0000-C000-000000000046}, stored as bytes 84 10 0C 00 00 00 00 00 C0 00 00 00 00 00 00 46. [spec] [observed]
- Unused entries are all zero except the three link fields, which are FFFFFFFF. [spec]
- The children of a storage form a **red-black tree** ordered first by name length, then by comparing the upper-cased UTF-16 names. [MS-CFB] allows the simplest valid tree: every node black, the tree a plain balanced binary search tree. rubrapack builds that - a balanced tree over the sorted names, all nodes black - and `msi.dll` opens it. [spec] [observed]
- Times may be zero. Writing zero times (and no other clock input) is what makes a CFB writer deterministic. [spec]

30.5 A simple, deterministic writer

rubrapack lays the file out in this order, all runs contiguous:

```
header | FAT sectors | DIFAT sectors | directory | mini FAT | mini stream | large streams in order
```

1. Sort stream sizes into "mini" (< 4096) and "regular".
2. Mini streams are packed back to back in 64-byte units; the mini stream length is the sum.
3. Count directory, mini FAT and mini stream sectors, then FAT and DIFAT sectors (iterate).
4. Fill FAT chains for each contiguous run, write the directory tree, mini FAT, FAT, DIFAT, header.

30.6 Reading safely

A reader must refuse, rather than follow: chains that loop (a chain never reaching FFFFFFFE within the number of sectors in the file), sector numbers beyond the file, a chain shorter than the stream size needs, directory links out of range, sibling trees that visit a node twice, and counts beyond sane limits. None of this costs much, and a crafted `.msi` otherwise hangs or crashes the reader.

30.7 Worked example: the tutorial's hello.msi

The first package of the tutorial (chapter 2), built with `--reproducible`, is 28672 bytes: the header sector and 6 sectors of 4096 bytes. Its header:

Offset	Bytes	Field	Value
0x00	d0 cf 11 e0 a1 b1 1a e1	signature	Compound File
0x18	3e 00	minor version	0x003E
0x1A	04 00	major version	4
0x1C	fe ff	byte order	0xFFFFE
0x1E	0c 00	sector shift	12: $2^{12} = 4096$
0x20	06 00	mini sector shift	6: 64
0x28	01 00 00 00	directory sectors	1
0x2C	01 00 00 00	FAT sectors	1
0x30	01 00 00 00	first directory sector	1
0x38	00 10 00 00	mini stream cutoff	4096
0x3C	02 00 00 00	first mini FAT sector	2
0x40	01 00 00 00	mini FAT sectors	1
0x44	fe ff ff ff	first DIFAT sector	FFFFFFFE
0x4C	00 00 00 00	DIFAT[0]: first FAT sector	0

The sectors, and the FAT entry of each (the next sector of its chain):

Sector	At offset	FAT entry	Holds
0	0x1000	FFFFFFFD	FAT
1	0x2000	FFFFFFFE	directory
2	0x3000	FFFFFFFE	mini FAT
3	0x4000	FFFFFFFE	mini stream
4	0x5000	5	cab1.cab
5	0x6000	FFFFFFFE	cab1.cab

The root directory entry, the first 128 bytes of sector 1:

Offset	Bytes	Field	Value
0	52 00 6f 00 6f 00 74 00 20 00 45 00 6e 00 74 00 72 00 79 00 ...	name	Root Entry (UTF-16LE)
64	16 00	name length	22
66	05	type	5 (root)
67	01	colour	1 (black)
76	02 00 00 00	child	entry 2
80	84 10 0c 00 00 00 00 00 00 c0 00 00 00 00 00 00 46	CLSID	{000C1084-0000-0000-C000-000000000046}
116	03 00 00 00	start	sector 3
120	00 0f 00 00 00 00 00 00	size	3840

The root's size is the mini stream's: 3840 bytes, holding 19 of the 20 streams in 64-byte mini sectors; 1 stream is large enough for regular sectors. The MSI database inside the compound file continues with the streams' names and contents.

31 The MSI database inside the compound file

In this chapter

31.1	Streams	125
	Stream names	125
31.2	The string pool: <code>_StringPool</code> and <code>_StringData</code>	126
	Code page	126
31.3	System tables	126
	Column type bits (<code>_Columns.Type</code>)	126
31.4	Table streams	127
31.5	IDT archive files (what <code>MsiDatabaseExport</code> writes)	127
31.6	Reading safely	128
31.7	Worked example: the tutorial's <code>hello.msi</code>	128

Microsoft documents the *meaning* of the Windows Installer tables (Microsoft Learn, "Database Tables"), but not how a database is laid out in the compound file. Everything on this page about the layout was established by creating databases through `msi.dll`, reading the streams back byte for byte, and checking that databases written from these rules open, export and install. [observed] unless marked otherwise.

31.1 Streams

Every table is one stream directly under the root storage. There are no sub-storages in an ordinary package.

Stream names

A table named `T` is stored under the name `U+4840` followed by `T` **packed** two characters per UTF-16 unit:

1. Map each character of this 64-character alphabet to 0..63: 0-9 -> 0..9, A-Z -> 10..35, a-z -> 36..61, . -> 62, _ -> 63.
2. Two alphabet characters in a row become one unit $0 \times 3800 + \text{first} + (\text{second} \ll 6)$.
3. A single alphabet character (the last one, or one followed by a character outside the alphabet) becomes $0 \times 4800 + \text{value}$.
4. Any other character is written as itself.

The packed name (with the `U+4840` prefix for tables) must fit the 31-unit CFB name limit, which allows table names of up to 60 alphabet characters. Unpacking reverses the steps.

Example: `_Columns` -> `U+4840 U+3B3F U+43F2 U+4438 U+45B1`.

Other streams:

Stream	Name
a Binary/Icon cell (OBJECT column)	<Table>.<key> packed the same way without the <code>U+4840</code> prefix, e.g. <code>Binary . Logo</code> (compound keys joined with .)
an embedded cabinet	the name after # in <code>Media.Cabinet</code> (e.g. <code>cab1.cab</code>), packed, no prefix
summary information	<code>\x05SummaryInformation</code> - not packed (see Summary information)

31.2 The string pool: `_StringPool` and `_StringData`

All strings of all tables live once in a shared pool; table cells hold string ids.

`_StringPool`:

```
u32 header      low 31 bits: the database code page; bit 31: string references are 3 bytes
then per string id 1, 2, 3, ... (id 0 is "no string" and is not stored):
u16 length      byte length in _StringData
u16 refs        bits 0-14: reference count; bit 15: set exactly when the string has a byte >=
0x80
```

- **Long strings** (64 KiB of bytes or more): the entry is `length = 0`, `refs` as above (non-zero), followed by an extra `u32` with the full byte length. The pair still describes one id. Seen: a 70002-byte string used once -> 0000 8001 00011172.
- An entry with `length = 0` and `refs = 0` is a free id.
- **Reference count** = the number of cells that hold the string, plus, per table, 1 for its `_Tables` row and, per column, 1 for the table name in `_Columns` and 1 for the column name. (Checked by recounting every database `msi.dll` produced.)
- `_StringData` is all the strings' bytes concatenated in id order, in the database code page. Lengths are **bytes**, not characters.
- **An empty string is never stored**: writing "" into a cell stores null.
- With more than 65535 distinct strings the header's bit 31 is set and every string reference in every table becomes 3 bytes (low 16 bits, then the high byte).

Code page

The database code page is the pool header's low bits: 0 when never set, otherwise what was set through the `_ForceCodepage` import. **65001 (UTF-8) works for every table** on current Windows: Korean, Japanese and supplementary-plane (e.g. U+20000, U+1F600) file and folder names, registry keys and values, shortcut names and the product name all install with exactly the right UTF-16 text, and dialogs display them - also under a system locale whose ANSI code page cannot hold Korean (en-US, 1252), so no separate Unicode setup program is needed. A UTF-16 code page (1200) is not possible here: the pool holds single-byte-unit strings. [observed] Windows Installer does not normalize names: a folder named with decomposed Hangul (U+1100 U+1161) is installed with exactly those code units, next to - not instead of - one named U+AC00. [observed]

31.3 System tables

Table	Columns	Key
<code>_Tables</code>	Name (string)	Name
<code>_Columns</code>	Table (string), Number (i2, 1-based), Name (string), Type (i2)	Table, Number

`_Tables` does not list itself or `_Columns`. A table with no rows has no stream at all, only its `_Tables` and `_Columns` rows. `_Validation`, when present, is an ordinary table.

Column type bits (`_Columns.Type`)

Bits	Meaning
0x00FF	width: maximum string length (0 = unlimited), or 2 / 4 for integers
0x0100	valid
0x0200	localizable
0x0400	non-binary (set for text, clear for OBJECT)
0x0800	string (set for text and OBJECT)

Bits	Meaning
0x1000	nullable
0x2000	part of the primary key

Values `msi.dll` uses for common SQL column types:

SQL	Type
CHAR(72) NOT NULL (key)	0x2D48
CHAR(72) (nullable)	0x1D48
CHAR(255) NOT NULL LOCALIZABLE	0x0FFF
LONGCHAR NOT NULL LOCALIZABLE	0x0F00
SHORT NOT NULL	0x0502
SHORT (nullable)	0x1502
SHORT NOT NULL (key)	0x2502
LONG NOT NULL	0x0104
OBJECT NOT NULL	0x0900

Integer columns do not have the non-binary bit.

31.4 Table streams

- A table stream is **column-major**: every row's cell of column 1, then every row's cell of column 2, and so on. The row count is the stream size divided by the row width.
- Cell widths: string reference 2 bytes (3 with long references), 16-bit integer 2 bytes, 32-bit integer 4 bytes, OBJECT 2 bytes.
- Integers are stored with the sign bit flipped: $\text{stored} = \text{value} \wedge 0x8000$ (16-bit) or $\text{value} \wedge 0x80000000$ (32-bit). **Stored 0 means null**. Consequently -32768 and -2147483648 cannot be stored.
- An OBJECT cell stores 1 when a stream exists, 0 for null.
- **Rows are sorted by the primary-key cells' stored numbers** - string ids, flipped integers - key columns in order. Windows Installer relies on this order.

A writer that assigns string ids in byte order of the strings gets rows sorted by string value for free, which is also the order an IDT export uses.

31.5 IDT archive files (what `MsiDatabaseExport` writes)

Useful as a reference output: a reader that exports byte-identical IDT files decodes the database the way Windows does.

```

line 1: column names separated by TAB
line 2: column types: s/S (string), l/L (localizable string), i/I (integer), v/V (binary),
        upper case = nullable, followed by the width (0 = unlimited; 2 or 4 for integers)
line 3: [<code page> TAB] <table name> TAB <key column> [TAB <key column> ...]
rows   : cells separated by TAB, lines end with CR LF

```

- The code page prefix on line 3 appears **only when the table holds a non-ASCII string**.
- Rows are sorted by primary-key **values** (strings by bytes, integers signed) - not by stored order.
- Null prints as an empty field. Binary cells print `<key>.ibd`; the bytes go to `<Table>/<key>.ibd`.
- TAB, CR and LF inside values are written as `0x10`, `0x11` and `0x19`. [spec]
- `_ForceCodepage.idt` is two empty lines then `<code page> TAB _ForceCodepage`.

- `_SummaryInformation.idt` has header lines `PropertyId TAB Value, i2 TAB 1255, _SummaryInformation TAB PropertyId` and one row per property. Dates are printed `yyyy/mm/dd hh:mm:ss` in the **local time of the machine doing the export**.

31.6 Reading safely

Check that the pool lengths add up to exactly the size of `_StringData`, that every string reference in a table points to a used id, that table stream sizes are a multiple of the row width, and that `_Columns` numbers each table's columns 1..n without gaps.

31.7 Worked example: the tutorial's hello.msi

The table `File` of the tutorial's first package (see Compound File Binary (CFB)) is the stream named `U+4840 U+430F U+422F`:

```
0x4840 (table)
F i: 0x3800 + 15 + (44 << 6) = 0x430F
l e: 0x3800 + 47 + (40 << 6) = 0x422F
```

`_StringPool` starts with `e9 fd 00 00`: the code page 65001 (UTF-8), bit 31 clear (2-byte string references). Then two u16 per string id; the first ids:

Id	Bytes	Length	Refs	String
1	09 00 01 00	9	1	#cab1.cab
2	01 00 01 00	1	1	.
3	01 00 01 00	1	1	1
4	05 00 03 00	5	3	1.0.0
5	07 00 01 00	7	1	1.2.3.4
6	04 00 02 00	4	2	1033

111 strings in all; their bytes, back to back, are the 1381 bytes of `_StringData`.

`_Columns` describes the table - its rows for `File`, the type decoded with the bit table above:

#	Column	Type	Meaning
1	File	0x2D48	key, string, non-binary, valid, width 72
2	Component_	0x0D48	string, non-binary, valid, width 72
3	FileName	0x0FFF	string, non-binary, localizable, valid, width 255
4	FileSize	0x0104	valid, width 4
5	Version	0x1D48	nullable, string, non-binary, valid, width 72
6	Language	0x1D14	nullable, string, non-binary, valid, width 20
7	Attributes	0x1502	nullable, non-binary, valid, width 2
8	Sequence	0x0104	valid, width 4

The `File` stream itself is 20 bytes: one row, stored column by column:

```
32 00 0e 00 6c 00 00 46 00 80 05 00 06 00 00 82 01 00 00 80
```

Column	Stored
File	32 00 -> id 50 Hello
Component_	0e 00 -> id 14 C_185f8db32271fe25f561
FileName	6c 00 -> id 108 hello.exe

Column	Stored
FileSize	00 46 00 80 -> $0x80004600 \wedge 0x80000000 = 17920$
Version	05 00 -> id 5 1.2.3.4
Language	06 00 -> id 6 1033
Attributes	00 82 -> $0x8200 \wedge 0x8000 = 512$
Sequence	01 00 00 80 -> $0x80000001 \wedge 0x80000000 = 1$

String cells are string ids; integers are stored with the top bit flipped, so 17920 ($0x00004600$) is stored $0x80004600$ and 0 stays free to mean null.

32 Summary information

In this chapter

32.1	Layout [spec] [observed]	130
32.2	Properties an installer package sets	130
32.3	The code page trap - keep summary strings ASCII [observed]	131
32.4	Worked example: the tutorial's hello.msi	131

Every package has a stream `\x05SummaryInformation` (the first character is U+0005; the name is not packed). It is an OLE property set as specified in [MS-OLEPS]; Microsoft Learn ("Summary Information Stream Property Set") says which properties an installer package uses.

32.1 Layout [spec] [observed]

```

u16  byte order      FFFE
u16  version         0 (msi.dll writes 0)
u32  OS field        msi.dll writes 0x00020206
16   CLSID           zero
u32  section count   1
16   FMTID           F29F85E0-4FF9-1068-AB91-08002B27B3D9, stored E0 85 9F F2 F9 4F 68 10 AB 91
08 00 2B 27 B3 D9
u32  section offset  48

section:
u32  section size
u32  property count
count x (u32 property id, u32 offset from the section start)
values, each padded to a multiple of 4 bytes:
VT_I2 (2)           u32 type, i16 value, 2 bytes padding
VT_I4 (3)           u32 type, i32 value
VT_LPSTR (30)       u32 type, u32 byte count including the NUL, bytes, NUL, padding
VT_FILETIME (64)    u32 type, u64 100-ns intervals since 1601-01-01 UTC

```

32.2 Properties an installer package sets

Id	Name	Type	Value
2	Title	LPSTR	e.g. Installation Database
3	Subject	LPSTR	product name
4	Author	LPSTR	manufacturer
5	Keywords	LPSTR	e.g. Installer
7	Template	LPSTR	<platform>;<language id>: x64;1033, Arm64;1042, Intel;1033
9	Revision Number	LPSTR	the package code : a GUID, new for every different package file
12, 13	Create / Last Save time	FILETIME	optional
14	Page Count	I4	minimum installer version: 200 for x64 and x86, 500 for Arm64
15	Word Count	I4	bit 1 = files are compressed (in cabinets); 2 for a normal package

Id	Name	Type	Value
18	Creating Application	LPSTR	any

32.3 The code page trap - keep summary strings ASCII [observed]

Property 1 (CodePage, VT_I2) says how the LPSTR bytes are encoded. On Windows 11 25H2:

- **65001:** msi.dll writes the strings correctly as UTF-8, but reads the code page back as the signed value -535, fails to convert every string (the installation log says "Failed to retrieve # 9 summary info property. GetLastError returned 87, the code page is -535") and refuses the package with error 1620.
- **1200 (UTF-16):** refused the same way (log: "... the code page is 1200"), even with complete UTF-16 strings and byte counts that follow [MS-OLEPS].
- **No code page property and ASCII strings:** installs.

So: leave property 1 out and write every summary string in ASCII. Localized product names belong in the Property table (ProductName), which uses the database code page - and 65001 works there (see The MSI database inside the compound file).

32.4 Worked example: the tutorial's hello.msi

The tutorial's first package: the stream is 332 bytes. Its header:

Offset	Bytes	Field	Value
0x00	fe ff	byte order	0xFFFFE
0x04	06 02 02 00	OS	0x00020206
0x18	01 00 00 00	section count	1
0x1C	e0 85 9f f2 ...	FMTID	F29F85E0-...
0x2C	30 00 00 00	section offset	48
0x30	1c 01 00 00	section size	284
0x34	09 00 00 00	property count	9

Then 9 pairs of (property id, offset), and the values:

Property	Offset	Type	Raw bytes	Value
2	0x80	VT_LPSTR	1e 00 00 00 16 00 00 00 49 6e 73 74 61 6c 6c 61 ...	Installation Database
3	0xA0	VT_LPSTR	1e 00 00 00 06 00 00 00 48 65 6c 6c 6f 00	Hello
4	0xB0	VT_LPSTR	1e 00 00 00 11 00 00 00 45 78 61 6d 70 6c 65 20 ...	Example Software
5	0xCC	VT_LPSTR	1e 00 00 00 0a 00 00 00 49 6e 73 74 61 6c 6c 65 ...	Installer
7	0xE0	VT_LPSTR	1e 00 00 00 09 00 00 00 78 36 34 3b 31 30 33 33 ...	x64;1033
9	0xF4	VT_LPSTR	1e 00 00 00 27 00 00 00 7b 45 32 32 43 44 37 45 ...	{E22CD7E9-3AF1-82AB-811D-BCF61D7A0757}
14	0x124	VT_I4	03 00 00 00 c8 00 00 00	200
15	0x12C	VT_I4	03 00 00 00 02 00 00 00	2
18	0x134	VT_LPSTR	1e 00 00 00 10 00 00 00 72 75 62 72 61 70 61 63 ...	rubrapack 0.7.0

A string value is its type (30), its byte count including the NUL, the bytes, the NUL, and padding to a multiple of 4. Property 9, the package code, is derived from the content under `--reproducible`; without it every build gets a new one.

33 The smallest package that installs, repairs, upgrades and uninstalls

In this chapter

33.1	Tables and columns	133
33.2	Property	134
33.3	Directory	134
33.4	Components, features, files	134
33.5	Empty folders	135
33.6	Media and the cabinet	135
33.7	Major upgrade and downgrade refusal	135
33.8	Running an installed program to register and unregister	135
33.9	Registry values	136
33.10	Shortcuts	136
33.11	Removing files and duplicating them	136
33.12	Environment variables	137
33.13	INI files	137
33.14	Searches and launch conditions	137
33.15	Services, fonts, permissions	138
33.16	Cabinets, administrative images, advertisement	138
33.17	Per-user and dual packages	138
33.18	Files in use	138
33.19	Sequences	138
33.20	Dialogs	139
	Choices during installation	140
	Several languages in one package	141
33.21	What this recipe does not cover yet	141

Table meanings are documented on Microsoft Learn ("Database Tables", "Standard Actions", "Suggested InstallExecuteSequence"). This page is a tested recipe: a package built exactly like this installs per machine, survives repair, upgrades an older version, refuses a downgrade and uninstalls cleanly on Windows 11. [observed]

33.1 Tables and columns

Types use the notation of The MSI database inside the compound file (s72 = CHAR(72), upper case = nullable, l = localizable, i2/i4 integers). Keys are in bold.

Table	Columns
Property	Property s72, Value l0
Directory	Directory s72, Directory_Parent S72, DefaultDir l255
Component	Component s72, ComponentId S38, Directory_ s72, Attributes i2, Condition S255, KeyPath S72
Feature	Feature s38, Feature_Parent S38, Title L64, Description L255, Display I2, Level i2, Directory_ S72, Attributes i2
FeatureComponent	Feature_ s38, Component_ s72

Table	Columns
File	File s72, Component_ s72, FileName l255, FileSize i4, Version S72, Language S20, Attributes I2, Sequence i4
MsiFileHash	File_ s72, Options i2, HashPart1..HashPart4 i4
CreateFolder	Directory_ s72, Component_ s72
Media	DiskId i2, LastSequence i4, DiskPrompt L64, Cabinet S255, VolumeLabel S32, Source S72
Upgrade	UpgradeCode s38, VersionMin S20, VersionMax S20, Language S255, Attributes i4, Remove S255, ActionProperty s72
CustomAction	Action s72, Type i2, Source S72, Target S255
InstallExecuteSequence	Action s72, Condition S255, Sequence I2
InstallUISequence	

`File.Sequence` and `Media.LastSequence` as 32-bit integers let a package hold more than 32767 files.

33.2 Property

`ProductCode`, `ProductName`, `ProductVersion` (a.b.c, a and b up to 255, c up to 65535; a fourth part is allowed but ignored when versions are compared), `Manufacturer`, `ProductLanguage` (1033, 1042, ...), `UpgradeCode`, `ALLUSERS=1` (per machine), `REBOOT=ReallySuppress`, `MSIRESTARTMANAGERCONTROL=Disable` (see "Files in use" below), and `SecureCustomProperties` listing the two upgrade properties below.

33.3 Directory

- `TARGETDIR` with no parent and `DefaultDir = SourceDir`.
- Each standard folder used (`ProgramFiles64Folder`, `ProgramFilesFolder`, ...) as a child of `TARGETDIR` with `DefaultDir = ..` Windows resolves these names itself. For an x64 or Arm64 package use the ...64Folder names, for x86 the plain ones.
- Your folders below them with `DefaultDir = SHORT|Long name`. The long name may be any Unicode text (with code page 65001). The short name must be a valid 8.3 ASCII name, unique among all files and folders of the same parent; when the long name already is a valid 8.3 name it can be written alone.

33.4 Components, features, files

- **One component per file**, the file as its key path. `ComponentId` is a GUID that must stay the same for the same resource at the same place for the life of the product (see Deterministic identities and reproducible packages). `Attributes` 256 (64-bit) for x64 and Arm64 packages, 0 for x86.
- A feature with `Level 1`; `Display 0` hides it. Map every component in `FeatureComponents`. A child feature names its parent in `Feature_Parent`. A feature whose `Level` is above `INSTALLLEVEL` (1 unless the package sets it) is not installed by default; `MsiConfigureFeature` can add it later, and its files then arrive. [observed]
- `File.FileName` is `SHORT|Long` like `DefaultDir`. `Attributes 512` marks the file vital. Sequence numbers 1..n are the order of the files inside the cabinet.
- Files without a version resource need an **MsiFileHash** row: the MD5 of the file, read as four little-endian 32-bit integers (stored as signed values). Installed files hash with `MsiGetFileHash` to exactly these values. Files with a version resource put the version in `File.Version` instead (and need no hash).

33.5 Empty folders

A folder that must exist without files gets its own Directory row, a component whose Directory_ is that folder and whose KeyPath is null (the folder itself is the key path), a CreateFolder row pointing at both, and a FeatureComponents row. CreateFolders makes it at install time; RemoveFolders removes it at uninstall if it is empty. [observed] With component attribute 16 (permanent) the folder stays after uninstall while everything else is removed. [observed]

33.6 Media and the cabinet

One row: DiskId 1, LastSequence n, Cabinet #cab1.cab - the # means "a stream inside this package named cab1.cab". The cabinet holds the files named by their File keys, in Sequence order (Cabinets, MSZIP and deflate). Summary Word Count 2 says the files are compressed into cabinets.

33.7 Major upgrade and downgrade refusal

Two Upgrade rows with the package's UpgradeCode:

VersionMin	VersionMax	Attributes	ActionProperty	Meaning
this version	-	0x102 (only detect, min inclusive)	NEWER_FOUND	same or newer version installed
-	this version	0x001 (migrate features)	OLDER_FOUND	older version installed: remove it

The Upgrade table matches by UpgradeCode, version and language only - not by architecture. If an x64 and an x86 build share one UpgradeCode, installing one removes the other as "older". Give each architecture its own UpgradeCode when both may be installed side by side (verified: two architectures with separate codes install and uninstall independently). [observed]

An error custom action (type 19, Target = the message; it is a formatted string, so [ProductName] works) conditioned on NEWER_FOUND stops the installation with error 1603 when a same-or-newer version is present. Every version needs a new ProductCode (and every package file a new package code).

33.8 Running an installed program to register and unregister

Some products must register themselves (an input method, a shell extension) through their own program rather than through table rows. The pattern that keeps install, repair, upgrade and removal atomic uses the component action state of that program's component C (\$C = what this installation does to it, ?C = its state before; 2 absent, 3 local) [spec], with type 18 actions (Source = the program's File key, Target = its arguments), all deferred (0x400) and not impersonated (0x800):

Action	Type	Where	Condition	Runs
...UndoRollback	18+0x100+0x400+0x800+0x400	before RemoveFiles	\$C=2 AND ?C=3	register (rollback of the next row)
...Undo	18+0x400+0x800	after it, before RemoveFiles	\$C=2 AND ?C=3	unregister
...DoRollback	18+0x100+0x400+0x800+0x400	after InstallFiles	\$C>2 AND ?C<>3	unregister
...RedoRollback	same	after it	\$C>2 AND ?C=3	register
...Do	18+0x400+0x800	after both	\$C>2	register

A rollback action must be sequenced *before* the action it undoes: rollback runs the script backwards, and only rollback actions already in the script are run. Rollback actions ignore their exit code (0x40); forward actions do not, so a failing register or unregister fails the installation and rolls it back. Observed on Windows 11: repair runs ...Do again; a failing unregister rolls the removal back and

the program (still present) registers again; when an upgrade fails, the new package's ...DoRollback runs, then the old version's files return and the old package's own ...UndoRollback registers the old program again. [observed]

Nothing that writes to the script may stand between InstallInitialize and RemoveExistingProducts: with any deferred or rollback custom action there, every upgrade stops with error 2613 ("RemoveExistingProducts action sequenced incorrectly"). [observed]

33.9 Registry values

Registry (**Registry** s72, Root i2, Key l255, Name L255, Value L0, Component_ s72) is written by WriteRegistryValues (5000) and undone at uninstall; RemoveRegistry (same columns without Value) by RemoveRegistryValues (2600). Root 0 = HKCR, 1 = HKCU, 2 = HKLM. The value's first characters choose its type: #x binary (hex), #% expandable string, # followed by digits a DWORD (up to #4294967295), [~] anywhere a multi-string ([~]a[~]b[~]); a plain string that starts with # is written as ##. A Name of - in RemoveRegistry deletes the whole key. A failed installation restores overwritten values and removed keys. [observed]

A component whose key path is a registry value has attribute 4 and KeyPath = the Registry row. The component's 64-bit attribute (256) chooses the registry view: without it, a 64-bit package writes to the 32-bit view (WOW6432Node). A 32-bit component in a 64-bit package is legal; a 64-bit component in a 32-bit package is not.

REG_QWORD cannot be expressed in the Registry table. rubrapack writes it with a DLL custom action from the Binary table (type 1): an immediate action reads the component action states and the current values and passes two lists as CustomActionData - one to a deferred action that writes or deletes, one to its rollback twin that restores what was there. The DLL's bitness must match the package (the engine loads it into a custom action server of the package's architecture). [observed: x64 and x86]

File types and URL schemes are plain Registry rows under Root 0 (HKCR) in the program's component, not the Extension/Verb/ProgId tables (those bring advertisement and repair on first use): .ext (default) = the ProgId; ProgId (default) = its description, ProgId\DefaultIcon = [#File], 0, ProgId\shell\open\command = "[#File]" "%1"; for a scheme, scheme (default) = URL:<description>, URL Protocol = empty (a Null Value with a Name writes an empty string), and the same DefaultIcon and shell\open\command. Root 0 follows the installation: HKLM\Software\Classes per machine, HKCU\Software\Classes per user. Opening a file of a type only this program claims then starts it directly. [observed]

33.10 Shortcuts

Shortcut (**Shortcut** s72, Directory_ s72, Name l128 SHORT|Long without .lnk, Component_ s72, Target [#FileKey] for a non-advertised shortcut, Arguments (formatted), Description (plain text), Hotkey, Icon_, IconIndex, ShowCmd, WkDir = a Directory key) is written by CreateShortcuts (4500) and removed by RemoveShortcuts (3200). Put the shortcut in its target file's component. A folder made only for shortcuts is not removed by itself: add a RemoveFile row (FileName null, DirProperty = the folder, InstallMode 2) for it and each parent below the standard folder. With ALLUSERS=1, ProgramMenuFolder and DesktopFolder are the all-users Start menu and the Public Desktop. A failed installation leaves no shortcut or folder; repair recreates a deleted one. [observed]

33.11 Removing files and duplicating them

RemoveFile (**FileKey** s72, Component_ s72, FileName L255 with */? or null for the folder, DirProperty s72, InstallMode i2: 1 install, 2 uninstall, 3 both) runs in RemoveFiles when its component is installed (1) or removed (2). A component with no key path file and a Directory_ works as the

carrier.DuplicateFile (**FileKey** s72, Component_ s72 = the source's component, File_ s72, DestName L255 SHORT|Long, DestFolder S72) runs in DuplicateFiles (4210) and RemoveDuplicateFiles. Files removed at install come back when the installation fails; a folder that existed before the installation is left in place at uninstall, even when empty. [observed]

33.12 Environment variables

Environment (**Environment** s72, Name l255, Value L255 formatted, Component_ s72), written by WriteEnvironmentStrings (5200) and RemoveEnvironmentStrings. Name prefixes: * system variable (without it a per-user one), = create or set, - undo when the component is removed. Value [~];x appends ;x, x;[~] prepends. With -, uninstall deletes a set variable and takes only the appended or prepended part out of a variable that existed before; without - nothing is undone. A failed installation restores the previous values. [observed]

33.13 INI files

IniFile (**IniFile** s72, FileName l255 SHORT|Long, DirProperty S72, Section l96, Key l128, Value l255 formatted, Action i2: 0 add line, 1 create line, 3 add tag, Component_ s72) is applied by WriteIniValues (5100) and undone when the component is removed; RemoveIniFile (same columns, Value nullable, Action 2 remove line, 4 remove tag) by RemoveIniValues, which runs before InstallFiles. An add tag appends ,value to a comma list and uninstall removes just that item. Non-ASCII file, section, key and value names work. A failed installation restores the file. [observed]

33.14 Searches and launch conditions

AppSearch (**Property, Signature_**) at 50 in both sequences sets the property from a locator with the same signature: RegLocator (Root, Key, Name, Type 2 = raw value, +16 = 64-bit view), DrLocator (Path may start with a folder property, like [System64Folder]) with a Signature row for a file (FileName, MinVersion), or CompLocator (ComponentId, Type 1 = key file). The property must be public and listed in SecureCustomProperties to reach the server side.

A search can give a directory its default: a directory's property set before CostFinalize is where the directory resolves. rubrapack searches into a property of its own (RpFound_<ID>) and copies it with a type-51 action (Source = the directory, Target = [RpFound_<ID>]) at 51 in both sequences under the condition RpFound_<ID> AND NOT <DIR>, so a directory given on the command line is left alone. The registry locator then has Type 0 (a folder, which must exist; the value comes back with a trailing backslash) instead of 2: **AppSearch looks a folder-type RegLocator up in the Signature table, which must exist, even empty** - without it the installation stops with 2228. [observed: a /qn major upgrade installs into the folder the earlier version recorded; DIR= on the command line wins]

rubrapack's install folder guard (guard = true) is an immediate DLL custom action (type 1, the helper DLL in Binary) at 1010 in InstallExecuteSequence, after CostFinalize has resolved the directories and before InstallValidate, under NOT Installed. It reads the Directory keys from a property, gets each path with MsiGetTargetPath, and refuses a path in which an existing part is a reparse point or not a folder, or whose folder exists with an owner (GetNamedSecurityInfo) other than S-1-5-18, S-1-5-32-544 or TrustedInstaller; it then shows the message with MsiProcessMessage(INSTALLMESSAGE_ERROR) and returns ERROR_INSTALL_FAILURE, so the installation ends with 1603 before any file is written. [observed]

LaunchCondition (**Condition**, Description formatted) at 100 in both sequences stops the installation with the description when a condition is false. [observed]

33.15 Services, fonts, permissions

ServiceInstall (Name, DisplayName and Description formatted, ServiceType 0x10, StartType 2/3/4, ErrorControl 1, StartName null or NT AUTHORITY\LocalService, Arguments formatted, Component_ = the exe's component) with InstallServices (5800); ServiceControl Event flags 0x1 start on install, 0x2 stop on install, 0x20 stop on removal, 0x80 delete on removal, Wait 1, with StopServices (1900), DeleteServices (2000), StartServices (5900). Font (File_, FontTitle null = read from the font) with RegisterFonts/UnregisterFonts; the file must be in FontsFolder itself. MsiLockPermissionsEx (**MsiLockPermissionsEx**, LockObject, Table = CreateFolder, File, Registry or ServiceInstall, **SDDLText** (the column name), Condition) needs Windows Installer 5.0 (summary page count 500); a folder gets it through a CreateFolder row. A failed installation leaves no service or font. [observed]

33.16 Cabinets, administrative images, advertisement

Several cabinets: one Media row each (DiskId 1..n, LastSequence = the last file's sequence in it); embedded ones are streams named in Cabinet with #, external ones are files next to the package named in Cabinet without # (long names work). AdminExecuteSequence (CostInitialize 800, FileCost 900, CostFinalize 1000, InstallValidate 1400, InstallInitialize 1500, InstallAdminPackage 3900, InstallFiles 4000, InstallFinalize 6600) and AdminUISequence make msiexec /a write an uncompressed image that installs like the original; AdvtExecuteSequence (CostInitialize, CostFinalize, InstallValidate, InstallInitialize, PublishFeatures 6300, PublishProduct 6400, InstallFinalize) makes msiexec /jm advertise the product. [observed]

33.17 Per-user and dual packages

The single-package form: ALLUSERS=2 with MSIINSTALLPERUSER=1 installs per user - the engine points ProgramFilesFolder/ProgramFiles64Folder at %LOCALAPPDATA%\Programs and the menu and desktop folders at the user's - and ALLUSERS=1 MSIINSTALLPERUSER="" installs per machine. Summary Word Count bit 8 says no elevation is needed. Registry Root -1 (HKMU) is HKCU per user and HKLM per machine; an Environment name without * is a user variable. Once a per-user product is installed the engine deletes MSIINSTALLPERUSER, so a launch condition that tests it must read Installed OR ... or the product can no longer be removed; in general, write every launch condition as Installed OR (...). [observed]

33.18 Files in use

When a file to be replaced or removed is held by a running program (a DLL loaded into it):

- With Restart Manager, the default, Windows Installer tries to **shut down every program holding such a file, even in a silent installation** (/qn), and restarts it afterwards; if one does not close, the whole installation fails (1601). [observed]
- With MSIRESTARTMANAGERCONTROL=Disable in the Property table, the engine moves the held file aside (C:\Config.Msi*.rbf), puts the new file in place at once and returns 0; the program keeps running on the old copy, programs started afterwards load the new file, and the old copy is deleted at the next restart. [observed]
- A property given on the command line reaches only the package being installed. During an upgrade, the removal of the old version follows **the old package's** Property table - author the property from the first version on. [observed]

33.19 Sequences

InstallExecuteSequence (conditions in brackets):

```
FindRelatedProducts 25, <refuse-downgrade action> 30 [NEWER_FOUND], CostInitialize 800,
FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, InstallValidate 1400,
```

```
InstallInitialize 1500, RemoveExistingProducts 1501, ProcessComponents 1600,
UnpublishFeatures 1800, RemoveFiles 3500, RemoveFolders 3600, CreateFolders 3700,
InstallFiles 4000, RegisterUser 6000, RegisterProduct 6100, PublishFeatures 6300,
PublishProduct 6400, InstallFinalize 6600
```

InstallUISequence: FindRelatedProducts 25, the refuse-downgrade action 30 [NEWER_FOUND], CostInitialize 800, FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, ExecuteAction 1300 (and the dialogs, see Dialogs).

RemoveExistingProducts right after InstallInitialize puts the removal of the old version inside the new installation's transaction, so a failed upgrade rolls back to the old version. [observed: a deferred custom action that fails right after InstallFiles makes the upgrade end with 1603, and the old version is registered again with its files byte for byte; a failed first installation leaves no files and no registration.]

33.20 Dialogs

A package with dialogs carries these tables (types as above):

Table	Columns
Dialog	Dialog s72, HCentering i2, VCentering i2, Width i2, Height i2, Attributes I4, Title L128, Control_First s50, Control_Default S50, Control_Cancel S50
Control	Dialog_ s72, Control s50, Type s20, X i2, Y i2, Width i2, Height i2, Attributes I4, Property S72, Text L0, Control_Next S50, Help L50
ControlEvents	Dialog_ s72, Control_ s50, Event s50, Argument s255, Condition S255, Ordering I2
ControlCondition	Dialog_ s72, Control_ s50, Action s50, Condition s255
EventMapping	Dialog_ s72, Control_ s50, Event s50, Attribute s50
TextStyle	TextStyle s72, FaceName s32, Size i2, Color I4, StyleBits I2
UIText	Key s72, Text L255
Binary	Name s72, Data v0 (a stream)
RadioButton	Property s72, Order i2, Value s64, X i2, Y i2, Width i2, Height i2, Text L0, Help L50
ComboBox	Property s72, Order i2, Value s64, Text L64

What the engine checks when it shows a dialog (each seen as an error dialog carrying the number):

- **Tab order.** Control_Next must form one cycle through every control that has one, starting at Control_First. A chain that leaves a control out, or ends at a control whose Control_Next is empty, stops the installation with 2810 or 2809. Static text takes no part in the cycle. [observed] rubrapack checks every dialog it builds and refuses to write a broken one (RP0010).
- **The error dialog.** The dialog named by the ErrorDialog property needs a Text control named ErrorText and an Icon control named ErrorIcon (2835 otherwise); its Attributes carry the error-dialog bit (0x10000), and push buttons named A, C, I, N, O, R, Y with EndDialog arguments ErrorAbort .. ErrorYes are shown as the message needs. [observed]
- **Brackets in texts.** Text is a formatted string: [Next] is read as a property and shows as nothing. [observed] Quote button names in running text instead.
- **Files in use.** With Restart Manager off, the engine shows the dialog named FilesInUse (buttons ending with Retry, Ignore, Exit) with a ListBox bound to FileInUseProcess. The **ListBox table must exist, even empty**: without it the engine logs 2205 and skips the dialog, and the installation goes on as if Ignore was chosen. In a major upgrade the dialog comes twice (the new files, then the old version's removal). After Ignore the held file is moved aside and the installation ends with 0. [observed]

- **Error icon.** The engine puts its own icon for the message type into `ErrorIcon` (a warning triangle for a launch condition), whatever the control's `Binary` holds. [observed]
- **List order.** A `ComboBox` lists its items alphabetically unless the control has the `Sorted` attribute (0x10000); with it they follow the `Order` column. [observed] rubrapack sets it.
- **Elevation.** A per-machine installation started interactively by a non-elevated user asks for consent (UAC) when the execute sequence starts, on the secure desktop. [observed]
- **Code page.** With the database code page 65001, Korean titles, texts and RTF license text show correctly under a Korean and under an English (1252) system locale. observed, see [msi-database.md]

rubrapack's sets: every dialog is 370 x 270 dialog units with a banner strip (a `Bitmap` control 0,0,370,44 whose picture is `Binary.RpBanner`), the title in bold (`{\RpTitle}`, a `TextStyle`) and a line of description in it, a `Line` at 44 and at 234, and `Back / Next / Cancel` at y 243. The `InstallUISequence` adds the welcome (1230, `NOT Installed`) or the maintenance dialog (1240, `Installed AND NOT RESUME AND NOT Preselected`), the progress dialog (1280, modeless, subscribed to `ActionText` and `SetProgress` through `EventMapping`) and the three exit dialogs at -1 (success), -2 (cancelled) and -3 (fatal). The install folder dialog puts the folder's `Directory` key in a `PathEdit`, and its `Next` runs `SetTargetPath` before `NewDialog`, which checks the path. The license text is a `ScrollableText` control holding RTF: plain text becomes `\uN?` escapes (characters above U+FFFF as a surrogate pair), one `\par` per line.

Author pages are ordinary `Dialog` rows in the same frame; `Control_Next` runs through their controls in position order and on to `Back`, `Next` and `Cancel`. A radio group is a `RadioButtonGroup` control whose buttons are `RadioButton` rows (positions relative to the group); a drop-down list is a `ComboBox` control with `ComboList` (0x20000) and `Sorted`, filled from `ComboBox` rows. Their properties are added to `SecureCustomProperties`, so values chosen in the dialogs or given on the command line reach the execute sequence. [observed]

Choices during installation

What rubrapack writes for the choices a user makes [observed, Windows 11 26100]:

- **Features.** `Feature.Attributes` 0x10 (`UIDisallowAbsent`) keeps "will be unavailable" out of the tree's menu; 0x02 (`FollowParent`) installs a feature where its parent is. A feature that is off unless a condition holds is a `Condition` row (**Feature**_s38, **Level** i2, `Condition` S255) with `Level` 0 and the condition `NOT Installed AND NOT (<when>)`. **Without NOT Installed the row turns the feature off again at removal, when the property is gone, and its files are left behind.**
- **Components.** when on a file, a registry value, an environment variable or an INI value is the component's `Condition`, evaluated when the component is first installed (no `Transitive` attribute: a repair keeps what is there). A shortcut lives in its target file's component; one with `when` gets a component of its own in the shortcut's folder, whose key path is a registry value under root -1 (`HKMU`: `HKLM` or `HKCU` as installed), with the attribute 0x4.
- **Icons.** An `Icon` row (**Name** s72 ending in `.ico`, `Data` v0) holds the `.ico`; `ARPPRODUCTICON` names it for installed apps, which the engine registers as the product's `ProductIcon` under `HKLM\SOFTWARE\Classes\Installer\Products` (not as `DisplayIcon`), and `Shortcut.Icon_ / IconIndex` 0 give a shortcut its own.
- **Keep.** A file left at removal is a component with 0x10 (`Permanent`).
- **Launch.** A type-34 custom action with 0xC0 (asynchronous, no wait): `Source` = the program's folder, `Target` = "[#FileKey]" args. The finished dialog's `Finish` runs it through `DoAction` before `EndDialog` under `RPLAUNCH` = "1" `AND NOT Installed`; `Installed` still has its value from the start of the setup, so repair and removal do not start it. It runs in the setup's client process, with the rights of the user who started the setup.
- **Change.** The maintenance dialog's `Change` is a `NewDialog` to the dialog with the `SelectionTree`; that dialog's `Back` goes to the maintenance dialog under `Installed` and to the install folder dialog under `NOT Installed`; the ready dialog's `EndDialog Return` then applies the new feature states.

- **Scope.** In a dual package (ALLUSERS=2, MSIINSTALLPERUSER=1 by default) the folder properties follow the per-user default when the dialogs start. The scope page's Next sets [ALLUSERS] and [MSIINSTALLPERUSER] ({} clears it), then puts the install folder's path under [%ProgramW6432] (or [%ProgramFiles(x86)] for an x86 package) or [LocalAppDataFolder] Programs into the directory's property and runs SetTargetPath; the execute sequence resolves the other folders itself with the final scope.

Several languages in one package

The dialog tables hold one language at a time, but every text they show can come from a property, so one set of dialogs can speak several languages. What the engine does [observed, Windows 11 26100]:

- A Text, PushButton, CheckBox, RadioButton or ComboBox text of [RpT_X], and a Dialog Title of [P], show the property's value; each dialog reads it when it is created, so a value changed by one dialog shows on the next.
- A text style prefix at the start of the value ({} \RpTitle_ko} . . .) applies.
- **The value is not formatted again:** [ProductName] inside a property's value shows literally. The text must be formatted when it is put into the property - by a [RpT_X] ControlEvent, whose Argument is formatted when the event runs, or by a type-51 custom action, whose Target is.
- Setting [DefaultUIFont] by an event changes the face of every control without its own style on the next dialog.
- A dialog's Control_First must be visible when the dialog opens (2836 otherwise); a control hidden by ControlCondition may stay in the tab cycle.

rubrapack's layout with [ui] languages: English is language 0. For every text X and language L, RpT_X_L in the Property table holds the text with [ProductName], [Manufacturer] and [ProductVersion] put in at build time (and the title style for headings); RpT_X holds the English one. The InstallUISequence starts with type-51 actions: RPLANGUAGE = L when unset and UserLanguageID (17), then SystemLanguageID (18), is one of L's LANGIDs, else en (19); then at 21 one action per text copies [RpT_X_L] into RpT_X under RPLANGUAGE = "L", and one sets DefaultUIFont. A text that uses other properties is copied as its formatted source instead. The language page (RpLanguageDlg, 1225, a RadioButtonGroup on RPLANGUAGE) replaces the welcome and maintenance rows; its Next runs the same copies as ControlEvents, then NewDialog to the welcome or the maintenance page. A license per language is a ScrollableText per language in the same place, each with Show/Hide ControlConditions on RPLANGUAGE. The UIText table has one language: English.

33.21 What this recipe does not cover yet

Custom actions other than the error type, the register pair and the REG_QWORD helper above; and _Validation (needed by validation tools, not by the installer). These pages grow as rubrapack implements them.

34 Program files: machine type and version

In this chapter

34.1	Finding the headers [spec]	142
34.2	The version resource [spec]	142
34.3	What goes into the package [observed]	142
34.4	Reading safely	143

A package writer reads two facts from every Portable Executable (.exe, .dll, ...) it packs: the machine type, to refuse a file built for the wrong architecture, and the version resource, because Windows Installer treats versioned and unversioned files differently. Layout from the Microsoft PE/COFF specification and the VS_VERSIONINFO documentation. [spec]

34.1 Finding the headers [spec]

```
offset 0      "MZ"
offset 0x3C   u32  e_lfanew: offset of the PE signature
e_lfanew      "PE\0\0"
+4            COFF header: u16 machine, u16 number of sections, ..., u16 optional header size
(at +16)
+24           optional header: u16 magic 0x10B (PE32) or 0x20B (PE32+); the data directories
              start at +96 (PE32) or +112 (PE32+), preceded by their count
then          section table: 40 bytes per section (virtual size +8, virtual address +12,
              raw size +16, raw offset +20)
```

Machine values: 0x8664 x64, 0xAA64 Arm64, 0x014C x86. A file that starts with MZ but has no valid PE\0\0 header is an ordinary data file.

34.2 The version resource [spec]

1. Data directory 2 is the resource table (an RVA and size). Map RVAs to file offsets through the section table.
2. Resource directories are three levels deep: type -> name -> language. Each directory has a 16-byte header (named and ID entry counts at +12 and +14) followed by 8-byte entries (u32 name or ID, u32 offset; the high bit of the offset marks a sub-directory).
3. Type 16 (RT_VERSION), its first name, its first language: the leaf is a data entry (u32 RVA, u32 size).
4. The data is VS_VERSIONINFO: u16 length, u16 value length, u16 type, the UTF-16 key VS_VERSION_INFO with its NUL, padding to 4 bytes, then VS_FIXEDFILEINFO whose first u32 is 0xFEEF04BD. dwFileVersionMS (+8) and dwFileVersionLS (+12) give the version HIWORD(MS).LOWORD(MS).HIWORD(LS).LOWORD(LS).

34.3 What goes into the package [observed]

- Versioned file: File.Version = the four-part version, File.Language = the language MsiGetFileVersion reports, which is the first language of VarFileInfo\Translation (for example 1033, 1042); no MsiFileHash row. Installed files read back with the same version.
- Unversioned file: no version, a MsiFileHash row (The smallest package that installs, repairs, upgrades and uninstalls).

- Machine type: an x64 package takes x64 files, an Arm64 package Arm64 files, an x86 package x86 files. A deliberate exception (a 32-bit helper in a 64-bit package) should be declared, not silently accepted; the component's own bitness does not change.

34.4 Reading safely

Check every offset and size against the file length before using it, map RVAs only through a section that holds the whole range, and stop at the three expected directory levels - crafted files with loops or out-of-range pointers are common.

35 Cabinets, MSZIP and deflate

In this chapter

35.1	Cabinet layout [spec: MS-CAB]	144
	Block checksum [spec]	144
35.2	MSZIP [spec: MS-MCI]	144
35.3	A deflate encoder in brief [spec: RFC 1951]	145
35.4	Worked example: the tutorial's hello.msi	145

35.1 Cabinet layout [spec: MS-CAB]

```
CFHEADER (36 bytes when no reserve fields):
  "MSCF", u32 0, u32 cabinet size, u32 0, u32 offset of the first CFFILE, u32 0,
  u8 minor version 3, u8 major version 1, u16 folder count, u16 file count, u16 flags (0),
  u16 set id, u16 cabinet index (0)
CFFOLDER (8 bytes each):
  u32 offset of the folder's first CFDATA, u16 number of CFDATA blocks, u16 compression
  (0 = none, 1 = MSZIP)
CFFILE (per file):
  u32 uncompressed size, u32 offset inside the folder's uncompressed data, u16 folder index,
  u16 date, u16 time, u16 attributes, NUL-terminated name
CFDATA (per block):
  u32 checksum, u16 compressed bytes, u16 uncompressed bytes (at most 32768), data
```

- In an MSI, file names in the cabinet are the File table keys, which are ASCII.
- DOS date (year - 1980) << 9 | month << 5 | day, time hour << 11 | minute << 5 | second / 2. A fixed date (rubrapack writes 1980-01-01 00:00) keeps output deterministic; attributes 0x20.
- One folder holds at most 65535 blocks (about 2 GiB); more data needs more folders or cabinets.

Block checksum [spec]

```
checksum(bytes, seed):
  xor every complete little-endian u32 of the bytes into seed;
  the 1-3 remaining bytes form one more u32 with the FIRST remaining byte in the HIGHEST
  position used (3 left: b0<<16 | b1<<8 | b2; 2 left: b0<<8 | b1; 1 left: b0), xor it in.
block checksum = checksum(the 4 bytes "compressed size, uncompressed size" as stored,
                           checksum(block data, 0))
```

A checksum of 0 means "not computed"; readers accept it.

35.2 MSZIP [spec: MS-MCI]

Each CFDATA block of a folder with compression type 1 holds:

```
"CK" (43 4B), then one complete raw deflate stream (RFC 1951) for this block's
uncompressed bytes (at most 32768), ending with a final block.
```

The decompressor keeps the previous blocks of the same folder as its dictionary, so an encoder *may* refer back into earlier blocks. An encoder that never does (each block compressed on its own) produces valid MSZIP too. rubrapack gives each block the previous block's 32 KiB of uncompressed data as a preset dictionary (about 9% smaller on mixed data than blocks on their own), and since that dictionary is the input, not the output, the blocks are still compressed on several threads at

once, with the same bytes whatever their number. It starts a new folder at a file boundary before 65535 blocks; the dictionary does not cross folders. Windows extracts all of it: a package with twelve folders installs, repairs and removes with every file byte-identical. [observed]

35.3 A deflate encoder in brief [spec: RFC 1951]

1. **Match finding (LZ77):** hash the next 3 bytes, walk a chain of earlier positions with the same hash within 32 KiB, keep the longest match (3..258 bytes). Longer chain walks give better compression; a "lazy" step that also tries the next position and emits a literal when that match is longer helps noticeably.
2. **Blocks:** group up to a few ten thousand symbols. For each block compute the size of three encodings and write the smallest:
 - fixed Huffman codes (literal/length 0-143: 8 bits, 144-255: 9, 256-279: 7, 280-287: 8; distances 5 bits);
 - dynamic Huffman codes: code lengths limited to 15 bits (7 for the code-length alphabet); package-merge gives optimal limited lengths; send HLIT, HDIST, HLEN, the code-length code in the order 16 17 18 0 8 7 9 6 10 5 11 4 12 3 13 2 14 1 15, then the run-length coded lengths (16 = repeat previous 3-6, 17 = zeros 3-10, 18 = zeros 11-138);
 - stored (byte-aligned LEN, NLEN, raw bytes; at most 65535 per stored block).
3. Huffman codes are written most significant bit first; everything else least significant bit first.
4. Always give at least two symbols a code length (pair a lone symbol with a dummy), so every code is complete - some decoders reject incomplete codes.

Check an encoder by inflating its output with an independent implementation (for example the zlib library with raw deflate, window bits -15) and an MSI built with it by installing it and comparing installed files' hashes (Checking your output against Windows).

35.4 Worked example: the tutorial's hello.msi

The cabinet of the tutorial's first package is the stream cab1.cab, 6776 bytes. CFHEADER:

Offset	Bytes	Field	Value
0x00	4d 53 43 46	signature	MSCF
0x08	78 1a 00 00	cabinet size	6776
0x10	2c 00 00 00	first CFFILE	44
0x18	03 01	version	1.3
0x1A	01 00	folders	1
0x1C	01 00	files	1
0x1E	00 00	flags	0
0x20	00 00	set id	0

CFFOLDER at 0x24: 42 00 00 00 01 00 01 00 - first CFDATA at 66, 1 block, compression 1 (MSZIP).

CFFILE entries - the name is the File table key:

Offset	size	offset in folder	folder	date	attr	Name
0x2C	17920	0	0	1980-01-01	0x20	Hello

The first CFDATA at 66: checksum 80 35 aa c0, 6702 compressed bytes for 17920 uncompressed. Its data begins 43 4b ed 5c 0d 74: CK, then deflate. The first deflate byte ed = 11101101; its lowest bit is BFINAL = 1, the next two BTYPE = 2 (dynamic codes).

36 Deterministic identities and reproducible packages

In this chapter

36.1	Name-based GUIDs (UUID version 8) [spec: RFC 9562]	146
36.2	Keys	146
36.3	Reproducible packages	147

Windows Installer identifies things by GUIDs and keys that must be **stable**: the same component must keep the same ComponentId in every version, or repair and upgrade misbehave. Generating them from the content, instead of storing them by hand, keeps them stable and makes builds reproducible. This page describes rubrapack's rules; other rules work too as long as they are stable.

36.1 Name-based GUIDs (UUID version 8) [spec: RFC 9562]

```
digest = SHA-256( u32le(len(tag)) || tag
                  || u32le(1)                                     (format version)
                  || for each field: u32le(len(field)) || field ) (fields in UTF-8)
uuid   = digest[0..16], then byte 6 = (byte 6 & 0x0F) | 0x80 (version 8)
                  byte 8 = (byte 8 & 0x3F) | 0x80 (variant 10)
text   = "{" + hex(bytes 0-3) "-" hex(4-5) "-" hex(6-7) "-" hex(8-9) "-" hex(10-15) + "}"
        upper case, bytes in order
```

Length prefixes keep ("ab", "c") and ("a", "bc") apart; the tag keeps different kinds of identity apart.

Identity	Tag	Fields
ComponentId	rubrapack. component	upgrade code, scope (machine), component architecture, logical target path (<dir id>/<file name>), key path kind (file), key path name (the File key)
ProductCode	rubrapack. product	upgrade code, architecture, scope, the version's first three parts
package code (reproducible builds)	rubrapack. package	hex SHA-256 of the package built with an all-zero package code

The ProductCode changes with the three-part version - which is what a major upgrade needs - and the ComponentIds do not.

36.2 Keys

Table keys (File, Component, Directory) are identifiers: [A-Za-z_][A-Za-z0-9_.*]*, at most 72 characters. rubrapack uses the author's IDs for files and folders as given and derives the rest:

```
key = prefix + "_" + first 20 lower-case hex digits of SHA-256(logical name)
    prefix C for components (logical name: the File key), D for intermediate folders
    (logical name: "<standard folder>/<part>/<part>..."), F for files from wildcards
```

36.3 Reproducible packages

A package is byte-for-byte reproducible when nothing depends on the clock, the machine or the order of hash tables: fixed CFB directory times (zero), fixed cabinet dates, string ids assigned in byte order of the strings, rows sorted by key, streams in a fixed order, no random GUIDs. The one identity that must differ between package files is the package code; derive it from the content (as above) when reproducibility matters, and use a random one otherwise.

37 Authenticode signatures

In this chapter

37.1	Where the signature lives in a PE file	148
37.2	The PE digest	148
37.3	The CMS SignedData	149
37.4	MSI packages	149
37.5	Timestamps	150
37.6	ECDSA	150
37.7	What Windows reports	150
37.8	Worked example: the tutorial's hello.msi	150

What a program has to write so that Windows accepts a signed PE file (.exe, .dll) or MSI package. Facts are tagged as in Package formats for implementers: **[spec]** for Microsoft's "Windows Authenticode Portable Executable Signature Format", the PE/COFF specification and RFC 5652 (CMS); **[observed]** for files signed by Windows' own signer (PowerShell Set-AuthenticodeSignature, which uses mssign32.dll) and checked with Get-AuthenticodeSignature / WinVerifyTrust on Windows 11.

37.1 Where the signature lives in a PE file

- The certificate table: data directory entry 4 (its entry is 8 bytes: file offset, size) of the optional header. The directory starts at optional header + 96 for PE32 (magic 0x10B) and + 112 for PE32+ (magic 0x20B); NumberOfRvaAndSizes must be at least 5. Unlike every other directory entry, its "address" is a **file offset**, not an RVA. [spec]
- At that offset, at the end of the file: WIN_CERTIFICATE { u32 dwLength, u16 wRevision = 0x0200, u16 wCertificateType = 0x0002 (PKCS_SIGNED_DATA), bCertificate[] }, where bCertificate is a DER CMS ContentInfo. dwLength includes the 8-byte header and is rounded up to 8; the extra bytes are zero. The directory entry's size equals dwLength. [spec] [observed]
- Before the table is appended, the file is padded with zeros to a multiple of 8. The padding is hashed (it is "data after the sections", below). [spec]
- The optional header's CheckSum (optional header + 64) is updated. Windows' signer adjusts the existing value rather than recomputing it: for a file whose checksum was not the PE algorithm's value, Windows keeps the difference, a fresh computation does not. CheckSum is outside the digest, so both verify. [observed]

37.2 The PE digest

In this order, with SHA-256: [spec]

1. The headers, from 0 to SizeOfHeaders, without CheckSum (4 bytes) and without the certificate table directory entry (8 bytes).
2. Every section with SizeOfRawData > 0, in increasing PointerToRawData order, SizeOfRawData bytes from PointerToRawData.
3. The data after them: from SizeOfHeaders + sum(SizeOfRawData) to the end of the file minus the certificate table.

Hashing the file linearly with the two holes gives the same result only when the sections follow each other without gaps; the steps above are the definition.

37.3 The CMS SignedData

Windows' signer writes, and accepts: [observed]

```
ContentInfo { signedData (1.2.840.113549.1.7.2), [0] SignedData {
  version 1,
  digestAlgorithms SET { sha256 + NULL },
  encapContentInfo { SPC_INDIRECT_DATA (1.3.6.1.4.1.311.2.1.4),
    [0] SpcIndirectDataContent }          -- the SEQUENCE itself, no OCTET
STRING
  certificates [0] { the signer's certificate, then intermediates; no self-signed root },
  signerInfos SET { SignerInfo {
    version 1, IssuerAndSerialNumber of the signer,
    digestAlgorithm sha256 + NULL,
    signedAttrs [0] { SpcSpOpusInfo, contentType, SpcStatementType, messageDigest },
    signatureAlgorithm rsaEncryption + NULL,
    signature (RSA PKCS#1 v1.5) } } } }
```

- `SpcIndirectDataContent ::= SEQUENCE { data, messageDigest DigestInfo }`. For a PE file data is `SEQUENCE { SPC_PE_IMAGE_DATA (1.3.6.1.4.1.311.2.1.15), SpcPeImageData }` and Windows writes `SpcPeImageData` as `30 09 03 01 00 A0 04 A2 02 80 00`: no flags, and a file link that is an empty Unicode string. `messageDigest` holds the PE digest. [observed]
- The **messageDigest attribute** is the hash of the `SpcIndirectDataContent` contents **without** its `SEQUENCE` tag and length - not of the whole element, and not of the PE file. [observed]
- The signed attributes are stored with tag `[0] (0xA0)` but the signature is computed over the same bytes with tag `SET (0x31)`, sorted as a DER SET OF. The four attributes Windows writes: `SpcSpOpusInfo (1.3.6.1.4.1.311.2.1.12) = SEQUENCE { [0] { [0] "" }, [1] { [0] "" } }`, `contentType = SPC_INDIRECT_DATA`, `SpcStatementType (1.3.6.1.4.1.311.2.1.11) = SEQUENCE { individual code signing (1.3.6.1.4.1.311.2.1.21) }`, `messageDigest`. No signing time. [observed]
- RSA PKCS#1 v1.5 is deterministic: a program that writes exactly this structure with the same key produces the same bytes as Windows. rubrapack does, except the CheckSum difference above. [observed]

37.4 MSI packages

Microsoft does not publish how an MSI package is hashed. The rules below were worked out by signing packages with Windows' signer and comparing: rubrapack's signature and Ex streams came out byte-identical to Windows' for the same package and key, and packages whose directory entries were given non-zero state bits, times and CLSIDs told the fields apart. [observed]

- The signature is the stream `\005DigitalSignature` at the root of the compound file: the same CMS `ContentInfo` as for a PE file, with `data = SEQUENCE { SPC_SIPINFO (1.3.6.1.4.1.311.2.1.30), SpcSipInfo }`, where Windows writes `SpcSipInfo` as `SEQUENCE { 2, OCTET STRING <the MSI SIP GUID {000C10F1-0000-0000-C000-000000000046} in its little-endian byte order>, 0, 0, 0, 0, 0 }`.
- Windows also writes `\005MsiDigitalSignatureEx`: 32 bytes, the SHA-256 of a "prehash" of the compound file's directory: The root's own times are not part of it (Windows sets the root's modification time when it saves the signed file).
 - the root entry's CLSID (16 bytes) and state bits (4 bytes, little-endian);
 - then, for every stream at the root except the two signature streams, in the order of their names compared as UTF-16LE bytes: the name (UTF-16LE, no terminating NUL), the stream size as 8 bytes little-endian, and the entry's creation and modification times as stored (8 + 8 bytes). A stream's CLSID and state bits are not part of it.

- The digest in `SpcIndirectDataContent` is the hash of: the 32-byte `MsiDigitalSignatureEx` value (when that stream is written), then every stream's contents in the same name order (again without the two signature streams), then the root CLSID.
- Only streams at the root were observed; a package with storages (embedded transforms, sub-databases) is not covered here, and rubrapack refuses to sign one.
- A signature covers what is inside the `.msi` only: cabinets outside it (Media table Cabinet values that do not start with #) are not part of the digest.

37.5 Timestamps

A signature without a timestamp stops being valid when the signing certificate expires. With an RFC 3161 timestamp, Windows checks the certificate at the stamped time instead. What `mssign32!SignerTimeStampEx2` with `SIGNER_TIMESTAMP_RFC3161` writes: [observed]

- The request's message imprint is the hash (here SHA-256) of the `SignerInfo`'s signature value: the `OCTET STRING`'s contents, without its tag and length.
- The server's `TimeStampToken` (a CMS SignedData over a `TSTInfo`, RFC 3161) goes into the `SignerInfo` as its unsigned attributes, after the signature: [1] { SEQUENCE { 1.3.6.1.4.1.311.3.3.1, SET { TimeStampToken } } }.
- Nothing else in the signature changes: the signed attributes, the signature and the outer certificates are the same bytes as before; the server's certificates travel inside the token. For an MSI package the `\005DigitalSignature` stream grows accordingly (from the mini stream to regular sectors, for a token of a few kilobytes); `\005MsiDigitalSignatureEx` and the digest do not change, since the signature streams are not part of them.
- rubrapack writes exactly this; with the same key and token its PE file is byte-identical to Windows', and so is the MSI signature stream.

`Set-AuthenticodeSignature -TimestampServer` writes an older form instead: a PKCS#9 `counterSignature` attribute (1.2.840.113549.1.9.6) and the server's certificates added to the outer certificates. Windows accepts both. [observed]

37.6 ECDSA

An ECDSA P-256 signer works for PE files, MSI packages and MSIX packages alike (Windows 11 26100 calls all three `Valid` with the root trusted). Windows' signer writes the `SignerInfo`'s `signatureAlgorithm` as `id-ecPublicKey` (1.2.840.10045.2.1) with NULL parameters - not `ecdsa-with-SHA256` - and the signature value as the DER SEQUENCE { INTEGER `r`, INTEGER `s` }. rubrapack writes the same, with deterministic nonces (RFC 6979), and reads either OID. [observed]

37.7 What Windows reports

`Get-AuthenticodeSignature` distinguishes the cases a signer needs (and names the time-stamping certificate in `TimeStamperCertificate` when there is a timestamp): `Valid` (digest, signature and chain to a trusted root), `UnknownError` with "the certificate chain ... not trusted" (digest and signature are right, the root is not trusted on this machine), `HashMismatch` (the file changed after signing), `NotSigned`. So a signature can be checked for correctness before any root is trusted. [observed]

37.8 Worked example: the tutorial's hello.msi

The digest of the tutorial's first package, computed here from the rules above without any key - a signature made with any key carries these same values. The streams in order of their UTF-16LE names:

#	name (UTF-16LE)	Name	Size
1	05 00 53 00 75 00 ...	\005SummaryInformation	332

#	name (UTF-16LE)	Name	Size
2	26 41 65 38 be 41 ...	cab1.cab	6776
3	40 48 0c 46 f6 45 ...	CustomAction	8
4	40 48 0d 43 35 42 ...	Directory	18
5	40 48 0f 42 e4 45 ...	FeatureComponents	4
6	40 48 0f 42 e4 45 ...	Feature	16
7	40 48 0f 43 2f 42	File	20
8	40 48 16 42 27 43 ...	Media	14
9	40 48 3f 3b f2 43 ...	_Columns	552
10	40 48 3f 3f 77 45 ...	_StringData	1381
11	40 48 3f 3f 77 45 ...	_StringPool	448
12	40 48 52 44 f6 45 ...	InstallExecuteSequence	120
13	40 48 52 44 f6 45 ...	InstallUISequence	42
14	40 48 59 45 f2 44 ...	Property	40
15	40 48 7f 3f 64 41 ...	_Tables	32
16	40 48 8c 44 f0 44 ...	Component	12
17	40 48 ca 41 30 43 ...	AdminExecuteSequence	48
18	40 48 ca 41 30 43 ...	AdminUISequence	24
19	40 48 ca 41 f9 45 ...	AdvExecuteSequence	42
20	40 48 de 44 6a 45 ...	Upgrade	32

The prehash is the root's CLSID and state bits (20 bytes) and, per stream, its name, 8-byte size and two zero times: 802 bytes in all. Its SHA-256, the `\005MsiDigitalSignatureEx` value:

```
99 02 fe fb ce eb 28 a8 61 dc 7d ce 9a a9 83 fe a1 b7 c0 e1 28 5e a6 80 ee e7 f2 fe 73 dd c6 0a
```

The digest in `SpcIndirectDataContent` - SHA-256 over that value, every stream's bytes in the same order, and the root CLSID:

```
88 20 af 0f be 36 3d e2 1c fc 9d 96 5c f9 40 0d 2c 25 23 69 2f c0 2e ba 1a 17 25 8c bf 03 d8 a2
```

38 Checking your output against Windows

In this chapter

38.1	The database opens and decodes	152
38.2	The package installs	152
38.3	Checks before writing	153
38.4	Microsoft's ICE rules [observed]	153
38.5	Things that look fine but are not	154

Microsoft does not publish the MSI table encoding, so the reference is Windows itself. These checks need only a Windows machine and the Windows Installer API (`msi.dll`); no SDK tools.

38.1 The database opens and decodes

1. `MsiOpenDatabase` your file read-only - the compound file and string pool are acceptable.
2. `MsiDatabaseExport` every table (and `_ForceCodepage`, `_SummaryInformation`) to IDT files.
3. Compare with what your own reader exports, or with the IDT of a reference database made through `msi.dll` (`MsiOpenDatabase` with create mode, `MsiDatabaseImport` of a `_ForceCodepage.idt` containing 65001, SQL CREATE TABLE/INSERT via views). Byte-identical IDT means your encoding matches.

38.2 The package installs

Drive the installer from a program rather than `msiexec`, so every step is synchronous and returns an error code:

Step	API	Expect
silent UI	<code>MsiSetInternalUI(INSTALLUILEVEL_NONE)</code>	
verbose log	<code>MsiEnableLog</code> (all modes)	read it when anything fails
install	<code>MsiInstallProduct(path, "")</code>	0
names	FindFirstFile on each installed path, compare the entry's own UTF-16 name exactly	equal
content	<code>MsiGetFileHash</code> of each installed file vs your <code>MsiFileHash</code> row	equal
registration	<code>MsiGetProductInfo(ProductCode, InstalledProductName)</code>	your ProductName
repair	delete one installed file, <code>MsiReinstallProduct(ProductCode, ...)</code>	file back, same name
upgrade	install version N+1 over N	old ProductCode gone, new one installed
downgrade	install version N again	refused (1603, the log names your refusing action)
features	<code>MsiQueryFeatureState</code> ; <code>MsiConfigureFeature(..., INSTALLSTATE_LOCAL)</code>	levels above <code>INSTALLLEVEL</code> absent until turned on
rollback	copy of version N+1 with a deferred custom action that fails after <code>InstallFiles</code> (type 34 + 1024, "[SystemFolder]cmd.exe" /c exit 1) and a new package code, installed over N	1603 (log: error 1722); N still registered, its files unchanged

Step	API	Expect
uninstall	MsiConfigureProduct(ProductCode, ..., INSTALLSTATE_ABSENT)	0; folders, keys and registration gone

Per-machine installs need an elevated process. Keep test products on their own UpgradeCodes and remove them in the same run.

38.3 Checks before writing

msi.dll accepts many broken databases and only fails at install time, if at all (it does not even enforce column widths). A writer should check its own tables first. rubrapack's build refuses to write a package that breaks any of these (diagnostics RP2001-RP2015):

- every cell has the column's kind; no null in a non-nullable column; strings no longer than the column width, counted in UTF-16 units (a non-BMP character counts two), and valid UTF-8; 16-bit integers within -32767..32767 and 32-bit ones not -2147483648 (those values mean null);
- no duplicate primary keys; references (Component.Directory_, File.Component_, FeatureComponents, CreateFolder, MsiFileHash.File_, Directory_Parent, Feature_Parent) point at existing rows; every sequence action is a standard action, a CustomAction or a Dialog;
- ProductCode, ProductName, ProductVersion, Manufacturer, ProductLanguage are present; GUIDs are upper case in braces, component GUIDs are unique, ProductCode differs from UpgradeCode;
- each File.Sequence is unique and covered by a Media.LastSequence;
- standard actions keep their order (costing before InstallValidate, file actions between InstallInitialize and InstallFinalize);
- the 64-bit component attribute (256) matches the summary Template platform (x64/Arm64 vs Intel);
- a file key path is a file of the same component;
- each Upgrade.ActionProperty is upper case (public) and listed in SecureCustomProperties - otherwise a per-machine install does not pass it to the server side.

38.4 Microsoft's ICE rules [observed]

The Windows SDK's "MSI Tools" include MsiVal2 and darice.cub, the Internal Consistency Evaluators (ICEs) that Microsoft's own authoring tools run: MsiVal2 package.msi darice.cub -f prints what fails. They run only on Windows, and they go further than the checks above - they know the meaning of the tables, not only their shape. What they report for rubrapack's test packages (every source of its test suite, 39 packages), and why:

- **ICE03, "Missing specifications in _Validation"** for every column: the _Validation table describes each column for validators only; Windows Installer does not read it, and rubrapack does not write it.
- **ICE82**, duplicate sequence numbers: the language actions of a package with several dialog languages share one number. They are independent set-property actions, so their order does not matter.
- **ICE34**, the language page's radio property has no Property row: on purpose, because the command line (RPLANGUAGE=ko) must win over the detection, which only runs NOT RPLANGUAGE.
- **ICE43 and ICE57**, a non-advertised shortcut in a component whose key path is a file: they assume shortcut folders are per user. In a per-machine package (ALLUSERS=1) the Start menu and desktop are the all-users folders, and a per-user or dual package installs once per user, so the "first user only" problem these ICEs guard against does not arise.
- **ICE52**, a private property in AppSearch (RpFound_<ID>, a remembered folder): private on purpose, so the command line cannot set it; AppSearch runs in both sequences, so it needs no passing to the server side.

Three findings were real and are fixed (0.4.3): the finished, cancelled and failed pages were missing from AdminUISequence, so an administrative installation (`msiexec /a`) with the full UI ended without them (ICE20); a component whose key path is its folder (an INI entry, a `[remove]`) was not listed in `CreateFolder` (ICE18); and a package with dialogs had no `ControlCondition` table, which ICE17 reads for every dialog (it stopped with error 2228 instead of checking).

38.5 Things that look fine but are not

- An `msi.dll`-made database is not a complete specification: `msi.dll` stores 65001 summary strings correctly and still cannot read them back (Summary information).
- `MsiDatabaseExport` prints summary dates in local time; compare dates in UTC.
- A successful `MsiOpenDatabase` says nothing about row order or reference counts - only installation does. Test with an install, not just an open.

39 MSIX packages

In this chapter

39.1	The ZIP archive	155
39.2	The block map (AppxBlockMap.xml)	155
39.3	[Content_Types].xml	156
39.4	The manifest (AppxManifest.xml)	156
39.5	The virtual registry (Registry.dat, User.dat)	156
39.6	The virtual file system (VFS\...)	157
39.7	Extensions	157
39.8	Bundles (.msixbundle)	158
39.9	Signatures (AppxSignature.p7x)	159
39.10	Installing an unsigned package	159
39.11	Worked example: the tutorial's hello.msix	160

What a program has to write so that Windows reads, installs and runs an MSIX package. Facts are tagged as in Package formats for implementers: **[spec]** for Microsoft Learn (package manifest and block map schemas) and ECMA-376 Part 2 (Open Packaging Conventions); **[observed]** for packages written by Windows' own packaging API (IAppxFactory/IAppxPackageWriter in AppxPackaging.dll, part of Windows) and for rubrapack's packages read back through IAppxPackageReader and installed with Add-AppxPackage on Windows 11. rubrapack writes what is described here.

39.1 The ZIP archive

- The payload files first, then AppxManifest.xml, AppxBlockMap.xml and [Content_Types].xml, in that order. [observed]
- Every entry is ZIP64, whatever its size: the local header has version needed 4.5, flag 0x0008 (data descriptor), CRC and sizes 0 and no extra field; the data is followed by a ZIP64 data descriptor (PK\7\8, CRC-32, compressed and plain size as 8 bytes each). The central directory entry has made-by and needed 4.5 (MS-DOS), sizes and offset 0xFFFFFFFF and a ZIP64 extra field (0x0001, 24 bytes: plain size, compressed size, local header offset). The archive ends with a ZIP64 end record (size 44), its locator, and an end record whose counts and offsets are all 0xFFFF / 0xFFFFFFFF. [observed]
- Methods: stored (0) and deflate (8). AppxManifest.xml, AppxBlockMap.xml and [Content_Types].xml are deflated even when the payload is stored. [observed]
- Entry names are OPC part names: / between folders and every byte outside A-Z a-z 0-9 - . _ ~ percent-encoded, UTF-8 bytes included - data\<U+C790> %%(1).txt (a Hangul syllable, a space, %, # and parentheses) is stored as data/%EC%9E%90%20%25%23%281%29.txt - and the UTF-8 name flag is not set. [Content_Types].xml keeps its brackets. [observed]
- The dates in the headers are not read; Windows writes the time of writing, rubrapack 1980-01-01 00:00 so that a package does not change from one build to the next. [observed]

39.2 The block map (AppxBlockMap.xml)

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<BlockMap xmlns="http://schemas.microsoft.com/appx/2010/blockmap"
           xmlns:b4="http://schemas.microsoft.com/appx/2021/blockmap" IgnorableNamespaces="b4"
```

```

        HashMethod="http://www.w3.org/2001/04/xmlenc#sha256">
    <File Name="data\text.txt" Size="218890" LfhSize="43">
        <Block Hash="(base64 SHA-256 of 65536 plain bytes)" Size="(compressed bytes of this
block)"/>
        ...
        <b4:FileHash Hash="(base64 SHA-256 of the whole file)"/>
    </File>
    ...
</BlockMap>

```

- One File per payload file and one for AppxManifest.xml; the block map and [Content_Types].xml are not listed. Name is the path as written, with \, not percent-encoded; Size is the plain size; LfhSize the local header's size (30 + the ZIP name's length). An empty file has no Block. [spec] [observed]
- One Block per 65536 bytes of plain data; Hash is the base64 SHA-256 of that plain block. [spec]
- A deflated file is one independent raw deflate part per block: each part decodes with a fresh decoder (nothing refers back into an earlier block) and ends with an empty stored block (00 00 FF FF), and after the last part comes an empty final block (03 00). The block's Size counts the bytes of its part, so the compressed size of the file is the sum of the Sizes plus 2 (an empty file: 2 bytes, no block). Stored files have no Size on their blocks. Incompressible data is deflated as well (into stored blocks). [observed]
- b4:FileHash appears only for files of more than one block. [observed]
- Windows' reader checks every block's hash as the file is read, and refuses a package whose file differs from its block map, both when reading and when installing. [observed]

39.3 [Content_Types].xml

One line: a Default per file extension in order of first use (lower case), xml as application/vnd.ms-appx.manifest+xml, and Override PartName="/AppxBlockMap.xml" as application/vnd.ms-appx.blockmap+xml. rubrapack adds an Override for each file without an extension, and one for /AppxManifest.xml when a payload .xml file took the xml default. [observed]

39.4 The manifest (AppxManifest.xml)

The smallest desktop application that Windows installs and starts (namespaces foundation/windows10, uap/windows10, restrictedcapabilities): [spec] [observed]

- Identity: Name (3-50 characters of A-Z a-z 0-9 . -), Publisher (the signing certificate's subject), Version (four parts, each at most 65535), ProcessorArchitecture (x64, arm64, x86).
- Properties: DisplayName, PublisherDisplayName, Logo (a PNG in the package).
- Dependencies/TargetDeviceFamily Name="Windows.Desktop" with MinVersion and MaxVersionTested.
- Resources/Resource Language.
- Application Id Executable EntryPoint="Windows.FullTrustApplication" with uap:VisualElements (DisplayName, Description, BackgroundColor, Square150x150Logo, Square44x44Logo), and the restricted capability runFullTrust.
- Paths in the manifest use \ and name files of the package.

39.5 The virtual registry (Registry.dat, User.dat)

Registry hives at the package root (the REGF format: Registry hive files (REGF)). Measured with a packaged app on Windows 11 (26100): [observed]

- Registry.dat: its REGISTRY\MACHINE\SOFTWARE key stands for HKLM\Software (the hive's root itself does not); REGISTRY\MACHINE\SOFTWARE\WOW6432Node\... is what the app sees in the 32-bit view.

Microsoft's description ("registry.dat serves as the logical equivalent of HKLM\Software") is about what the app sees, not about the hive's layout.

- User.dat: its root stands for HKCU (Software\... beneath it is HKCU\Software\...).
- The app reads the package's keys merged into the real registry; nothing is written to the machine's registry, and nothing is left after removal.
- Windows' packaging tools write these hives with the Offline Registry Library (its mark "OfRg" is in the base block).

39.6 The virtual file system (VFS\...)

Files under VFS\<folder> appear to the app at the real location; the real folder does not change. Measured: ProgramFilesX64 (%ProgramFiles%), SystemX64 (System32), Common AppData (%ProgramData%) - and none for AppData or Local AppData, as Microsoft Learn says. The other names rubrapack uses (ProgramFilesX86, ProgramFilesCommonX64/X86, SystemX86, Windows) are those Microsoft Learn lists. [spec] [observed]

39.7 Extensions

What rubrapack writes into an application's <Extensions> (after uap:VisualElements), and only this: a source feature with no row here is an error, and one whose Windows build is above the package's MinVersion asks for min-version to be raised. [spec] Microsoft Learn, "Integrate your desktop app with Windows using packaging extensions" and the element pages. The last column is what was checked by installing a package on Windows 11 (26100). [observed]

Source	Element (category)	Namespace	Min build	Capability	Checked on Windows 11
[assoc]	uap:Extension windows. fileTypeAssociation > uap3:FileTypeAssociation (Name, Parameters) > uap: DisplayName, uap: SupportedFileTypes > uap: FileType	uap, uap3	14393	runFullTrust	opening a .rpx/.rpy file starts the program with the file
[protocol]	uap3:Extension windows. protocol > uap3:Protocol (Name, Parameters)	uap3	14393	runFullTrust	starting a scheme: URI starts the program with it
[msix-extension] alias	uap3:Extension windows. AppExecutionAlias (Executable, EntryPoint) > uap3:AppExecutionAlias > desktop:ExecutionAlias (Alias)	uap3, desktop	14393	runFullTrust	the alias appears in % LOCALAPPDATA%\ Microsoft\WindowsApps and starts the program
[msix-extension] startup task	desktop:Extension windows.startupTask (Executable, EntryPoint) > desktop:StartupTask (TaskId, Enabled, DisplayName)	desktop	14393	runFullTrust	registered after the first start
[shortcut] Desktop	desktop7:Extension windows.shortcut > desktop7:Shortcut (File \$(Desktop)\<name>.lnk, Icon, Arguments, Description)	desktop7	19645	runFullTrust	the shortcut is on the user's desktop and starts the program

Source	Element (category)	Namespace	Min build	Capability	Checked on Windows 11
[shortcut] [Programs] StartMenu	one: the application's own Start entry	-	-	-	the Start menu lists the application
[font]	uap4:Extension windows. sharedFonts > uap4: SharedFonts > uap4:Font (File Fonts\<name>), in the first application	uap4	15063	-	other programs see the font while the package is installed, and not after

- The namespaces are declared on Package, and made ignorable, only when used, so a package without extensions keeps the manifest described above.
- Parameters, Arguments: literal text (MSI's [...] is refused); Windows puts the file or URI where %1 is.
- FileTypeAssociation Name: the prog-id in lower case; [assoc] tables that share a prog-id become one association with several FileTypes.
- Everything goes into the application whose executable the source names as target; a target that is not an application's executable is an error.

39.8 Bundles (.msixbundle)

A ZIP archive laid out like a package (ZIP64 entries with data descriptors), as Windows' bundle writer (IAppxBundleWriter) makes it: [observed]

- The packages first, stored (method 0) under their file names, in the order they were added; then AppxMetadata/AppxBundleManifest.xml, AppxBlockMap.xml and [Content_Types].xml, deflated.
- The block map lists the bundle manifest only (AppxMetadata\AppxBundleManifest.xml); the packages carry their own block maps.
- [Content_Types].xml: Default msix = application/vnd.ms-appx, Default xml = application/vnd.ms-appx.bundlemanifest+xml, and an Override for /AppxBlockMap.xml.
- The bundle manifest, CRLF line ends, tab indents, no line end after the last tag:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<Bundle xmlns="http://schemas.microsoft.com/appx/2013/bundle" SchemaVersion="5.0"
xmlns:b4="http://schemas.microsoft.com/appx/2018/bundle" xmlns:b5="http://schemas.microsoft.com/appx/2019/bundle" IgnorableNamespaces="b4 b5">
  <Identity Name="Example.App" Publisher="CN=Example" Version="1.0.0.0"/>
  <Packages>
    <Package Type="application" Version="1.0.0.0" Architecture="x64"
FileName="Example.App_1.0.0.0_x64.msix" Offset="66" Size="61340">
      <Resources>
        <Resource Language="en-US"/>
      </Resources>
      <b4:Dependencies>
        <b4:TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.17763.0"
MaxVersionTested="10.0.26100.0"/>
      </b4:Dependencies>
    </Package>
  </Packages>
</Bundle>
```

- Offset is where the package's bytes start in the bundle (after its local header), Size their length. Resources and Dependencies repeat the package manifest's Resource and TargetDeviceFamily elements.

- The bundle's Version is the writer's argument (a 64-bit number, 16 bits per part); rubrapack gives the packages' version. Every package must have the bundle's Name and Publisher, one version, and an architecture of its own.
- rubrapack's bundle of the same packages has the same entries at the same offsets and the same manifest, block map and content types, byte for byte; only the ZIP dates differ (1980 in rubrapack's). Windows installs from it the package for its own architecture: x64 on an x64 machine even when x86 and arm64 are there, x86 from a bundle of x86 alone. [observed]

39.9 Signatures (AppxSignature.p7x)

Worked out against Windows' signer (mssign32!SignerSignEx2 with APPX_SIP_CLIENT_DATA; the PowerShell cmdlet cannot sign a package) and checked by installing rubrapack's signed packages. [observed]

- Publisher in the manifest must be the signing certificate's subject written the way Windows displays it: the RDNs from last to first, A=value joined by , - a certificate made with /CN=Example/O=Example Ltd needs O=Example Ltd, CN=Example. Otherwise signing fails with 0x8007000B.
- Signing rewrites the archive the way the signer does: the payload's local records, the manifest and the block map stay as they were; [Content_Types].xml gets <Override PartName="/AppxSignature.p7x" ContentType="application/vnd.ms-appx.signature"/>; the signature is added last, deflated, with its sizes in the local header (version needed 2.0, no data descriptor); the central directory is written the ZIP32 way (no ZIP64 extra fields) where sizes and offsets allow, and the end record's disk numbers are 0.
- AppxSignature.p7x is PKCX followed by a CMS SignedData as in Authenticode signatures, with these differences: data is SEQUENCE { SpcSipInfo (1.3.6.1.4.1.311.2.1.30), SEQUENCE { INTEGER 0x01010000, OCTET STRING <SIP GUID>, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0 } } with the package SIP GUID bytes 4B DF C5 0A 07 CE E2 4D B7 6E 23 C8 39 A0 9F D1 or the bundle SIP GUID bytes B3 58 5F 0F DE AA 9A 4B A4 34 95 74 2D 92 EC EB; the signed attributes are contentType and messageDigest only.
- The DigestInfo's digest is not one hash but APPX followed by records of a 4-byte tag and a SHA-256:
 - AXPC: the archive from its start to the signature entry's local header;
 - AXCD: the central directory without the signature entry, then the ZIP64 end record, its locator and the end record, all as if the signature entry did not exist (the central directory starting where the signature's local header is). Windows reads the signature entry's place from its ZIP32 fields: a signature entry described the ZIP64 way (0xFFFFFFFF there) is a HashMismatch - so is the package whose central directory is rewritten that way after signing;
 - AXCT: [Content_Types].xml (the plain bytes); AXBM: AppxBlockMap.xml;
 - AXCI: AppxMetadata/CodeIntegrity.cat, only when the package has one.
- Windows' signer also adds AppxMetadata/CodeIntegrity.cat - a catalog of the package's program files, signed by the same key - when the package holds any. It is optional: a package signed without it installs and runs. rubrapack does not write one; it checks AXCI when it is there.
- A bundle: every package inside is signed first (each with its own AppxSignature.p7x), then the bundle is written around them and signed with the bundle SIP GUID, without AXCI.
- An unsigned package that needs -AllowUnsigned and the publisher OID (below) is not signed: signing refuses a publisher with that OID.

39.10 Installing an unsigned package

- Publisher must end in 01D.2.25.311729368913984317654407730594956997722=1; then Add-AppxPackage -AllowUnsigned installs it on Windows 11. [spec]
- A package with an executable needs an elevated process (otherwise 0x80073D2B: an unsigned package cannot hold an executable activation). Developer mode is not needed. [observed]

- Add-AppxPackage from a network logon (an SSH session) fails at "PLM initialization" with 0x80070005; it works in an interactive session. [observed]
- A package with file types leaves, after removal, an empty OpenWithProgids key under HKCU\Software\Classes\.<ext> - Windows' own doing, not the package's. [observed]

39.11 Worked example: the tutorial's hello.msix

The MSIX of tutorial chapter 17 (--unsigned-test, x64) is 11009 bytes. It begins with the local header of its first file, hello.exe:

Offset	Bytes	Field	Value
0x00	50 4b 03 04	signature	PK\3\4
0x04	2d 00	version needed	45 (4.5, ZIP64)
0x06	08 00	flags	0x0008: sizes follow the data
0x08	08 00	method	8 (deflate)
0x0A	00 00 21 00	time, date	1980-01-01 00:00
0x0E	00 00 00 00 00 00 00 00 00 00 00 00	CRC, sizes	0 (in the data descriptor)
0x1A	09 00 00 00	name, extra length	9, 0
0x1E	68 65 6c 6c 6f 2e 65 78 65	name	hello.exe

The 6706 deflated bytes of hello.exe follow, then its data descriptor: 50 4b 07 08 ac a3 c6 2e 32 1a 00 00 00 00 00 00 00 46 00 00 00 00 00 00 - PK\7\8, the CRC-32 2EC6A3AC, and the compressed and plain sizes as 8 bytes each (6706, 17920).

Its entry in AppxBlockMap.xml:

```
<File Name="hello.exe" Size="17920" LfhSize="39">
  <Block Hash="K7HbHzXLkhwTo3dcUr0vs35D1YX27qM0exD0TR8e2k=" Size="6704"/>
```

The file is smaller than 64 KiB, so it is one block. Its Hash is the base64 of the SHA-256 of the 17920 plain bytes (K7HbHzXLkhwTo3dcUr0vs35D1YX27qM0exD0TR8e2k= computed here), Size the 6704 bytes of its deflate part - the compressed size 6706 minus the 2-byte final block - and LfhSize the local header's 39 bytes (30 + the 9-byte name).

40 Registry hive files (REGF)

In this chapter

40.1	Layout	161
40.2	The base block	161
40.3	Keys (nk)	162
40.4	Subkey lists (lh, ri)	162
40.5	Values (vk) and value lists	162
40.6	The security cell (sk)	162
40.7	Reproducible hives	162

The file format of Registry.dat and User.dat in an MSIX package. Microsoft does not publish it; every fact here is **[observed]**: hives written by Windows' Offline Registry Library (offreg.dll, 10.0.26100, the writer Windows' packaging tools use - its mark is in the base block) for chosen key/value sets, and rubrapack's hives for the same sets read back on Windows with OROpenHive and RegLoadAppKey into exactly the keys and values Windows enumerates in its own. Only what a freshly written hive needs is described: no transaction logs, no volatile keys, no free-cell reuse.

40.1 Layout

- A 4096-byte base block, then hive bins. Offsets in cells ("cell offsets") count from the end of the base block (file offset - 4096); "none" is 0xFFFFFFFF.
- A bin: hbin, u32 its own cell offset, u32 its size (4096 or a multiple), 8 bytes zero, u64 time (0), u32 spare (0); 32 bytes in all. Cells follow; a cell that does not fit in the current bin starts a new bin sized to hold it; the rest of a bin is one free cell.
- A cell: i32 size (negative while in use, a multiple of 8, the size field included), then its content. A cell never crosses its bin's end.

40.2 The base block

Offset	Field	Value
0	signature	regf
4, 8	sequence numbers	1, 1 (unequal = not closed cleanly; readers refuse or replay a log)
12	u64 time	0 is accepted
20, 24	version	1, 5
28, 32	type, format	0, 1
36	root key cell offset	
40	size of all bins	
44	clustering	1
0xB0	offreg's mark	0fRg, then u32 1
508	checksum	XOR of the 127 dwords before it; 0 is written as 1, 0xFFFFFFFF as 0xFFFFFFFFE

Everything else is zero.

40.3 Keys (nk)

nk, u16 flags, u64 time, u32 access bits (0), u32 parent (none for the root), u32 subkey count, u32 volatile subkey count (0), u32 subkey list, u32 volatile list (none), u32 value count, u32 value list, u32 security cell, u32 class (none), u32 longest subkey name, u32 longest class (0), u32 longest value name, u32 largest value data, u32 work var (0), u16 name length, u16 class length (0), then the name.

- Flags: 0x20 when the name is stored as Latin-1 ("compressed": every UTF-16 unit below 256), else the name is UTF-16LE. The root key of an offreg hive is named ROOT and has flags 0x20 only.
- The longest names are counted in UTF-16 bytes, whatever the stored form.
- Names compare without case: each UTF-16 unit mapped by Unicode simple upper case.

40.4 Subkey lists (lh, ri)

- lh, u16 count, then per subkey: u32 key cell, u32 hash. Sorted by upper-cased name.
- Hash: $h = 0$; for each UTF-16 unit of the name, $h = h * 37 + \text{upper}(\text{unit})$ (32-bit wrap).
- More than 507 subkeys: an ri (u16 count, u32 per lh list) of lh lists of up to 507.

40.5 Values (vk) and value lists

- A value list is a plain cell of u32 value cell offsets, in the order the values were set.
- vk, u16 name length, u32 data size, u32 data offset, u32 type, u16 flags, u16 spare, then the name. Flag 1 = Latin-1 name; the unnamed default value has an empty name and flags 0.
- Data of up to 4 bytes (an empty value too) is kept in the data offset field, and the size's top bit is set.
- Up to 16344 bytes: one data cell. From 16345 bytes: a db cell (db, u16 segment count, u32 segment list), a list cell of u32 segment cells, and segments of 16344 bytes each - every segment cell allocated at full size, the last one too: Windows stops reading a value whose last segment cell is shorter.

40.6 The security cell (sk)

One sk cell shared by every key: sk, u16 0, u32 next and u32 previous (both itself), u32 reference count (the number of keys), u32 descriptor length, then a self-relative security descriptor. offreg's: owner and group Administrators; SYSTEM and Administrators full access, Everyone and RESTRICTED read, all container-inherit.

40.7 Reproducible hives

With every time field 0 and cells laid out depth first (a key; after the root key the security cell; the key's subkey lists, its value list, each value followed by its data; then its subkeys in sorted order), the same keys and values give the same bytes on any host. Windows reads such a hive without complaint.